

Gaussian Process Emulator for Planetary System Distributions via MFPCA

Yiming Chen

2026-04-30

```
library(tidyr)
library(dplyr)
library(ks)
library(MFPCA)
library(funData)
library(fda)
library(zoo)
library(MASS)
raw_data <- read.table("C:/Users/SOS/Desktop/Planet emulator/data/mockobserved.txt",
                      skip = 7, header = FALSE, sep = "", fill = TRUE)
colnames(raw_data) <- c("sigma_z", "period_ratio", "num_transiting",
                      "hill_spacing", "duration_ratio")

safe_log <- function(x) log(pmax(x, 1e-8))

clean_data <- raw_data %>%
  mutate(
    log_period = safe_log(period_ratio),
    log_duration = safe_log(duration_ratio),
    log_hill = safe_log(hill_spacing)
  ) %>%
  filter(across(c(log_period, log_duration, log_hill), ~ is.finite(.)))

# Filter sigmas with sufficient observations
sigma_counts <- table(clean_data$sigma_z)
valid_sigmas1 <- as.numeric(names(sigma_counts[sigma_counts >= 40]))
filtered_data <- clean_data %>% filter(sigma_z %in% valid_sigmas1)

# Additional filtering for unique values
sigma_summary <- filtered_data %>%
  group_by(sigma_z) %>%
  summarise(
    uniq_period = n_distinct(log_period),
    uniq_dur = n_distinct(log_duration),
    uniq_hill = n_distinct(log_hill),
    .groups = "drop"
  )
good_sigmas <- sigma_summary %>%
  filter(uniq_period >= 5, uniq_dur >= 5, uniq_hill >= 5) %>%
  pull(sigma_z)
```

```

valid_sigmas <- intersect(valid_sigmas1, good_sigmas)

cat("Number of valid sigma values:", length(valid_sigmas), "\n")

## Number of valid sigma values: 124

# =====
# 3. ADAPTIVE KDE & SMOOTHING (TO CREATE FUNCTIONAL DATA)
# =====
# Define Grids
x_range <- quantile(filtered_data$log_period, c(0.01, 0.99))
y_range <- quantile(filtered_data$log_duration, c(0.01, 0.99))
z_range <- quantile(filtered_data$log_hill, c(0.01, 0.99))

x_grid <- seq(x_range[1], x_range[2], length.out = 200)
y_grid <- seq(y_range[1], y_range[2], length.out = 200)
z_grid <- seq(z_range[1], z_range[2], length.out = 200)

# Adaptive KDE Function
adaptive_kde_1d <- function(data, grid, abram_exp, clip_low, clip_high) {
  data <- as.numeric(data)

  # Pilot bandwidth
  pilot_bw <- tryCatch({
    max(hpi(x = data) * 0.4, 0.01)
  }, error = function(e) {
    max(bw.nrd0(data) * 0.4, 0.01)
  })

  pilot_dens <- density(data, bw = pilot_bw, n = 512)
  interp_vals <- approx(pilot_dens$x, pilot_dens$y, xout = data, rule = 2)$y
  interp_vals[!is.finite(interp_vals) | interp_vals <= 0] <-
    min(interp_vals[interp_vals > 0], na.rm = TRUE)

  g <- exp(mean(log(interp_vals)))
  scaling_factors <- (interp_vals / g)^abram_exp
  scaling_factors <- pmin(pmax(scaling_factors, clip_low), clip_high)

  result <- numeric(length(grid))
  for (i in seq_along(data)) {
    bw_i <- pilot_bw * scaling_factors[i]
    result <- result + dnorm((grid - data[i]) / bw_i) / bw_i
  }
  result <- result / length(data)

  # Ensure non-negative and normalize
  result <- pmax(result, 1e-10)
  delta <- diff(range(grid)) / (length(grid) - 1)
  result / (sum(result) * delta)
}

# Generate Densities
dens_mat_x <- list()

```

```

dens_mat_y <- list()
dens_mat_z <- list()
sigma_list <- numeric(0)

cat("Computing KDE for each sigma...\n")

## Computing KDE for each sigma...

for (s in valid_sigmas) {
  group <- filtered_data %>% filter(sigma_z == s)
  dens_mat_x[[length(dens_mat_x) + 1]] <- adaptive_kde_1d(group$log_period, x_grid, 0.0, 1.0, 2.0)
  dens_mat_y[[length(dens_mat_y) + 1]] <- adaptive_kde_1d(group$log_duration, y_grid, -0.2, 0.6, 2.0)
  dens_mat_z[[length(dens_mat_z) + 1]] <- adaptive_kde_1d(group$log_hill, z_grid, -0.1, 1.2, 2.0)
  sigma_list <- c(sigma_list, s)
}
dens_mat_x <- do.call(rbind, dens_mat_x)
dens_mat_y <- do.call(rbind, dens_mat_y)
dens_mat_z <- do.call(rbind, dens_mat_z)

cat("Density matrices created. Dimensions:", nrow(dens_mat_x), "x", ncol(dens_mat_x), "\n")

## Density matrices created. Dimensions: 124 x 200

# =====
# DYNAMIC SUPPORT MASKING
# =====
# This function forces tails to be flat based on each sigma's actual data range.
# WHY: KDE can produce spurious density in regions with no data (extrapolation noise).
# By masking grid points outside each sigma's true data range, we eliminate this.
# =====

apply_dynamic_mask <- function(density_matrix, grid, raw_data_list, sigma_vals, buffer = 0.10) {
  # Make a copy of the matrix to modify
  masked_matrix <- density_matrix

  for (i in seq_along(sigma_vals)) {
    s <- sigma_vals[i]

    # Get raw data for this sigma (use character key for list lookup)
    raw_vals <- raw_data_list[[as.character(s)]]

    if (is.null(raw_vals) || length(raw_vals) == 0) next

    # Find the true data bounds for this sigma
    bounds <- range(raw_vals, na.rm = TRUE)
    range_width <- diff(bounds)

    # Define "Safe Zone" (Data Range + buffer%)
    lower_cutoff <- bounds[1] - buffer * range_width
    upper_cutoff <- bounds[2] + buffer * range_width

    # Identify grid points OUTSIDE this sigma's safe zone

```

```

    mask_indices <- which(grid < lower_cutoff | grid > upper_cutoff)

    # Force them to a near-zero density value before smoothing.
    masked_matrix[i, mask_indices] <- 1e-12
  }

  return(masked_matrix)
}

# Prepare raw data lists (split by sigma)
list_x <- split(filtered_data$log_period, filtered_data$sigma_z)
list_y <- split(filtered_data$log_duration, filtered_data$sigma_z)
list_z <- split(filtered_data$log_hill, filtered_data$sigma_z)

# Apply the mask (AFTER KDE, BEFORE smoothing)
cat("Applying dynamic support masking (10% buffer)...\n")

## Applying dynamic support masking (10% buffer)...

dens_mat_x_masked <- apply_dynamic_mask(dens_mat_x, x_grid, list_x, valid_sigmas, buffer = 0.10)
dens_mat_y_masked <- apply_dynamic_mask(dens_mat_y, y_grid, list_y, valid_sigmas, buffer = 0.10)
dens_mat_z_masked <- apply_dynamic_mask(dens_mat_z, z_grid, list_z, valid_sigmas, buffer = 0.10)

# =====
# DIRECT X/Z GRID REPRESENTATION
# =====
# X (Period) and Z (Hill spacing) enter MFPCA directly on their masked KDE
# grids. Geometry variables are NA placeholders because reconstruction decodes
# directly on the density grids.
# =====
x_direct_masked <- dens_mat_x_masked
z_direct_masked <- dens_mat_z_masked

peak_loc_x_all <- peak_height_x_all <- peak_width_x_all <-
  peak_left_width_x_all <- peak_right_width_x_all <- peak_skew_x_all <-
  rep(NA_real_, nrow(dens_mat_x))
peak_loc_z_all <- peak_height_z_all <- peak_width_z_all <-
  peak_left_width_z_all <- peak_right_width_z_all <- peak_skew_z_all <-
  rep(NA_real_, nrow(dens_mat_z))

ref_peak_x_loc <- ref_peak_x_width <- ref_peak_x_left_width <-
  ref_peak_x_right_width <- ref_peak_z_loc <- ref_peak_z_width <-
  ref_peak_z_left_width <- ref_peak_z_right_width <- NA_real_

cat("Using masked X/Z KDE grids directly.\n")

## Using masked X/Z KDE grids directly.

# =====
# B-Spline Smoothing
# =====
# SETTINGS:

```

```

# - nbasis: Controls flexibility. Higher = more wiggles possible.
# - lambda: Smoothing penalty. Higher = stiffer splines.
# =====
nbasis <- 80          # Higher flexibility to capture sharp multi-modal peaks
lambda <- 1e-5        # Lighter smoothing to preserve peak structure

cat("B-spline settings: nbasis =", nbasis, ", lambda =", lambda, "\n")

## B-spline settings: nbasis = 80 , lambda = 1e-05

x_basis <- create.bspline.basis(range(x_grid), nbasis)
y_basis <- create.bspline.basis(range(y_grid), nbasis)
z_basis <- create.bspline.basis(range(z_grid), nbasis)

# Smooth the MASKED density matrices
x_fd <- smooth.basis(x_grid, t(x_direct_masked), fdPar(x_basis, 2, lambda))$fd
y_fd <- smooth.basis(y_grid, t(dens_mat_y_masked), fdPar(y_basis, 2, lambda))$fd
z_fd <- smooth.basis(z_grid, t(z_direct_masked), fdPar(z_basis, 2, lambda))$fd

# =====
# 4. SQUARE-ROOT (HELLINGER) TRANSFORMATION & MFPCA
# =====
# Evaluate smoothed densities
x_smooth <- t(eval.fd(x_grid, x_fd))
y_smooth <- t(eval.fd(y_grid, y_fd))
z_smooth <- t(eval.fd(z_grid, z_fd))

# Ensure non-negative (smoothing can create small negatives)
x_smooth <- pmax(x_smooth, 0)
y_smooth <- pmax(y_smooth, 0)
z_smooth <- pmax(z_smooth, 0)

# Normalize each smoothed curve as a density before the Hellinger map.
normalize_density_matrix <- function(dens_matrix, grid) {
  delta <- diff(range(grid)) / (length(grid) - 1)
  t(apply(dens_matrix, 1, function(row) {
    row <- pmax(as.numeric(row), 0)
    mass <- sum(row) * delta
    if (!is.finite(mass) || mass <= 0) {
      row <- rep(1 / diff(range(grid)), length(grid))
      mass <- sum(row) * delta
    }
    row / mass
  })))
}

# Square-root density transform. Variable names retain the old x_clr/y_clr/z_clr
# convention so downstream GP code does not need a full rewrite.
sqrt_density_transform_matrix <- function(dens_matrix, grid) {
  sqrt(normalize_density_matrix(dens_matrix, grid))
}

x_clr <- sqrt_density_transform_matrix(x_smooth, x_grid)

```

```

y_clr <- sqrt_density_transform_matrix(y_smooth, y_grid)
z_clr <- sqrt_density_transform_matrix(z_smooth, z_grid)

# Create funData objects in square-root density space
multifd_sqrt <- multiFunData(
  funData(argvals = list(x_grid), X = x_clr),
  funData(argvals = list(y_grid), X = y_clr),
  funData(argvals = list(z_grid), X = z_clr)
)

# Run MFPCA
M_components <- 30 # Ensure we don't exceed available components
uniex <- list(
  list(type = "splines1Dpen", bs = "ps", m = 2, k = 75),
  list(type = "splines1Dpen", bs = "ps", m = 2, k = 50),
  list(type = "splines1Dpen", bs = "ps", m = 2, k = 75)
)

# Calculate weights (inverse variance)
calc_weights <- function(X) { 1 / max(mean(apply(X, 1, var)), 1e-10) }
weights <- c(calc_weights(x_clr), calc_weights(y_clr), calc_weights(z_clr))
weights <- weights / mean(weights)

cat("Running MFPCA with", M_components, "components...\n")

## Running MFPCA with 30 components...

set.seed(42)
mf pca_fit <- MFPCA(multifd_sqrt, M = M_components, uniExpansions = uniex, weights = weights)

# Extract Scores
scores <- as.data.frame(mf pca_fit$scores)
if (nrow(scores) != length(sigma_list)) {
  stop(sprintf("MFPCA score rows (%d) do not match sigma_list length (%d).",
    nrow(scores), length(sigma_list)))
}
if (ncol(scores) == 0) {
  stop("MFPCA returned zero score columns. Check mf pca_fit$scores.")
}
M_components <- ncol(scores)
ufpca_scores_long <- data.frame(theta = sigma_list, scores)
colnames(ufpca_scores_long)[-1] <- paste0("PC", seq_len(ncol(scores)))

ind_var_pct <- round(100 * mf pca_fit$values / sum(mf pca_fit$values), 2)
cat("MFPCA complete. Variance explained per component (top 5):",
  paste(ind_var_pct[1:min(5, M_components)], collapse = ", "), "%\n")

## MFPCA complete. Variance explained per component (top 5): 21.68, 7.39, 6.26, 5.23, 5.06 %

cat(" Cumulative variance (all", M_components, "components):",
  round(sum(ind_var_pct), 2), "%\n")

```

```
## Cumulative variance (all 30 components): 99.99 %
```

```
# =====  
# SIGN ALIGNMENT (GLOBAL, ONE DECISION PER PC)  
# =====  
# In a single MFPCA run the eigenfunction direction is fixed but arbitrary.  
# We make ONE global sign choice per PC so that positive scores correspond to  
# positive correlation with theta, giving a consistent physical interpretation.  
# This replaces the previous per-observation local-flip approach, which was  
# mathematically incorrect and was corrupting the score series.  
# =====  
  
cat("\n=== SIGN ALIGNMENT (Global per PC) ===\n")
```

```
##  
## === SIGN ALIGNMENT (Global per PC) ===
```

```
# Sort by theta for alignment and all downstream plotting  
sigma_order <- order(ufpca_scores_long$theta)  
ufpca_scores_sorted <- ufpca_scores_long[sigma_order, ]  
  
n_pcs <- ncol(scores)  
ufpca_scores_corrected <- ufpca_scores_sorted  
pc_signs <- rep(1, n_pcs)  
  
cat(sprintf("%-6s %12s %12s %s\n", "PC", "corr(sigma)", "acf(lag1)", "Action"))
```

```
## PC      corr(sigma)    acf(lag1) Action
```

```
cat(paste(rep("-", 48), collapse = ""), "\n")
```

```
## -----
```

```
for (pc in 1:n_pcs) {  
  pc_col <- paste0("PC", pc)  
  raw <- ufpca_scores_sorted[[pc_col]]  
  sigma_vec <- ufpca_scores_sorted$theta  
  
  # Global correlation with theta determines orientation convention  
  corr_sigma <- cor(raw, sigma_vec)  
  
  if (is.finite(corr_sigma) && corr_sigma < 0) {  
    ufpca_scores_corrected[[pc_col]] <- -raw  
    pc_signs[pc] <- -1  
    action <- "flipped"  
  } else {  
    action <- "kept"  
  }  
  
  aligned <- ufpca_scores_corrected[[pc_col]]  
  acf_val <- cor(aligned[-length(aligned)], aligned[-1])  
}
```

```

if (pc <= 10)
  cat(sprintf("PC%-4d %12.3f %12.3f %s\n", pc, corr_sigma, acf_val, action))
}

```

```

## PC1      -0.332      0.112 flipped
## PC2      -0.088      0.024 flipped
## PC3       0.046     -0.136 kept
## PC4       0.029      0.043 kept
## PC5       0.352     -0.003 kept
## PC6       0.067     -0.024 kept
## PC7       0.171      0.096 kept
## PC8       0.064     -0.023 kept
## PC9      -0.055     -0.084 flipped
## PC10     -0.168     -0.104 flipped

```

```

cat("\n")

```

```

# Restore original order after alignment
original_order <- order(sigma_order)
ufpca_scores_long <- ufpca_scores_corrected[original_order, ]

# Keep the decoder consistent with the sign-aligned scores used by the GP.
for (var_idx in seq_along(mfpca_fit$functions)) {
  if (nrow(mfpca_fit$functions[[var_idx]]@X) != n_pcs) {
    stop(sprintf("Cannot sign-align MFPCA functions for variable %d: expected %d rows, found %d",
      var_idx, n_pcs, nrow(mfpca_fit$functions[[var_idx]]@X)))
  }
  mfpca_fit$functions[[var_idx]]@X <- sweep(mfpca_fit$functions[[var_idx]]@X, 1, pc_signs, "*")
}
scores_aligned <- sweep(as.matrix(scores), 2, pc_signs, "*")
colnames(scores_aligned) <- paste0("PC", seq_len(n_pcs))
mfpca_fit$scores <- scores_aligned
scores <- as.data.frame(scores_aligned)

# =====
# SCORE NOISE ESTIMATION: LOESS residuals only
# =====
# Each KDE is estimated from ~20-50 observations, introducing substantial noise
# in the MFPCA scores. LOESS is retained only as an empirical residual estimator:
# sigma2_resid,k = Var(raw score_k - LOESS trend_k).
#
# The GP below is fit to LOESS-smoothed sign-aligned scores and receives
# sigma2_loess_se,k as a fixed observation-noise variance in the likelihood.
# =====

cat("Applying loess smoothing to PC scores (span = 0.25)...\n")

```

```

## Applying loess smoothing to PC scores (span = 0.25)...

```

```

ufpca_scores_smoothed <- ufpca_scores_long
theta_all <- ufpca_scores_long$theta

```



```

for (pc in 1:n_pcs) {
  pc_col <- paste0("PC", pc)
  lo      <- loess(ufpca_scores_long[[pc_col]] ~ theta_all, span = 0.25)
  ufpca_scores_smoothed[[pc_col]] <- fitted(lo)
}

# Autocorrelation report (sorted by theta)
sigma_order2 <- order(ufpca_scores_long$theta)
cat("\n=== Autocorrelation Check (Lag-1, sorted by theta) ===\n")

##
## === Autocorrelation Check (Lag-1, sorted by theta) ===

cat("Threshold: >0.5 [OK], 0.3-0.5 [WARN], <0.3 [BAD]\n")

## Threshold: >0.5 [OK], 0.3-0.5 [WARN], <0.3 [BAD]

cat(sprintf("%-6s %10s %10s\n", "PC", "Raw", "Smoothed"))

## PC          Raw    Smoothed

cat(paste(rep("-", 30), collapse = ""), "\n")

## -----

for (pc in 1:min(10, n_pcs)) {
  pc_col <- paste0("PC", pc)
  raw_s   <- ufpca_scores_long[sigma_order2, pc_col]
  smooth_s <- ufpca_scores_smoothed[sigma_order2, pc_col]

  acf_raw   <- cor(raw_s[-length(raw_s)], raw_s[-1])
  acf_smooth <- cor(smooth_s[-length(smooth_s)], smooth_s[-1])
  status     <- if (acf_smooth > 0.5) "[OK]" else if (acf_smooth > 0.3) "[WARN]" else "[BAD]"

  cat(sprintf("PC%-4d %10.3f %10.3f %s\n", pc, acf_raw, acf_smooth, status))
}

## PC1          0.112      0.982 [OK]
## PC2          0.024      0.936 [OK]
## PC3         -0.136      0.888 [OK]
## PC4          0.043      0.942 [OK]
## PC5         -0.003      0.983 [OK]
## PC6         -0.024      0.950 [OK]
## PC7          0.096      0.978 [OK]
## PC8         -0.023      0.974 [OK]
## PC9         -0.084      0.961 [OK]
## PC10        -0.104      0.963 [OK]

```

```

# Save raw (sign-corrected) scores for diagnostics and prediction-noise estimation.
ufpca_scores_raw_aligned <- ufpca_scores_long # sign-corrected, original order

# ---- KEY CHANGE (Apr29 fix): LOESS-smoothed scores are the GP targets ----
cat("\nLOESS-smoothed scores will be passed to GP fitting.\n")

```

```

##
## LOESS-smoothed scores will be passed to GP fitting.

```

```

cat("Raw-minus-smoothed residuals are restored at prediction time.\n")

```

```

## Raw-minus-smoothed residuals are restored at prediction time.

```

```

# Per-PC full residual variance:
# sigma2_resid_k = Var(raw score_k - LOESS trend_k)
# This captures the noise stripped by LOESS and is added back for prediction.
sigma2_resid_k <- sapply(1:n_pcs, function(pc) {
  pc_col <- paste0("PC", pc)
  var(ufpca_scores_raw_aligned[[pc_col]] - ufpca_scores_smoothed[[pc_col]],
      na.rm = TRUE)
})
sigma2_resid_k <- pmax(sigma2_resid_k, 1e-8)

# LOESS estimation uncertainty used as the fixed GP observation noise. The final
# n_eff is recomputed after the train/test split from n_train_theta.
LOESS_SPAN_SCORE <- 0.25
n_eff <- max(1L, floor(LOESS_SPAN_SCORE * nrow(ufpca_scores_raw_aligned)))
sigma2_loess_se_k <- pmax(sigma2_resid_k / n_eff, 1e-10)

cat("\nInitial noise decomposition from LOESS residuals:\n")

```

```

##
## Initial noise decomposition from LOESS residuals:

```

```

cat(sprintf(" Full residual var (PC1-5): %.4f %.4f %.4f %.4f %.4f\n",
  sigma2_resid_k[1], sigma2_resid_k[2], sigma2_resid_k[3],
  sigma2_resid_k[4], sigma2_resid_k[5]))

```

```

## Full residual var (PC1-5): 0.1666 0.0648 0.0521 0.0456 0.0391

```

```

cat(sprintf(" Provisional LOESS SE^2 (PC1-5): %.5f %.5f %.5f %.5f %.5f\n",
  sigma2_loess_se_k[1], sigma2_loess_se_k[2], sigma2_loess_se_k[3],
  sigma2_loess_se_k[4], sigma2_loess_se_k[5]))

```

```

## Provisional LOESS SE^2 (PC1-5): 0.00538 0.00209 0.00168 0.00147 0.00126

```

```

cat(sprintf(" provisional n_eff = %d\n", n_eff))

```

```

## provisional n_eff = 31

```

```
# Diagnostics retained for figures. These are not added post-hoc to GP draws.
```

```
pc_residual_var_global <- sigma2_resid_k  
pc_residual_sd <- sqrt(pc_residual_var_global)  
pc_local_resid_sd_mean <- pc_residual_sd
```

```
cat("\nFull residual SD summary")
```

```
##
```

```
## Full residual SD summary
```

```
cat(sprintf(" SD (PC1-5): %.3f %.3f %.3f %.3f %.3f\n",  
            pc_residual_sd[1], pc_residual_sd[2], pc_residual_sd[3],  
            pc_residual_sd[4], pc_residual_sd[5]))
```

```
## SD (PC1-5): 0.408 0.255 0.228 0.214 0.198
```

```
cat(sprintf(" Mean SD : %.3f Max SD: %.3f\n",  
            mean(pc_residual_sd), max(pc_residual_sd)))
```

```
## Mean SD : 0.153 Max SD: 0.408
```

```
cat(paste(rep("=", 70), collapse = ""), "\n\n")
```

```
## =====
```

```
# =====  
# MFPCA RECONSTRUCTION VALIDATION (BEFORE GP)  
# =====  
# This section checks how well MFPCA reconstructs the original densities  
# using the TRUE scores (not GP-predicted scores). This isolates MFPCA error  
# from GP prediction error.  
# =====
```

```
cat("\n=== Validating MFPCA Reconstruction (before GP) ===\n")
```

```
##
```

```
## === Validating MFPCA Reconstruction (before GP) ===
```

```
# Extract mean functions and eigenfunctions
```

```
mu_x_mfpca <- as.vector(mfpca_fit$meanFunction[[1]]@X)  
mu_y_mfpca <- as.vector(mfpca_fit$meanFunction[[2]]@X)  
mu_z_mfpca <- as.vector(mfpca_fit$meanFunction[[3]]@X)
```

```
phi_x_mfpca <- mfpca_fit$functions[[1]]@X
```

```
phi_y_mfpca <- mfpca_fit$functions[[2]]@X
```

```
phi_z_mfpca <- mfpca_fit$functions[[3]]@X
```

```
ind_var_explained <- 100 * mfpca_fit$values / sum(mfpca_fit$values)
```

```
# Inverse square-root density transform for reconstruction validation
```

```
inv_clr_validate <- function(sqrt_vec, grid) {
```

```

sqrt_vec <- pmax(as.numeric(sqrt_vec), 0)
dens_raw <- sqrt_vec^2
delta <- diff(range(grid)) / (length(grid) - 1)
mass <- sum(dens_raw) * delta
if (!is.finite(mass) || mass <= 0) {
  dens_raw <- rep(1 / diff(range(grid)), length(grid))
  mass <- sum(dens_raw) * delta
}
dens_raw / mass
}

# Reconstruct all observations using TRUE MFPCA scores
scores_true <- as.matrix(scores) # True MFPCA scores (n x K)

# Square-root density reconstruction: mean + scores %% eigenfunctions
recon_clr_x_true <- sweep(scores_true %% phi_x_mfpca, 2, mu_x_mfpca, "+")
recon_clr_y_true <- sweep(scores_true %% phi_y_mfpca, 2, mu_y_mfpca, "+")
recon_clr_z_true <- sweep(scores_true %% phi_z_mfpca, 2, mu_z_mfpca, "+")

# Convert square-root density functions back to densities on the original X/Y/Z grids
recon_dens_x_true <- t(apply(recon_clr_x_true, 1, inv_clr_validate, grid = x_grid))
recon_dens_y_true <- t(apply(recon_clr_y_true, 1, inv_clr_validate, grid = y_grid))
recon_dens_z_true <- t(apply(recon_clr_z_true, 1, inv_clr_validate, grid = z_grid))

# Compute ISE for MFPCA reconstruction
compute_ISE_mfpca <- function(f1, f2, grid) {
  delta <- diff(range(grid)) / (length(grid) - 1)
  sum((f1 - f2)^2) * delta
}

ise_mfpca_x <- sapply(1:nrow(dens_mat_x), function(i)
  compute_ISE_mfpca(recon_dens_x_true[i,], dens_mat_x[i,], x_grid))
ise_mfpca_y <- sapply(1:nrow(dens_mat_y), function(i)
  compute_ISE_mfpca(recon_dens_y_true[i,], dens_mat_y[i,], y_grid))
ise_mfpca_z <- sapply(1:nrow(dens_mat_z), function(i)
  compute_ISE_mfpca(recon_dens_z_true[i,], dens_mat_z[i,], z_grid))
ise_x_raw_space <- sapply(seq_len(nrow(dens_mat_x)), function(i)
  compute_ISE_mfpca(recon_dens_x_true[i, ], dens_mat_x[i, ], x_grid))
ise_x_final_density <- sapply(seq_len(nrow(dens_mat_x)), function(i)
  compute_ISE_mfpca(recon_dens_x_true[i, ], dens_mat_x[i, ], x_grid))
ise_z_raw_space <- sapply(seq_len(nrow(dens_mat_z)), function(i)
  compute_ISE_mfpca(recon_dens_z_true[i, ], dens_mat_z[i, ], z_grid))
ise_z_final_density <- sapply(seq_len(nrow(dens_mat_z)), function(i)
  compute_ISE_mfpca(recon_dens_z_true[i, ], dens_mat_z[i, ], z_grid))

cat("\nMFPCA Reconstruction Error (Mean ISE, using TRUE scores):\n")

##
## MFPCA Reconstruction Error (Mean ISE, using TRUE scores):

cat(" X (Period): ", sprintf("%.6f", mean(ise_mfpca_x)), "\n")

## X (Period): 1.412789

```

```

cat("  Y (Duration): ", sprintf("%.6f", mean(ise_mfpca_y)), "\n")

##  Y (Duration):  0.091281

cat("  Z (Hill):      ", sprintf("%.6f", mean(ise_mfpca_z)), "\n")

##  Z (Hill):      0.799931

cat("  X raw-space ISE:      ", sprintf("%.6f", mean(ise_x_raw_space)), "\n")

##  X raw-space ISE:      1.412789

cat("  X final density ISE: ", sprintf("%.6f", mean(ise_x_final_density)), "\n")

##  X final density ISE:  1.412789

cat("  Z raw-space ISE:      ", sprintf("%.6f", mean(ise_z_raw_space)), "\n")

##  Z raw-space ISE:      0.799931

cat("  Z final density ISE: ", sprintf("%.6f", mean(ise_z_final_density)), "\n")

##  Z final density ISE:  0.799931

# =====
# 6. INPUT WARPING GP (BETA CDF TRANSFORMATION)
# =====
# THE ROOT SOLUTION: Instead of fighting the GP to handle sharp peaks and flat
# trends simultaneously, we WARP the x-axis.
#
# CONCEPT:
# - Stretch regions where data is dense (the peaks)
# - Squash regions where data is sparse (flat regions)
# - After warping, the sharp "spike" becomes a nice wide "hill"
# - A standard, stable GP can then fit it perfectly!
#
# METHOD: Empirical CDF Warping
# - warp(x) = F(x) = proportion of training points
# - Dense regions
# - Sparse regions
# - Result: uniform distribution in warped space
#
# ADVANTAGE: No local windows, no manual constraints, no hacks!
# =====

# --- Matern 3/2 Covariance (allows sharp features) ---
matern32_cov <- function(X1, X2, lengthscale, sigma2) {
  X1 <- as.numeric(X1)
  X2 <- as.numeric(X2)

```

```

n1 <- length(X1)
n2 <- length(X2)

ell <- lengthscale
r <- outer(X1/ell, X2/ell, function(a, b) abs(a - b))
sqrt3_r <- sqrt(3) * r
K <- sigma2 * (1 + sqrt3_r) * exp(-sqrt3_r)

matrix(K, nrow = n1, ncol = n2)
}

# =====
# Input Warping Functions
# =====

# Create empirical CDF warping function from training data
# stretch_factor > 1: spreads data MORE (amplifies warping)
# stretch_factor = 1: standard ECDF (default, already uniform)
# stretch_factor < 1: reduces warping effect
create_ecdf_warper <- function(X_train, stretch_factor = 1.0) {
  X_train <- as.numeric(X_train)
  X_sorted <- sort(X_train)
  n <- length(X_sorted)

  # Empirical CDF: F(x) = proportion of points
  # Use linear interpolation for smooth warping
  list(
    X_sorted = X_sorted,
    n = n,
    stretch_factor = stretch_factor,
    # Warp function: x
    warp = function(x) {
      # For each x, find proportion of training points
      # Using approx for smooth interpolation
      ecdf_vals <- (1:n) / n
      warped <- approx(X_sorted, ecdf_vals, xout = x, rule = 2, ties = mean)$y

      # Apply stretch factor via power transform centered at 0.5
      # stretch_factor > 1: pushes values away from 0.5 (more spread)
      # stretch_factor < 1: pulls values toward 0.5 (less spread)
      if (stretch_factor != 1.0) {
        # Transform to [-1, 1], apply signed power, transform back
        centered <- 2 * warped - 1 # [0,1] -> [-1,1]
        sign_centered <- sign(centered)
        stretched <- sign_centered * abs(centered)^(1/stretch_factor)
        warped <- (stretched + 1) / 2 # [-1,1] -> [0,1]
      }
    },
    warped
  ),
  # Store range for reference
  x_min = min(X_sorted),
  x_max = max(X_sorted)
}

```

```

)
}

# =====
# Warped known-noise GP used by the current hybrid method
# =====
# Fits PC score targets with fixed known observation noise:
#  $y_j = f(u_j) + \text{eps}_j$ ,  $\text{eps}_j \sim N(0, \text{obs\_noise\_var})$ 
#
# obs_noise_var is fixed by the caller. In the main Apr29 fixed pipeline this is
# sigma2_loess_se_k, not the full residual variance. The optimizer only
# estimates the latent output scale and lengthscale; beta is profiled by GLS.
# Predictive variance is the latent posterior  $\text{Var}[f(\text{theta}^*)]$  unless
# add_observation_noise = TRUE is requested.
# =====

bounded_logit <- function(x, lower, upper, eps = 1e-6) {
  z <- (x - lower) / (upper - lower)
  qlogis(pmin(pmax(z, eps), 1 - eps))
}

fit_known_noise_warped_gp <- function(X_train, y_train, warper,
                                     obs_noise_var,
                                     min_ls = 0.08,
                                     max_ls = 1.0,
                                     kernel = "matern32",
                                     mean_type = "linear",
                                     standardize_y = TRUE,
                                     n_restarts = 7L) {
  X_train <- as.numeric(X_train)
  y_train <- as.numeric(y_train)
  n <- length(X_train)
  if (length(obs_noise_var) == 1L) {
    obs_noise_var <- rep(obs_noise_var, n)
  }
  obs_noise_var <- pmax(as.numeric(obs_noise_var), 1e-10)

  X_warped <- warper$warp(X_train)

  if (standardize_y) {
    y_mean <- mean(y_train)
    y_sd <- sd(y_train)
    if (y_sd < 1e-10) y_sd <- 1
    y_std <- (y_train - y_mean) / y_sd
  } else {
    y_mean <- 0
    y_sd <- 1
    y_std <- y_train
  }

  obs_noise_var_std <- pmax(obs_noise_var / (y_sd^2), 1e-10)

  cov_func <- matern32_cov

```

```

H <- cbind(1, X_warped)
y_var <- var(y_std)
if (!is.finite(y_var) || y_var < 1e-10) y_var <- 1

gp_negloglik <- function(params) {
  ls <- min_ls + (max_ls - min_ls) / (1 + exp(-params[1]))
  s2 <- exp(params[2])

  K <- cov_func(X_warped, X_warped, ls, s2) + diag(obs_noise_var_std, n)
  L <- tryCatch(chol(K), error = function(e) {
    tryCatch(chol(K + 1e-6 * diag(n)), error = function(e) NULL)
  })
  if (is.null(L)) return(1e10)

  Kinv_H <- backsolve(L, forwardsolve(t(L), H))
  Kinv_y <- backsolve(L, forwardsolve(t(L), y_std))

  beta <- tryCatch(
    solve(crossprod(H, Kinv_H) + 1e-8 * diag(ncol(H)), crossprod(H, Kinv_y)),
    error = function(e) c(mean(y_std), 0)
  )

  resid <- y_std - H %*% beta
  alpha <- backsolve(L, forwardsolve(t(L), resid))
  nll <- 0.5 * sum(resid * alpha) + sum(log(diag(L))) + 0.5 * n * log(2 * pi)

  if (!is.finite(nll)) 1e10 else nll
}

ls_starts <- unique(c(
  min_ls + 0.10 * (max_ls - min_ls),
  min_ls + 0.25 * (max_ls - min_ls),
  min_ls + 0.50 * (max_ls - min_ls),
  min_ls + 0.75 * (max_ls - min_ls),
  min_ls + 0.90 * (max_ls - min_ls)
))
s2_starts <- unique(pmax(c(0.25, 0.5, 1.0, 2.0, 4.0) * y_var, 1e-4))
start_grid <- expand.grid(ls = ls_starts, s2 = s2_starts)
start_grid$score <- abs(start_grid$ls - median(ls_starts)) +
  abs(log(start_grid$s2) - log(y_var))
start_grid <- start_grid[order(start_grid$score), , drop = FALSE]
start_grid <- head(start_grid, n_restarts)

opt_list <- vector("list", nrow(start_grid))
for (i in seq_len(nrow(start_grid))) {
  init_params <- c(
    bounded_logit(start_grid$ls[i], min_ls, max_ls),
    log(start_grid$s2[i])
  )
  opt_list[[i]] <- tryCatch(
    optim(init_params, fn = gp_negloglik, method = "L-BFGS-B",
      control = list(maxit = 300)),
    error = function(e) list(par = init_params, value = Inf, convergence = 1)
  )
}

```



```

    )
  }
  opt_values <- sapply(opt_list, function(o) if (!is.null(o$value)) o$value else Inf)
  opt <- opt_list[[which.min(opt_values)]]

  final_ls <- min_ls + (max_ls - min_ls) / (1 + exp(-opt$par[1]))
  final_s2 <- exp(opt$par[2])

  K_final <- cov_func(X_warped, X_warped, final_ls, final_s2) +
    diag(obs_noise_var_std, n)
  L_final <- tryCatch(chol(K_final), error = function(e) {
    chol(K_final + 1e-6 * diag(n))
  })

  Kinv_H <- backsolve(L_final, forwardsolve(t(L_final), H))
  Kinv_y <- backsolve(L_final, forwardsolve(t(L_final), y_std))
  beta_final <- as.numeric(
    solve(crossprod(H, Kinv_H) + 1e-8 * diag(ncol(H)), crossprod(H, Kinv_y))
  )

  resid_final <- y_std - H %*% beta_final
  alpha_final <- as.numeric(backsolve(L_final, forwardsolve(t(L_final), resid_final)))

  list(
    X_train = X_train,
    X_warped = X_warped,
    y_train = y_train,
    y_std = y_std,
    y_mean = y_mean,
    y_sd = y_sd,
    n_train = n,
    warper = warper,
    lengthscale = final_ls,
    sigma2 = final_s2,
    obs_noise_var = obs_noise_var,
    obs_noise_var_std = obs_noise_var_std,
    kernel = kernel,
    mean_type = mean_type,
    beta = beta_final,
    H = H,
    L = L_final,
    alpha = alpha_final,
    nll = opt$value,
    n_restarts = nrow(start_grid),
    model_type = "KNOWN_NOISE_GP"
  )
}

predict_known_noise_warped_gp <- function(model, X_new, return_var = FALSE,
                                           add_observation_noise = FALSE) {
  X_new <- as.numeric(X_new)
  n_new <- length(X_new)
  X_new_warped <- model$warper$warp(X_new)

```

```

cov_func <- matern32_cov

H_new <- cbind(1, X_new_warped)
mean_linear <- as.numeric(H_new %*% model$beta)
K_star <- cov_func(model$X_warped, X_new_warped, model$lengthscale, model$sigma2)
mu_gp <- as.numeric(crossprod(K_star, model$alpha))
mu_std <- mean_linear + mu_gp
mu <- mu_std * model$y_sd + model$y_mean

if (return_var) {
  K_ss_diag <- rep(model$sigma2, n_new)
  v <- forwardsolve(t(model$L), K_star)
  var_gp_std <- pmax(K_ss_diag - colSums(v^2), 0)
  var_gp <- var_gp_std * (model$y_sd^2)
  obs_var <- rep(mean(model$obs_noise_var), n_new)
  var_out <- var_gp + if (add_observation_noise) obs_var else 0

  return(list(
    mean = mu,
    var = var_out,
    var_gp = var_gp,
    var_noise = obs_var
  ))
}

mu
}

# =====
# Train/Test Split: HOLD OUT ENTIRE THETA VALUES
# =====
# Each theta value has one KDE-derived density curve and one MFPCA score vector.
# The GP is fit to the LOESS-smoothed theta-level score vector, with fixed
# per-component LOESS estimation uncertainty as the observation noise.
# =====

set.seed(123)
theta_list <- ufPCA_scores_long$theta
n_theta_unique <- length(theta_list)
n_test_theta <- max(2, round(0.2 * n_theta_unique))
n_train_theta <- n_theta_unique - n_test_theta

# Hold out entire theta values (evenly spaced across the range for good coverage)
test_theta_idx <- round(seq(2, n_theta_unique - 1, length.out = n_test_theta))
train_theta_idx <- setdiff(seq_len(n_theta_unique), test_theta_idx)

train_theta_vals <- theta_list[train_theta_idx]
test_theta_vals <- theta_list[test_theta_idx]

# One row per theta throughout
pc_cols <- paste0("PC", 1:M_components)
train_scores_raw <- ufPCA_scores_raw_aligned[train_theta_idx, pc_cols, drop = FALSE]
test_scores_raw <- ufPCA_scores_raw_aligned[test_theta_idx, pc_cols, drop = FALSE]

```

```

train_scores <- ufpca_scores_smoothed[train_theta_idx, pc_cols, drop = FALSE]
test_scores <- ufpca_scores_smoothed[test_theta_idx, pc_cols, drop = FALSE]
train_scores_mean <- train_scores
test_scores_mean <- test_scores
train_indices <- train_theta_idx
test_indices <- test_theta_idx

```

```

# Final LOESS SE2 nugget uses the actual training-set size.

```

```

n_eff <- max(1L, floor(LOESS_SPAN_SCORE * n_train_theta))
sigma2_loess_se_k <- pmax(sigma2_resid_k / n_eff, 1e-10)

```

```

cat("\n=== THETA-LEVEL TRAIN/TEST SPLIT ===\n")

```

```

##

```

```

## === THETA-LEVEL TRAIN/TEST SPLIT ===

```

```

cat("Goal: predict density at unseen theta values\n")

```

```

## Goal: predict density at unseen theta values

```

```

cat(" - Total theta values:      ", n_theta_unique, "\n")

```

```

## - Total theta values:      124

```

```

cat(" - Training theta values:    ", n_train_theta, "\n")

```

```

## - Training theta values:    99

```

```

cat(" - Test theta values:       ", n_test_theta, "\n")

```

```

## - Test theta values:       25

```

```

cat(" - Train theta range: [", round(min(train_theta_vals), 2), ",",
    round(max(train_theta_vals), 2), "]\n")

```

```

## - Train theta range: [ 23.39 , 175.88 ]

```

```

cat(" - Test theta values:", paste(round(test_theta_vals, 2), collapse = ", "), "\n")

```

```

## - Test theta values: 26.7, 32.73, 44.28, 48.2, 50.74, 55.65, 59.18, 63.26, 65.87, 70.37, 72.8, 75.1

```

```

cat(" - GP emulates the LOESS-smoothed 30D PC-score vector\n")

```

```

## - GP emulates the LOESS-smoothed 30D PC-score vector

```

```

cat(" - GP fixed noise is LOESS SE^2; full residual variance is restored for prediction\n")

## - GP fixed noise is LOESS SE^2; full residual variance is restored for prediction

cat("\nNoise decomposition used by the GP and posterior sampler:\n")

##
## Noise decomposition used by the GP and posterior sampler:

cat(sprintf(" Full residual var (PC1-5): %.4f %.4f %.4f %.4f %.4f\n",
            sigma2_resid_k[1], sigma2_resid_k[2], sigma2_resid_k[3],
            sigma2_resid_k[4], sigma2_resid_k[5]))

## Full residual var (PC1-5): 0.1666 0.0648 0.0521 0.0456 0.0391

cat(sprintf(" LOESS SE^2 nugget (PC1-5): %.5f %.5f %.5f %.5f %.5f\n",
            sigma2_loess_se_k[1], sigma2_loess_se_k[2], sigma2_loess_se_k[3],
            sigma2_loess_se_k[4], sigma2_loess_se_k[5]))

## LOESS SE^2 nugget (PC1-5): 0.00694 0.00270 0.00217 0.00190 0.00163

cat(sprintf(" n_eff = %d, variance ratio = %.1f\n",
            n_eff, mean(sigma2_resid_k / sigma2_loess_se_k)))

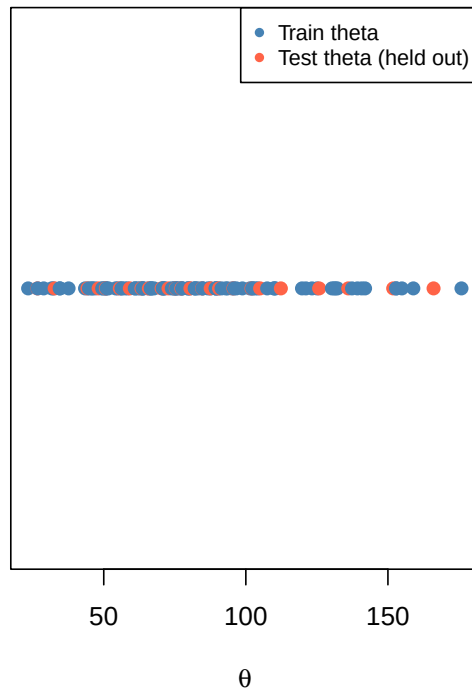
## n_eff = 24, variance ratio = 24.0

# Visualise the split
par(mfrow = c(1, 2), mar = c(4, 4, 3, 1))
plot(theta_list, rep(1, n_theta_unique), pch = 16, cex = 1.2,
     col = ifelse(seq_len(n_theta_unique) %in% train_theta_idx, "steelblue", "tomato"),
     yaxt = "n", ylim = c(0.5, 1.5),
     main = "Train / Test Theta Split",
     xlab = expression(theta), ylab = "")
legend("topright", c("Train theta", "Test theta (held out)"),
     pch = 16, col = c("steelblue", "tomato"), cex = 0.8)

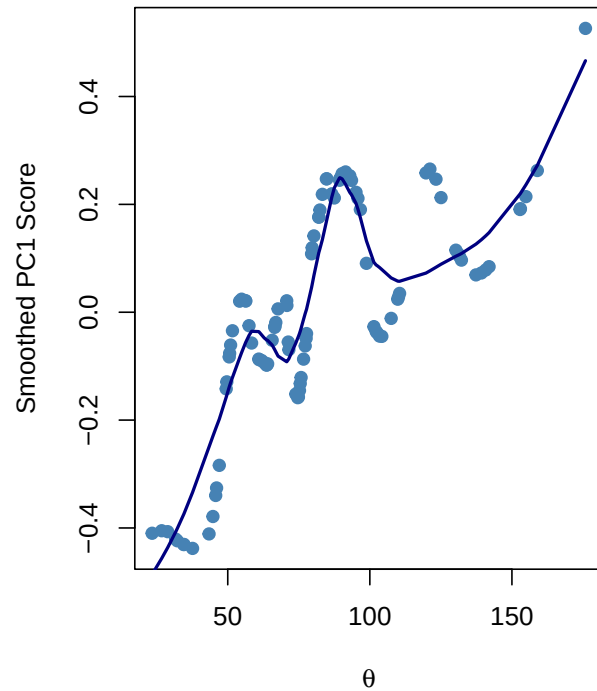
# Plot of PC1 smoothed training scores by theta
pc1_train <- train_scores[, "PC1"]
plot(train_theta_vals, pc1_train, pch = 19, col = "steelblue",
     main = "PC1 LOESS-Smoothed Score vs Theta (training)",
     xlab = expression(theta), ylab = "Smoothed PC1 Score")
lo <- loess(pc1_train ~ train_theta_vals, span = 0.5)
x_s <- sort(train_theta_vals)
lines(x_s, predict(lo, newdata = data.frame(train_theta_vals = x_s)),
     col = "navy", lwd = 2)

```

Train / Test Theta Split



PC1 LOESS-Smoothed Score vs Theta (tra



```
cat("\n[OK] Theta-level split complete\n")
```

```
##
## [OK] Theta-level split complete
```

```
cat(paste(rep("=", 70), collapse = ""), "\n\n")
```

```
## =====
```

Pre-GP check: raw and LOESS-smoothed MFPCA scores

```
# =====
# PRE-GP DIAGNOSTIC: Evaluate LOESS-smoothed MFPCA score targets
# =====
# The GP below is trained on ufPCA_scores_smoothed. This chunk checks what that
# smoothing step is doing before the GP sees the scores:
# 1. raw-vs-smoothed agreement by PC,
# 2. residual variance restored at prediction time,
# 3. train/test raw-vs-smoothed RMSE,
# 4. visual overlays of raw MFPCA scores, LOESS-smoothed scores, and residuals.
# =====

safe_cor <- function(a, b) {
  a <- as.numeric(a)
  b <- as.numeric(b)
```

```

ok <- is.finite(a) & is.finite(b)
if (sum(ok) < 3 || sd(a[ok]) < 1e-12 || sd(b[ok]) < 1e-12) {
  return(NA_real_)
}
cor(a[ok], b[ok])
}

score_smoothing_summary <- do.call(rbind, lapply(seq_along(pc_cols), function(j) {
  pc_col <- pc_cols[j]

  raw_all <- as.numeric(ufpca_scores_raw_aligned[[pc_col]])
  smooth_all <- as.numeric(ufpca_scores_smoothed[[pc_col]])
  resid_all <- raw_all - smooth_all

  raw_var <- var(raw_all, na.rm = TRUE)
  smooth_var <- var(smooth_all, na.rm = TRUE)
  resid_var <- var(resid_all, na.rm = TRUE)

  data.frame(
    PC = pc_col,
    raw_var = raw_var,
    smooth_var = smooth_var,
    resid_var = resid_var,
    resid_sd = sqrt(pmax(resid_var, 0)),
    loess_se2 = sigma2_loess_se_k[j],
    raw_smooth_cor = safe_cor(raw_all, smooth_all),
    smooth_var_share = smooth_var / pmax(raw_var, 1e-12),
    resid_var_share = resid_var / pmax(raw_var, 1e-12),
    train_rmse_raw_smooth = sqrt(mean((as.numeric(train_scores_raw[[pc_col]]) -
                                         as.numeric(train_scores_smoothed[[pc_col]]))^2,
                                         na.rm = TRUE)),
    test_rmse_raw_smooth = sqrt(mean((as.numeric(test_scores_raw[[pc_col]]) -
                                         as.numeric(test_scores_smoothed[[pc_col]]))^2,
                                         na.rm = TRUE)),
    stringsAsFactors = FALSE
  )
}))

score_smoothing_print <- score_smoothing_summary
num_cols <- vapply(score_smoothing_print, is.numeric, logical(1))
score_smoothing_print[num_cols] <- lapply(score_smoothing_print[num_cols],
                                          function(x) round(x, 4))

cat("\n=== PRE-GP SMOOTHED MFPCA SCORE DIAGNOSTIC ===\n")

##
## === PRE-GP SMOOTHED MFPCA SCORE DIAGNOSTIC ===

cat("GP target: LOESS-smoothed MFPCA scores\n")

## GP target: LOESS-smoothed MFPCA scores

```

```
cat("Raw scores remain the target for residual/noise accounting.\n\n")
```

```
## Raw scores remain the target for residual/noise accounting.
```

```
print(head(score_smoothing_print, 10), row.names = FALSE)
```

```
##      PC raw_var smooth_var resid_var resid_sd loess_se2 raw_smooth_cor
## PC1  0.2149    0.0418    0.1666    0.4082    0.0069         0.4750
## PC2  0.0733    0.0068    0.0648    0.2546    0.0027         0.3410
## PC3  0.0620    0.0078    0.0521    0.2282    0.0022         0.4030
## PC4  0.0519    0.0049    0.0456    0.2135    0.0019         0.3500
## PC5  0.0501    0.0101    0.0391    0.1978    0.0016         0.4689
## PC6  0.0497    0.0050    0.0422    0.2054    0.0018         0.3974
## PC7  0.0385    0.0062    0.0316    0.1777    0.0013         0.4250
## PC8  0.0379    0.0052    0.0325    0.1803    0.0014         0.3765
## PC9  0.0324    0.0025    0.0295    0.1717    0.0012         0.3005
## PC10 0.0316    0.0019    0.0293    0.1712    0.0012         0.2718
## smooth_var_share resid_var_share train_rmse_raw_smooth test_rmse_raw_smooth
##          0.1947          0.7755          0.4054          0.4112
##          0.0926          0.8851          0.2554          0.2462
##          0.1260          0.8399          0.2348          0.1947
##          0.0951          0.8792          0.2113          0.2181
##          0.2022          0.7805          0.1965          0.1990
##          0.0998          0.8487          0.2104          0.1795
##          0.1607          0.8200          0.1795          0.1667
##          0.1364          0.8583          0.1786          0.1833
##          0.0772          0.9102          0.1701          0.1748
##          0.0615          0.9267          0.1736          0.1578
```

```
cat("\nSummary over PCs used by the GP:\n")
```

```
##
## Summary over PCs used by the GP:
```

```
cat(sprintf(" Mean raw-smoothed correlation: %.3f\n",
            mean(score_smoothing_summary$raw_smooth_cor, na.rm = TRUE)))
```

```
## Mean raw-smoothed correlation: 0.379
```

```
cat(sprintf(" Mean smooth variance share: %.3f\n",
            mean(score_smoothing_summary$smooth_var_share, na.rm = TRUE)))
```

```
## Mean smooth variance share: 0.124
```

```
cat(sprintf(" Mean residual variance share: %.3f\n",
            mean(score_smoothing_summary$resid_var_share, na.rm = TRUE)))
```

```
## Mean residual variance share: 0.852
```

```
cat(sprintf(" Mean train raw-smooth RMSE:   %.4f\n",
            mean(score_smoothing_summary$train_rmse_raw_smooth, na.rm = TRUE)))
```

```
## Mean train raw-smooth RMSE:   0.1537
```

```
cat(sprintf(" Mean test raw-smooth RMSE:    %.4f\n",
            mean(score_smoothing_summary$test_rmse_raw_smooth, na.rm = TRUE)))
```

```
## Mean test raw-smooth RMSE:    0.1464
```

```
cat(sprintf(" Mean full residual variance:  %.5f\n",
            mean(score_smoothing_summary$resid_var, na.rm = TRUE)))
```

```
## Mean full residual variance:  0.02764
```

```
cat(sprintf(" Mean LOESS SE^2 nugget:       %.5f\n",
            mean(score_smoothing_summary$loess_se2, na.rm = TRUE)))
```

```
## Mean LOESS SE^2 nugget:       0.00115
```

```
score_order <- order(theta_list)
theta_sorted <- theta_list[score_order]
pc_plot_set <- seq_len(min(6, length(pc_cols)))

plot_raw_smoothed_scores <- function(pc_set, main_title) {
  n_plot <- length(pc_set)
  old_par <- par(no.readonly = TRUE)
  on.exit(par(old_par), add = TRUE)
  par(mfrow = c(ceiling(n_plot / 3), min(3, n_plot)),
      mar = c(4.0, 4.2, 3.0, 1.0),
      oma = c(0, 0, 3, 0))

  for (idx in seq_along(pc_set)) {
    pc <- pc_set[idx]
    pc_col <- paste0("PC", pc)
    raw_s <- as.numeric(ufpca_scores_raw_aligned[score_order, pc_col])
    smooth_s <- as.numeric(ufpca_scores_smoothed[score_order, pc_col])

    plot(theta_sorted, raw_s,
         type = "b", pch = 16, cex = 0.45, lwd = 0.8,
         col = adjustcolor("gray35", alpha.f = 0.55),
         main = paste0(pc_col, ": raw vs LOESS-smoothed"),
         xlab = expression(theta), ylab = "MFPCA score")
    lines(theta_sorted, smooth_s, col = "steelblue4", lwd = 2.3)
    points(test_theta_vals, ufpca_scores_raw_aligned[test_theta_idx, pc_col],
           pch = 21, bg = "tomato", col = "white", cex = 1.0)
    points(test_theta_vals, ufpca_scores_smoothed[test_theta_idx, pc_col],
           pch = 24, bg = "navy", col = "white", cex = 1.0)
    abline(h = 0, col = "gray80", lty = 2)
```



```

usr <- par("usr")
text(usr[1] + 0.03 * diff(usr[1:2]),
     usr[4] - 0.10 * diff(usr[3:4]),
     labels = sprintf("corr=%.2f | resid share=%.2f",
                      score_smoothing_summary$raw_smooth_cor[pc],
                      score_smoothing_summary$resid_var_share[pc]),
     adj = c(0, 0), cex = 0.78)

if (idx == 1) {
  legend("bottomright",
        c("Raw MFPCA score", "LOESS-smoothed score",
          "Held-out raw", "Held-out smoothed"),
        col = c(adjustcolor("gray35", alpha.f = 0.55), "steelblue4",
                "white", "white"),
        pt.bg = c(NA, NA, "tomato", "navy"),
        pch = c(16, NA, 21, 24),
        lty = c(1, 1, NA, NA),
        lwd = c(0.8, 2.3, NA, NA),
        cex = 0.72, bty = "n")
}
}
mtext(main_title, outer = TRUE, line = 1.0, cex = 1.15, font = 2)
}

plot_score_residuals <- function(pc_set, main_title) {
  n_plot <- length(pc_set)
  old_par <- par(no.readonly = TRUE)
  on.exit(par(old_par), add = TRUE)
  par(mfrow = c(ceiling(n_plot / 3), min(3, n_plot)),
      mar = c(4.0, 4.2, 3.0, 1.0),
      oma = c(0, 0, 3, 0))

  for (idx in seq_along(pc_set)) {
    pc <- pc_set[idx]
    pc_col <- paste0("PC", pc)
    raw_s <- as.numeric(ufpca_scores_raw_aligned[score_order, pc_col])
    smooth_s <- as.numeric(ufpca_scores_smoothed[score_order, pc_col])
    resid_s <- raw_s - smooth_s
    resid_sd <- sqrt(sigma2_resid_k[pc])

    plot(theta_sorted, resid_s,
         type = "h", lwd = 1.1,
         col = adjustcolor("firebrick3", alpha.f = 0.65),
         main = paste0(pc_col, ": raw - smoothed residual"),
         xlab = expression(theta), ylab = "Residual score")
    points(theta_sorted, resid_s, pch = 16, cex = 0.35,
           col = adjustcolor("firebrick4", alpha.f = 0.65))
    abline(h = 0, col = "gray35", lty = 1)
    abline(h = c(-2, 2) * resid_sd, col = "gray55", lty = 3)

    usr <- par("usr")
    text(usr[1] + 0.03 * diff(usr[1:2]),
         usr[4] - 0.10 * diff(usr[3:4]),

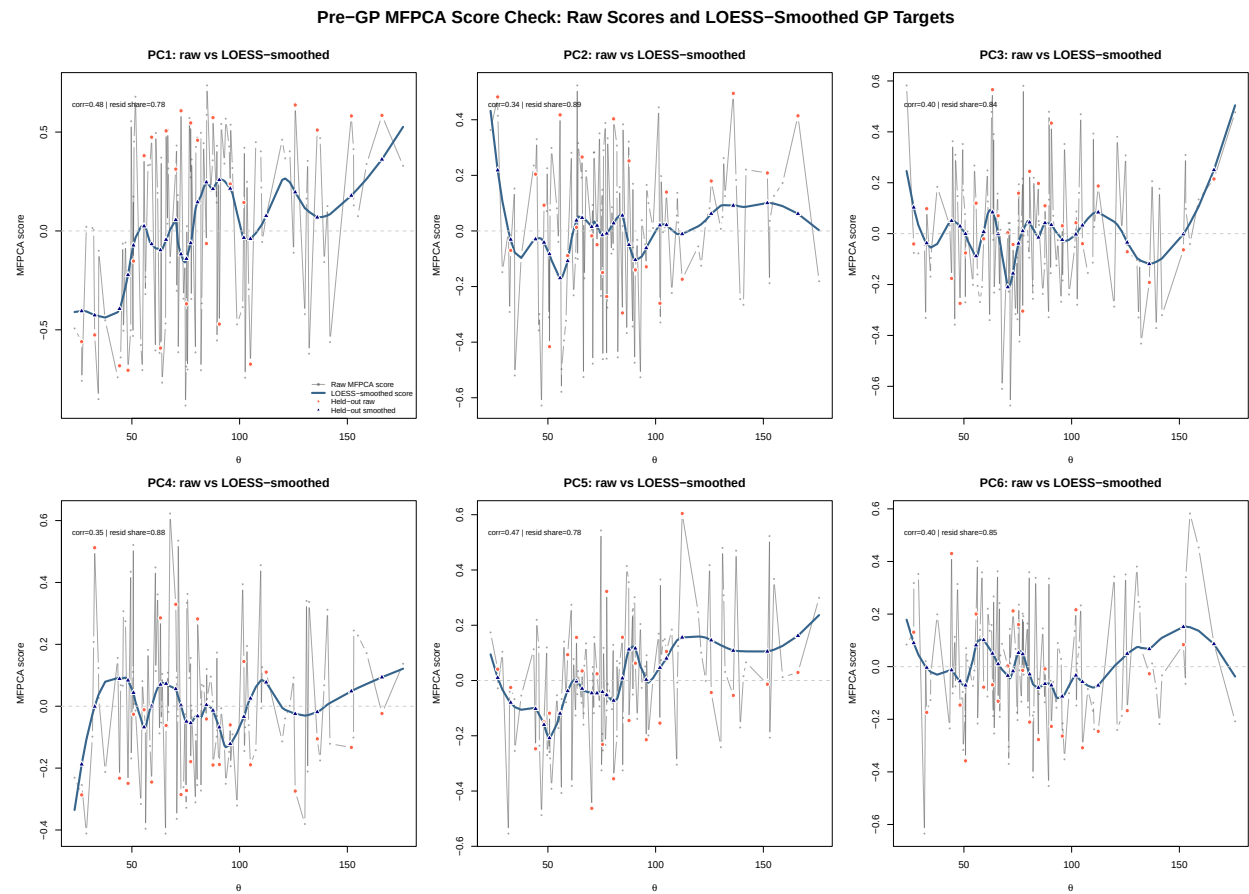
```

```

    labels = sprintf("Var(resid)=%.4f | SE^2=%.5f",
                      sigma2_resid_k[pc], sigma2_loess_se_k[pc]),
    adj = c(0, 0), cex = 0.78)
}
mtext(main_title, outer = TRUE, line = 1.0, cex = 1.15, font = 2)
}

plot_raw_smoothed_scores(
  pc_plot_set,
  "Pre-GP MFPCA Score Check: Raw Scores and LOESS-Smoothed GP Targets"
)

```

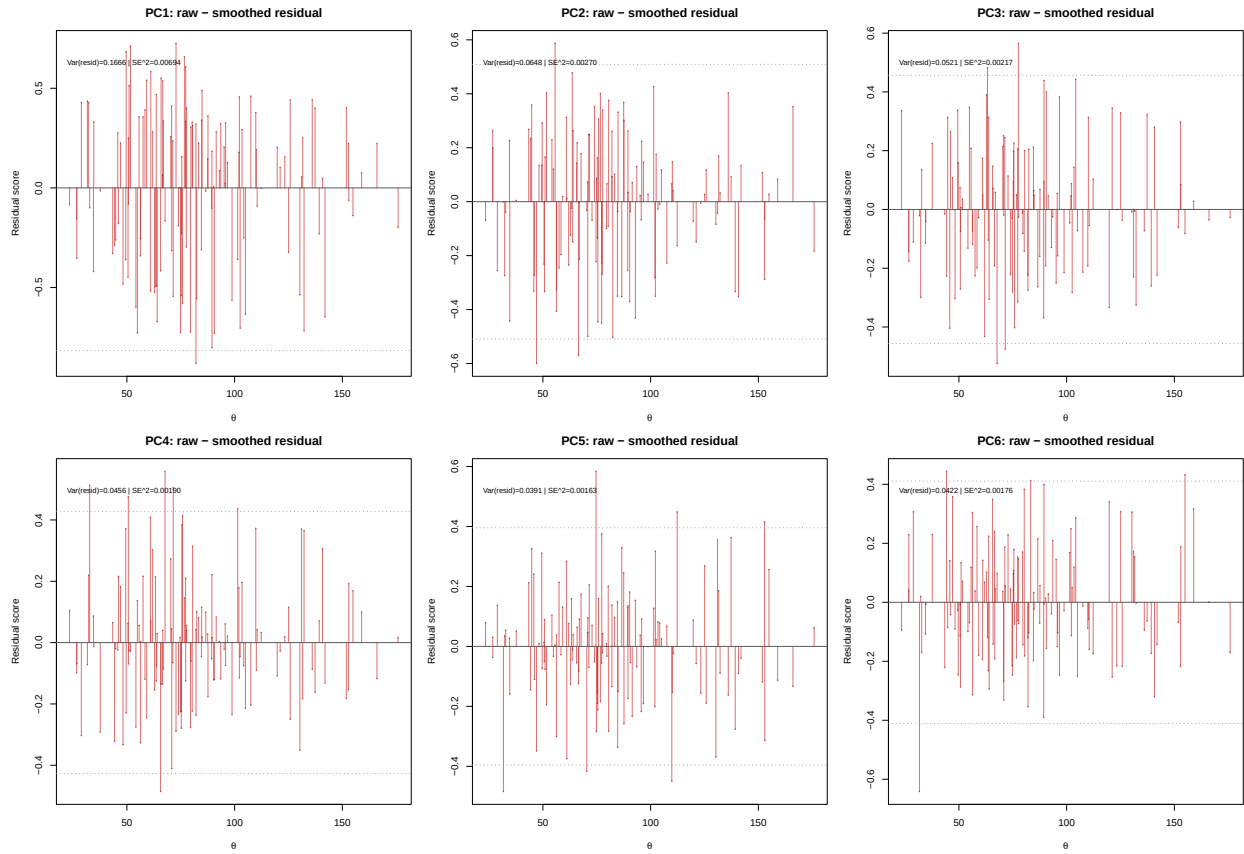


```

plot_score_residuais(
  pc_plot_set,
  "Pre-GP Noise Check: Residuals Restored in Posterior Score Variance"
)

```

Pre-GP Noise Check: Residuals Restored in Posterior Score Variance



Pre-GP check: density reconstruction from raw vs smoothed scores

```
# =====
# PRE-GP DIAGNOSTIC: Density reconstruction from raw vs LOESS-smoothed scores
# =====
# This chunk compares three density curves at representative theta values:
# 1. raw KDE target,
# 2. MFPCA reconstruction using raw sign-aligned MFPCA scores,
# 3. MFPCA reconstruction using LOESS-smoothed MFPCA scores.
#
# The goal is to isolate the effect of score smoothing before the GP is fit.
# =====

reconstruct_density_from_scores <- function(score_mat, mu_vec, phi_mat, grid) {
  sqrt_mat <- sweep(score_mat %*% phi_mat, 2, mu_vec, "+")
  t(apply(sqrt_mat, 1, inv_clr_validate, grid = grid))
}

scores_raw_for_recon <- as.matrix(ufpca_scores_raw_aligned[, pc_cols, drop = FALSE])
scores_smooth_for_recon <- as.matrix(ufpca_scores_smoothed[, pc_cols, drop = FALSE])

recon_dens_x_raw_scores <- reconstruct_density_from_scores(
  scores_raw_for_recon, mu_x_mfpca, phi_x_mfpca, x_grid
```

```

)
recon_dens_y_raw_scores <- reconstruct_density_from_scores(
  scores_raw_for_recon, mu_y_mfpca, phi_y_mfpca, y_grid
)
recon_dens_z_raw_scores <- reconstruct_density_from_scores(
  scores_raw_for_recon, mu_z_mfpca, phi_z_mfpca, z_grid
)

recon_dens_x_smoothed_scores <- reconstruct_density_from_scores(
  scores_smooth_for_recon, mu_x_mfpca, phi_x_mfpca, x_grid
)
recon_dens_y_smoothed_scores <- reconstruct_density_from_scores(
  scores_smooth_for_recon, mu_y_mfpca, phi_y_mfpca, y_grid
)
recon_dens_z_smoothed_scores <- reconstruct_density_from_scores(
  scores_smooth_for_recon, mu_z_mfpca, phi_z_mfpca, z_grid
)

ise_raw_score_recon_x <- sapply(seq_len(nrow(dens_mat_x)), function(i)
  compute_ISE_mfpca(recon_dens_x_raw_scores[i, ], dens_mat_x[i, ], x_grid))
ise_raw_score_recon_y <- sapply(seq_len(nrow(dens_mat_y)), function(i)
  compute_ISE_mfpca(recon_dens_y_raw_scores[i, ], dens_mat_y[i, ], y_grid))
ise_raw_score_recon_z <- sapply(seq_len(nrow(dens_mat_z)), function(i)
  compute_ISE_mfpca(recon_dens_z_raw_scores[i, ], dens_mat_z[i, ], z_grid))

ise_smoothed_score_recon_x <- sapply(seq_len(nrow(dens_mat_x)), function(i)
  compute_ISE_mfpca(recon_dens_x_smoothed_scores[i, ], dens_mat_x[i, ], x_grid))
ise_smoothed_score_recon_y <- sapply(seq_len(nrow(dens_mat_y)), function(i)
  compute_ISE_mfpca(recon_dens_y_smoothed_scores[i, ], dens_mat_y[i, ], y_grid))
ise_smoothed_score_recon_z <- sapply(seq_len(nrow(dens_mat_z)), function(i)
  compute_ISE_mfpca(recon_dens_z_smoothed_scores[i, ], dens_mat_z[i, ], z_grid))

ise_raw_vs_smoothed_x <- sapply(seq_len(nrow(dens_mat_x)), function(i)
  compute_ISE_mfpca(recon_dens_x_raw_scores[i, ],
    recon_dens_x_smoothed_scores[i, ], x_grid))
ise_raw_vs_smoothed_y <- sapply(seq_len(nrow(dens_mat_y)), function(i)
  compute_ISE_mfpca(recon_dens_y_raw_scores[i, ],
    recon_dens_y_smoothed_scores[i, ], y_grid))
ise_raw_vs_smoothed_z <- sapply(seq_len(nrow(dens_mat_z)), function(i)
  compute_ISE_mfpca(recon_dens_z_raw_scores[i, ],
    recon_dens_z_smoothed_scores[i, ], z_grid))

recon_score_smoothing_summary <- data.frame(
  Variable = c("X", "Y", "Z"),
  RawScore_vs_KDE_ISE = c(mean(ise_raw_score_recon_x),
    mean(ise_raw_score_recon_y),
    mean(ise_raw_score_recon_z)),
  SmoothedScore_vs_KDE_ISE = c(mean(ise_smoothed_score_recon_x),
    mean(ise_smoothed_score_recon_y),
    mean(ise_smoothed_score_recon_z)),
  Smoothed_minus_Raw_ISE = c(mean(ise_smoothed_score_recon_x - ise_raw_score_recon_x),
    mean(ise_smoothed_score_recon_y - ise_raw_score_recon_y),
    mean(ise_smoothed_score_recon_z - ise_raw_score_recon_z)),

```

```

RawScore_vs_SmoothedScore_ISE = c(mean(ise_raw_vs_smoothed_x),
                                   mean(ise_raw_vs_smoothed_y),
                                   mean(ise_raw_vs_smoothed_z)),
stringsAsFactors = FALSE
)

recon_score_smoothing_print <- recon_score_smoothing_summary
num_cols <- vapply(recon_score_smoothing_print, is.numeric, logical(1))
recon_score_smoothing_print[num_cols] <- lapply(recon_score_smoothing_print[num_cols],
                                              function(x) round(x, 5))

cat("\n=== PRE-GP DENSITY RECONSTRUCTION: RAW SCORES VS SMOOTHED SCORES ===\n")

##
## === PRE-GP DENSITY RECONSTRUCTION: RAW SCORES VS SMOOTHED SCORES ===

cat("Each row compares density reconstructions before GP fitting.\n")

## Each row compares density reconstructions before GP fitting.

print(recon_score_smoothing_print, row.names = FALSE)

## Variable RawScore_vs_KDE_ISE SmoothedScore_vs_KDE_ISE Smoothed_minus_Raw_ISE
##      X      1.41279      3.75386      2.34107
##      Y      0.09128      0.25790      0.16662
##      Z      0.79993      3.01599      2.21606
## RawScore_vs_SmoothedScore_ISE
##      1.87784
##      0.16848
##      1.73442

cat("\nInterpretation:\n")

##
## Interpretation:

cat(" - RawScore_vs_KDE_ISE is the ordinary MFPCA reconstruction error.\n")

## - RawScore_vs_KDE_ISE is the ordinary MFPCA reconstruction error.

cat(" - SmoothedScore_vs_KDE_ISE shows what LOESS score smoothing changes before GP.\n")

## - SmoothedScore_vs_KDE_ISE shows what LOESS score smoothing changes before GP.

cat(" - RawScore_vs_SmoothedScore_ISE is the density-space size of the smoothing effect.\n\n")

## - RawScore_vs_SmoothedScore_ISE is the density-space size of the smoothing effect.

```

```

theta_order_recon <- order(theta_list)
n_theta_panels <- min(4L, length(theta_order_recon))
theta_panel_idx <- theta_order_recon[round(seq(1, length(theta_order_recon),
                                             length.out = n_theta_panels))]

plot_density_recon_panel <- function(grid, kde, raw_recon, smooth_recon,
                                     main_title, show_legend = FALSE,
                                     ylab = "Density") {
  y_max <- max(c(kde, raw_recon, smooth_recon), na.rm = TRUE)
  plot(grid, kde, type = "l", col = "black", lwd = 2.6,
       ylim = c(0, 1.08 * y_max),
       main = main_title, xlab = "", ylab = ylab)
  lines(grid, raw_recon, col = "steelblue4", lwd = 2.1)
  lines(grid, smooth_recon, col = "darkorange3", lwd = 2.1, lty = 2)
  if (show_legend) {
    legend("topright",
          c("Raw KDE target", "Raw-score MFPCA recon",
            "Smoothed-score recon"),
          col = c("black", "steelblue4", "darkorange3"),
          lwd = c(2.6, 2.1, 2.1), lty = c(1, 1, 2),
          cex = 0.72, bty = "n")
  }
}

old_par <- par(no.readonly = TRUE)
par(mfrow = c(length(theta_panel_idx), 3),
    mar = c(3.2, 4.0, 2.4, 0.8),
    oma = c(2.0, 0, 3.2, 0))

for (row_i in seq_along(theta_panel_idx)) {
  i <- theta_panel_idx[row_i]
  theta_label <- sprintf("theta = %.3f", theta_list[i])

  plot_density_recon_panel(
    x_grid, dens_mat_x[i, ],
    recon_dens_x_raw_scores[i, ], recon_dens_x_smoothed_scores[i, ],
    paste("X |", theta_label),
    show_legend = row_i == 1L,
    ylab = "Density"
  )

  plot_density_recon_panel(
    y_grid, dens_mat_y[i, ],
    recon_dens_y_raw_scores[i, ], recon_dens_y_smoothed_scores[i, ],
    paste("Y |", theta_label),
    show_legend = FALSE,
    ylab = ""
  )

  plot_density_recon_panel(
    z_grid, dens_mat_z[i, ],
    recon_dens_z_raw_scores[i, ], recon_dens_z_smoothed_scores[i, ],
    paste("Z |", theta_label),

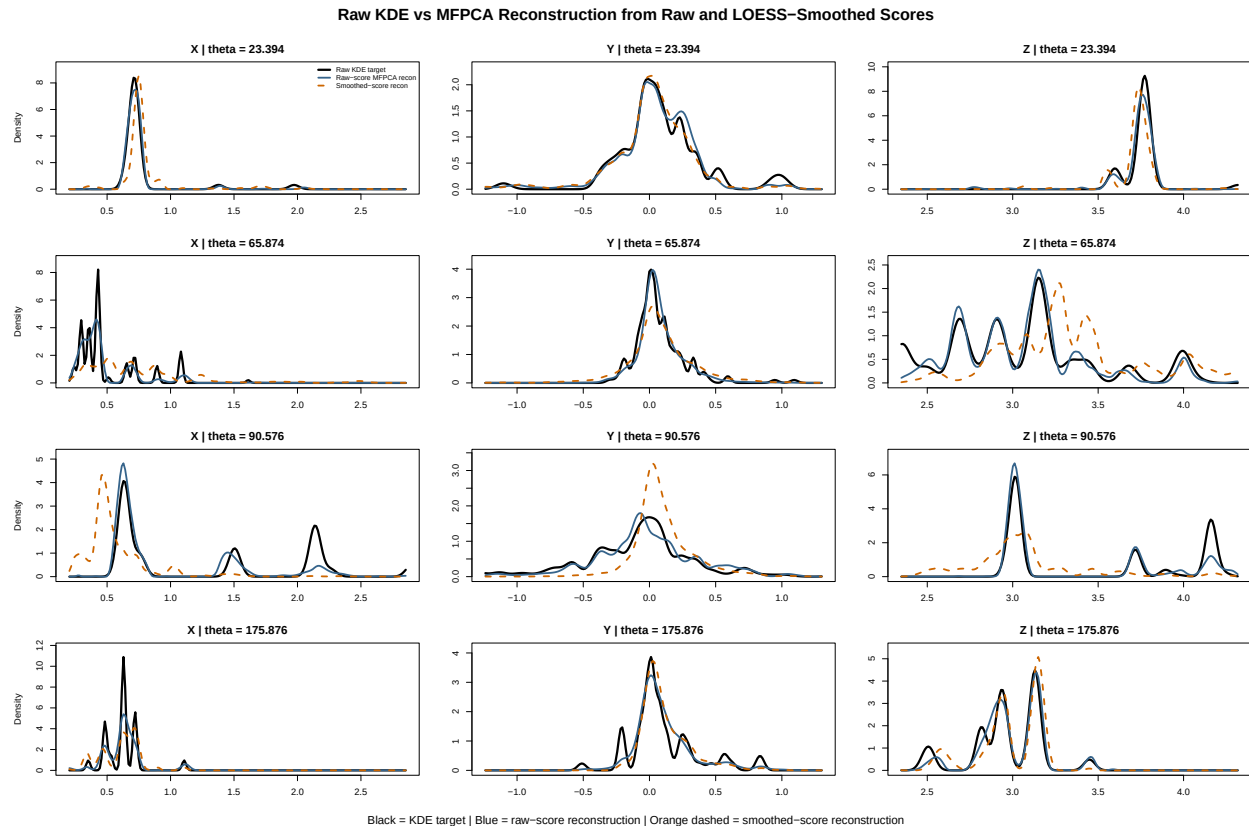
```

```

    show_legend = FALSE,
    ylab = ""
  )
}

mtext("Raw KDE vs MFPCA Reconstruction from Raw and LOESS-Smoothed Scores",
      outer = TRUE, line = 1.2, cex = 1.18, font = 2)
mtext("Black = KDE target | Blue = raw-score reconstruction | Orange dashed = smoothed-score reconstruction",
      outer = TRUE, side = 1, line = 0.4, cex = 0.85)

```



```

par(old_par)

old_par <- par(no.readonly = TRUE)
par(mfrow = c(1, 3), mar = c(4.2, 4.3, 3.1, 1.0), oma = c(0, 0, 2.4, 0))

plot_recon_ise_by_theta <- function(theta_vals, raw_ise, smooth_ise,
                                     main_title, ylab = "ISE vs KDE") {
  ord <- order(theta_vals)
  y_max <- max(c(raw_ise, smooth_ise), na.rm = TRUE)
  plot(theta_vals[ord], raw_ise[ord], type = "b",
        pch = 16, cex = 0.55, lwd = 1.2,
        col = "steelblue4", ylim = c(0, 1.08 * y_max),
        main = main_title, xlab = expression(theta), ylab = ylab)
  lines(theta_vals[ord], smooth_ise[ord], type = "b",
        pch = 17, cex = 0.55, lwd = 1.2,
        col = "darkorange3", lty = 2)
}

```

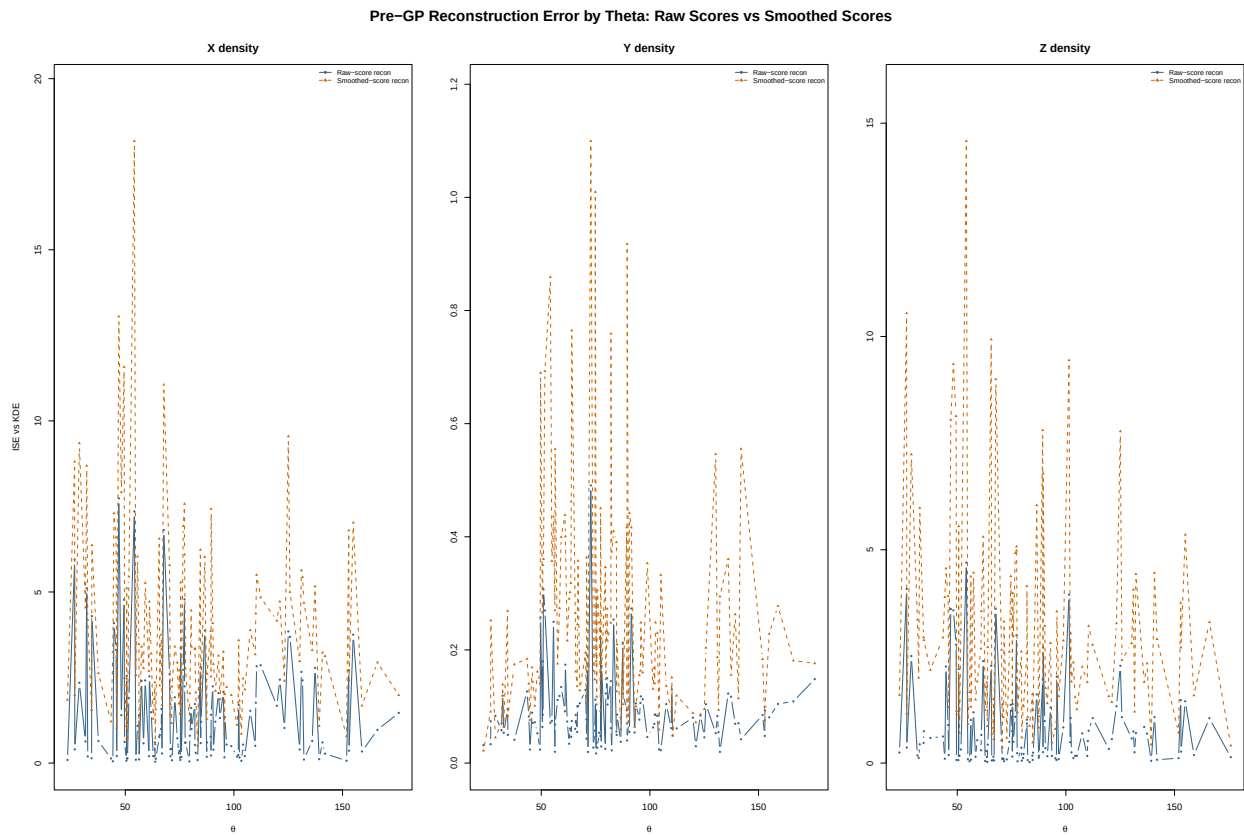
```

legend("topright",
      c("Raw-score recon", "Smoothed-score recon"),
      col = c("steelblue4", "darkorange3"),
      pch = c(16, 17), lty = c(1, 2), lwd = 1.2,
      cex = 0.72, bty = "n")
}

plot_recon_ise_by_theta(theta_list, ise_raw_score_recon_x,
                        ise_smoothed_score_recon_x, "X density")
plot_recon_ise_by_theta(theta_list, ise_raw_score_recon_y,
                        ise_smoothed_score_recon_y, "Y density", ylab = "")
plot_recon_ise_by_theta(theta_list, ise_raw_score_recon_z,
                        ise_smoothed_score_recon_z, "Z density", ylab = "")

mtext("Pre-GP Reconstruction Error by Theta: Raw Scores vs Smoothed Scores",
      outer = TRUE, line = 0.7, cex = 1.12, font = 2)

```



```
par(old_par)
```

```

# =====
# GP INPUT WARPING AND TRAINING: CURRENT METHOD ONLY
# =====
# Current model:
# 1. Fit one GP per PC on LOESS-smoothed score targets.
# 2. Use fixed known observation noise sigma2_loess_se_k in the GP likelihood.

```



```

# 3. Restore the full residual variance sigma2_resid_k later for score sampling.
# =====

# Scaled inputs for GP (one value per training/test theta)
x_center <- mean(train_theta_vals)
x_scale <- sd(train_theta_vals)
if (x_scale < 1e-10) x_scale <- 1

train_theta_raw <- train_theta_vals
test_theta_raw <- test_theta_vals
train_theta_norm <- (train_theta_raw - x_center) / x_scale
test_theta_norm <- (test_theta_raw - x_center) / x_scale

# ECDF input warping, matching the current paper method.
STRETCH_FACTOR <- 1.0
input_warper <- create_ecdf_warper(train_theta_norm, stretch_factor = STRETCH_FACTOR)
cat("Using Empirical CDF warping (stretch_factor=", STRETCH_FACTOR, ")\n", sep = "")

## Using Empirical CDF warping (stretch_factor=1)

# GP settings in warped [0,1] space.
MIN_LS <- 0.08
MAX_LS <- 1.00
KERNEL <- "matern32"
MEAN_TYPE <- "linear"
PREDICT_OBSERVATION_NOISE <- FALSE

cat("\n", paste(rep("=", 70), collapse = ""), "\n", sep = "")

##
## =====

cat("HYBRID LOESS-SMOOTHED KNOWN-NOISE GP TRAINING\n")

## HYBRID LOESS-SMOOTHED KNOWN-NOISE GP TRAINING

cat(paste(rep("=", 70), collapse = ""), "\n")

## =====

cat("Strategy: ECDF-warp theta, fit LOESS-smoothed PC scores, fixed LOESS-SE noise\n")

## Strategy: ECDF-warp theta, fit LOESS-smoothed PC scores, fixed LOESS-SE noise

cat(" - Kernel:", KERNEL, "\n")

## - Kernel: matern32

```

```

cat(" - Mean function:", MEAN_TYPE, "\n")

## - Mean function: linear

cat(" - Lengthscale bounds: [", MIN_LS, ", ", MAX_LS, "]\n", sep = "")

## - Lengthscale bounds: [0.08, 1]

cat(sprintf(" - LOESS SE^2 nugget (PC1-5): %s\n",
  paste(sprintf("%.5f", sigma2_loess_se_k[1:min(5, length(sigma2_loess_se_k))]),
    collapse = ", ")))

## - LOESS SE^2 nugget (PC1-5): 0.00694, 0.00270, 0.00217, 0.00190, 0.00163

cat(sprintf(" - Full residual var restored in sampling (PC1-5): %s\n",
  paste(sprintf("%.4f", sigma2_resid_k[1:min(5, length(sigma2_resid_k))]),
    collapse = ", ")))

## - Full residual var restored in sampling (PC1-5): 0.1666, 0.0648, 0.0521, 0.0456, 0.0391

cat(sprintf(" - LOESS effective n = %d\n", n_eff))

## - LOESS effective n = 24

cat(paste(rep("=", 70), collapse = ""), "\n\n")

## =====

pred_train_df <- data.frame(theta = train_theta_raw)
pred_test_df <- data.frame(theta = test_theta_raw)
gp_models <- vector("list", M_components)

for (pc_idx in seq_len(M_components)) {
  col_name <- paste0("PC", pc_idx)
  train_y <- as.numeric(train_scores[[col_name]])
  obs_noise_k <- sigma2_loess_se_k[pc_idx]

  model <- fit_known_noise_warped_gp(
    X_train = train_theta_norm,
    y_train = train_y,
    warper = input_warper,
    obs_noise_var = obs_noise_k,
    min_ls = MIN_LS,
    max_ls = MAX_LS,
    kernel = KERNEL,
    mean_type = MEAN_TYPE,
    standardize_y = TRUE
  )
}

```

```

gp_models[[pc_idx]] <- model
pred_train_df[[col_name]] <- predict_known_noise_warped_gp(
  model, train_theta_norm,
  add_observation_noise = PREDICT_OBSERVATION_NOISE
)
pred_test_df[[col_name]] <- predict_known_noise_warped_gp(
  model, test_theta_norm,
  add_observation_noise = PREDICT_OBSERVATION_NOISE
)

snr_k <- model$sigma2 / mean(model$obs_noise_var_std)
cat(sprintf(
  "PC%2d: LS=%.4f, sigma2=%.4f, loess_se2=%.5f, resid_var=%.4f, SNR(std)=%.2f, beta=[%.3f, %.3f]\n",
  pc_idx, model$lengthscale, model$sigma2, obs_noise_k,
  sigma2_resid_k[pc_idx], snr_k, model$beta[1], model$beta[2]
))
}

```

```

## PC 1: LS=0.0800, sigma2=0.4343, loess_se2=0.00694, resid_var=0.1666, SNR(std)=2.60, beta=[-1.587, 3.
## PC 2: LS=0.0908, sigma2=2.2615, loess_se2=0.00270, resid_var=0.0648, SNR(std)=5.84, beta=[1.243, -1.
## PC 3: LS=0.0818, sigma2=3.2552, loess_se2=0.00217, resid_var=0.0521, SNR(std)=11.77, beta=[0.102, 1.
## PC 4: LS=0.1014, sigma2=2.4470, loess_se2=0.00190, resid_var=0.0456, SNR(std)=6.45, beta=[-1.744, 2.
## PC 5: LS=0.0803, sigma2=0.5022, loess_se2=0.00163, resid_var=0.0391, SNR(std)=3.15, beta=[-0.972, 2.
## PC 6: LS=0.0800, sigma2=1.0996, loess_se2=0.00176, resid_var=0.0422, SNR(std)=3.14, beta=[0.520, -0.
## PC 7: LS=0.0940, sigma2=0.8711, loess_se2=0.00132, resid_var=0.0316, SNR(std)=4.18, beta=[-0.615, 1.
## PC 8: LS=0.1173, sigma2=1.5617, loess_se2=0.00135, resid_var=0.0325, SNR(std)=5.94, beta=[-1.366, 2.
## PC 9: LS=0.0884, sigma2=1.0093, loess_se2=0.00123, resid_var=0.0295, SNR(std)=2.07, beta=[0.257, -0.
## PC10: LS=0.0800, sigma2=0.5655, loess_se2=0.00122, resid_var=0.0293, SNR(std)=0.93, beta=[-1.094, 2.
## PC11: LS=0.1137, sigma2=3.7224, loess_se2=0.00106, resid_var=0.0255, SNR(std)=11.84, beta=[-1.494, 4
## PC12: LS=0.0800, sigma2=1.0377, loess_se2=0.00095, resid_var=0.0228, SNR(std)=4.73, beta=[-1.105, 1.
## PC13: LS=0.0800, sigma2=1.5525, loess_se2=0.00099, resid_var=0.0239, SNR(std)=2.66, beta=[-0.097, 0.
## PC14: LS=0.0936, sigma2=1.5208, loess_se2=0.00093, resid_var=0.0222, SNR(std)=5.13, beta=[0.790, -0.
## PC15: LS=0.1968, sigma2=12.9977, loess_se2=0.00080, resid_var=0.0192, SNR(std)=73.26, beta=[2.506, -
## PC16: LS=0.0800, sigma2=1.2834, loess_se2=0.00087, resid_var=0.0209, SNR(std)=2.33, beta=[-0.331, 0.
## PC17: LS=0.1408, sigma2=4.5908, loess_se2=0.00069, resid_var=0.0167, SNR(std)=23.06, beta=[2.404, -3
## PC18: LS=0.2160, sigma2=2.2798, loess_se2=0.00073, resid_var=0.0176, SNR(std)=5.43, beta=[0.371, 1.
## PC19: LS=0.0800, sigma2=1.0300, loess_se2=0.00067, resid_var=0.0160, SNR(std)=1.71, beta=[-0.561, 0.
## PC20: LS=0.1133, sigma2=1.0537, loess_se2=0.00048, resid_var=0.0114, SNR(std)=7.92, beta=[0.116, 0.2
## PC21: LS=0.1110, sigma2=1.3753, loess_se2=0.00056, resid_var=0.0135, SNR(std)=2.56, beta=[-1.510, 2.
## PC22: LS=0.1294, sigma2=1.3291, loess_se2=0.00046, resid_var=0.0109, SNR(std)=9.10, beta=[-1.137, 1.
## PC23: LS=0.0800, sigma2=0.4897, loess_se2=0.00038, resid_var=0.0091, SNR(std)=5.64, beta=[-1.831, 3.
## PC24: LS=0.0800, sigma2=0.5697, loess_se2=0.00049, resid_var=0.0119, SNR(std)=1.17, beta=[-1.222, 2.
## PC25: LS=0.0826, sigma2=1.6048, loess_se2=0.00041, resid_var=0.0097, SNR(std)=7.80, beta=[-1.517, 2.
## PC26: LS=0.0800, sigma2=0.8649, loess_se2=0.00043, resid_var=0.0104, SNR(std)=1.40, beta=[-1.211, 2.
## PC27: LS=0.0800, sigma2=1.5421, loess_se2=0.00042, resid_var=0.0100, SNR(std)=2.33, beta=[-0.738, 1.
## PC28: LS=0.0800, sigma2=1.4048, loess_se2=0.00036, resid_var=0.0085, SNR(std)=2.18, beta=[0.339, -0.
## PC29: LS=0.0800, sigma2=1.3995, loess_se2=0.00032, resid_var=0.0077, SNR(std)=4.52, beta=[-0.649, 0.
## PC30: LS=0.0800, sigma2=1.3535, loess_se2=0.00032, resid_var=0.0077, SNR(std)=2.85, beta=[0.289, -0.

```

```

# Verify fit: RMSE vs observed training scores
train_rmse <- sapply(seq_len(M_components), function(pc_idx) {
  col_name <- paste0("PC", pc_idx)
  true_vals <- as.numeric(train_scores[[col_name]])

```

```

    pred_vals <- pred_train_df[[col_name]]
    sqrt(mean((true_vals - pred_vals)^2))
  })
  cat("\n=== GP Fit Check (vs LOESS-smoothed score targets per theta) ===\n")

##
## === GP Fit Check (vs LOESS-smoothed score targets per theta) ===

cat("Mean training RMSE across PCs:", sprintf("%.4f", mean(train_rmse)), "\n")

## Mean training RMSE across PCs: 0.0133

if (mean(train_rmse) < 0.15) {
  cat("[OK] GP posterior mean tracks the smoothed training scores closely\n")
} else {
  cat("[INFO] Training RMSE is higher than expected for the smoothed-score target\n")
}

## [OK] GP posterior mean tracks the smoothed training scores closely

# =====
# 6b. VISUALIZING GP RESULTS IN PC SPACE
# =====

# Helper function for the current known-noise GP.
get_gp_predictions <- function(model, X_new, return_var = TRUE) {
  predict_known_noise_warped_gp(
    model, X_new, return_var = return_var,
    add_observation_noise = PREDICT_OBSERVATION_NOISE
  )
}

# --- Plot 1: GP Fit Curves for First 6 PCs ---
par(mfrow = c(3, 2), mar = c(4, 4, 3, 1))

# Create fine grid for smooth prediction curves
theta_fine <- seq(min(train_theta_norm) - 0.1, max(train_theta_norm) + 0.1, length.out = 200)
theta_fine_orig <- theta_fine * x_scale + x_center

for (pc_idx in 1:min(6, M_components)) {
  col_name <- paste0("PC", pc_idx)
  model <- gp_models[[pc_idx]]

  # Get smooth prediction curve with latent GP variance.
  pred_fine <- get_gp_predictions(model, theta_fine, return_var = TRUE)

  # One observed score vector per theta; no score averaging is used.
  train_true <- train_scores[, col_name]
  test_true <- test_scores[, col_name]

  train_pred <- pred_train_df[[col_name]]

```

```

test_pred <- pred_test_df[[col_name]]

ls_val <- model$lengthscale

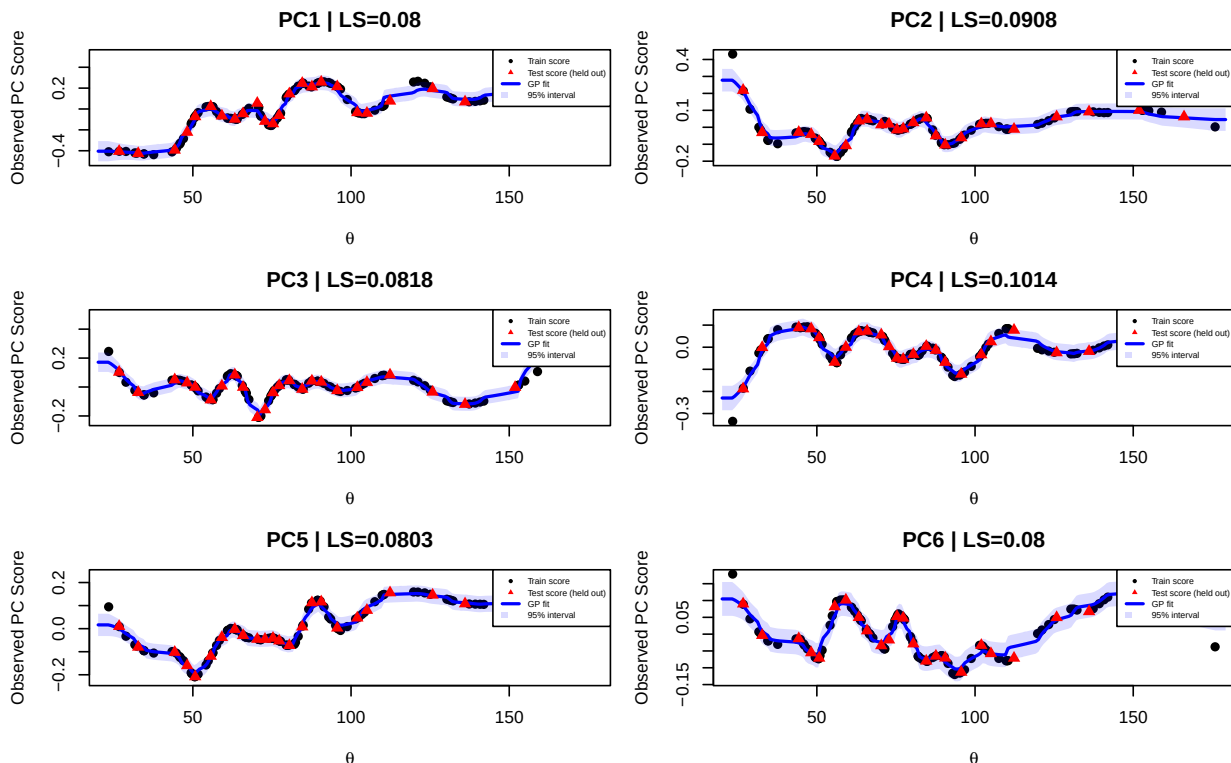
pred_sd <- sqrt(pmax(pred_fine$var, 0))
y_range <- range(c(train_true, test_true, pred_fine$mean + 1.96 * pred_sd,
                    pred_fine$mean - 1.96 * pred_sd), na.rm = TRUE)

plot(train_theta_raw, train_true, pch = 16, col = "black", cex = 1.2,
     xlim = range(c(train_theta_raw, test_theta_raw)),
     ylim = y_range,
     main = paste0(col_name, " | LS=", round(ls_val, 4)),
     xlab = expression(theta), ylab = "Observed PC Score")

polygon(c(theta_fine_orig, rev(theta_fine_orig)),
        c(pred_fine$mean + 1.96 * pred_sd, rev(pred_fine$mean - 1.96 * pred_sd)),
        col = rgb(0, 0, 1, 0.15), border = NA)
lines(theta_fine_orig, pred_fine$mean, col = "blue", lwd = 2)
points(test_theta_raw, test_true, pch = 17, col = "red", cex = 1.2)

legend("topright",
      c("Train score", "Test score (held out)", "GP fit", "95% interval"),
      pch = c(16, 17, NA, NA), lty = c(NA, NA, 1, NA),
      col = c("black", "red", "blue", rgb(0, 0, 1, 0.3)),
      fill = c(NA, NA, NA, rgb(0, 0, 1, 0.15)),
      border = c(NA, NA, NA, "transparent"),
      lwd = c(NA, NA, 2, NA), cex = 0.55, bg = "white")
}

```



```

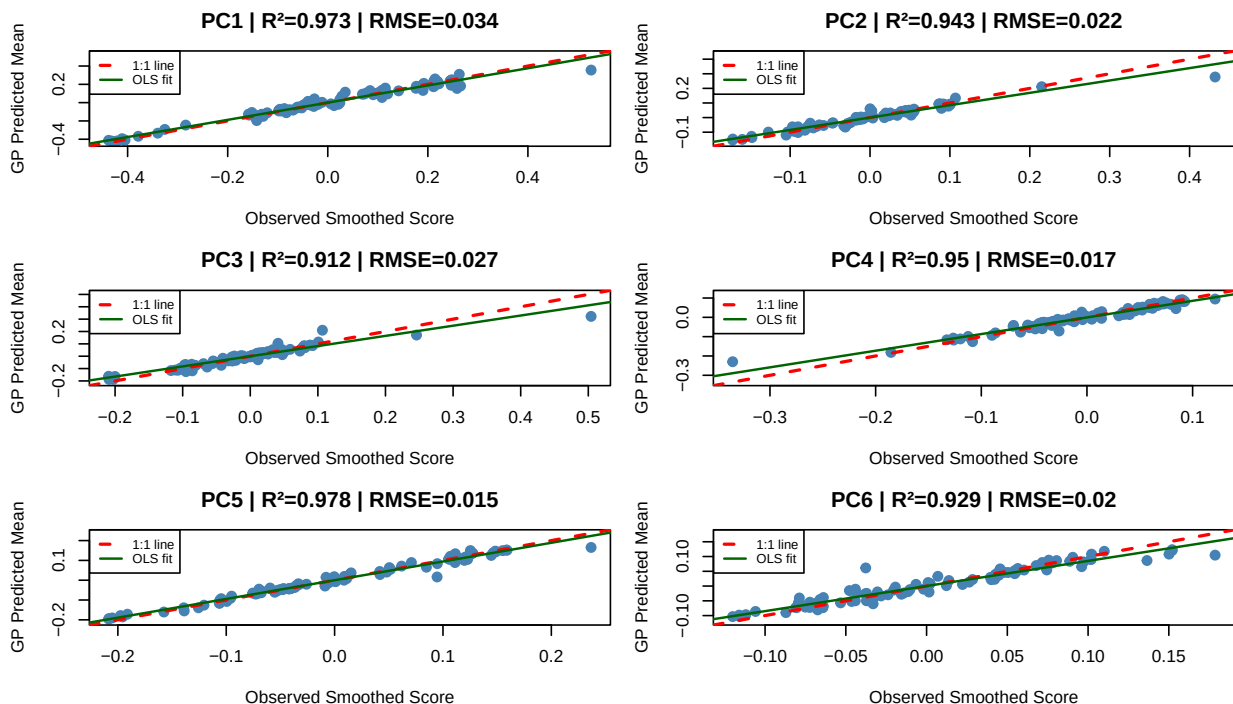
# --- Plot 2a: Mean Score
par(mfrow = c(3, 2), mar = c(4, 4, 3, 1), oma = c(0, 0, 3, 0))
for (pc_idx in 1:min(6, M_components)) {
  col_name <- paste0("PC", pc_idx)
  train_true <- train_scores[, col_name]
  train_pred <- pred_train_df[[col_name]]

  r2 <- if (var(train_true) > 1e-10 && var(train_pred) > 1e-10)
    cor(train_true, train_pred)^2 else 0
  rmse <- sqrt(mean((train_true - train_pred)^2))
  fit <- lm(train_pred ~ train_true)

  plot_range <- range(c(train_true, train_pred))
  plot(train_true, train_pred, pch = 19, col = "steelblue", cex = 1.2,
       xlim = plot_range, ylim = plot_range,
       main = paste0(col_name, " | R\u00b2=", round(r2, 3), " | RMSE=", round(rmse, 3)),
       xlab = "Observed Smoothed Score", ylab = "GP Predicted Mean")
  abline(0, 1, col = "red", lwd = 2, lty = 2)
  abline(fit, col = "darkgreen", lwd = 1.5)
  legend("topleft", c("1:1 line", "OLS fit"),
       col = c("red", "darkgreen"), lty = c(2, 1), lwd = c(2, 1.5), cex = 0.7)
}
mtext("Train Theta: GP Predicted Score vs Observed Score", outer = TRUE, cex = 1.1, font = 2)

```

Train Theta: GP Predicted Score vs Observed Score



```

# --- Plot 2b: Held-out smoothed score
par(mfrow = c(3, 2), mar = c(4, 4, 3, 1), oma = c(0, 0, 3, 0))
for (pc_idx in 1:min(6, M_components)) {
  col_name <- paste0("PC", pc_idx)

```

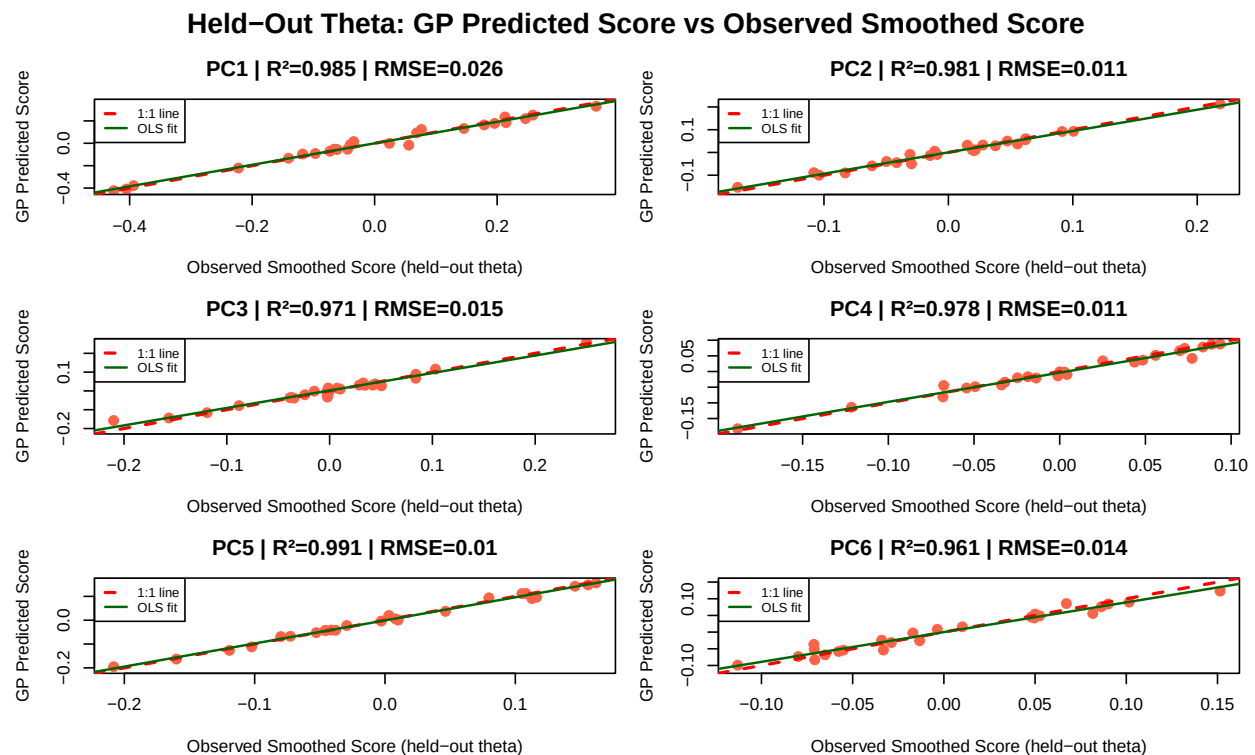
```

test_true <- test_scores[, col_name]
test_pred <- pred_test_df[[col_name]]

r2 <- if (var(test_true) > 1e-10 && var(test_pred) > 1e-10)
  cor(test_true, test_pred)^2 else 0
rmse <- sqrt(mean((test_true - test_pred)^2))
fit <- lm(test_pred ~ test_true)

plot_range <- range(c(test_true, test_pred))
plot(test_true, test_pred, pch = 19, col = "tomato", cex = 1.2,
     xlim = plot_range, ylim = plot_range,
     main = paste0(col_name, " | R\u00b2=", round(r2, 3), " | RMSE=", round(rmse, 3)),
     xlab = "Observed Smoothed Score (held-out theta)", ylab = "GP Predicted Score")
abline(0, 1, col = "red", lwd = 2, lty = 2)
abline(fit, col = "darkgreen", lwd = 1.5)
legend("topleft", c("1:1 line", "OLS fit"),
     col = c("red", "darkgreen"), lty = c(2, 1), lwd = c(2, 1.5), cex = 0.7)
}
mtext("Held-Out Theta: GP Predicted Score vs Observed Smoothed Score", outer = TRUE, cex = 1.1, font = 1)

```



```

# --- Plot 3: Residuals vs Theta (Test Set) ---
par(mfrow = c(3, 2), mar = c(4, 4, 3, 1))

for (pc_idx in 1:min(6, M_components)) {
  col_name <- paste0("PC", pc_idx)
  test_true <- test_scores[, col_name]
  test_pred <- pred_test_df[[col_name]]

```

```

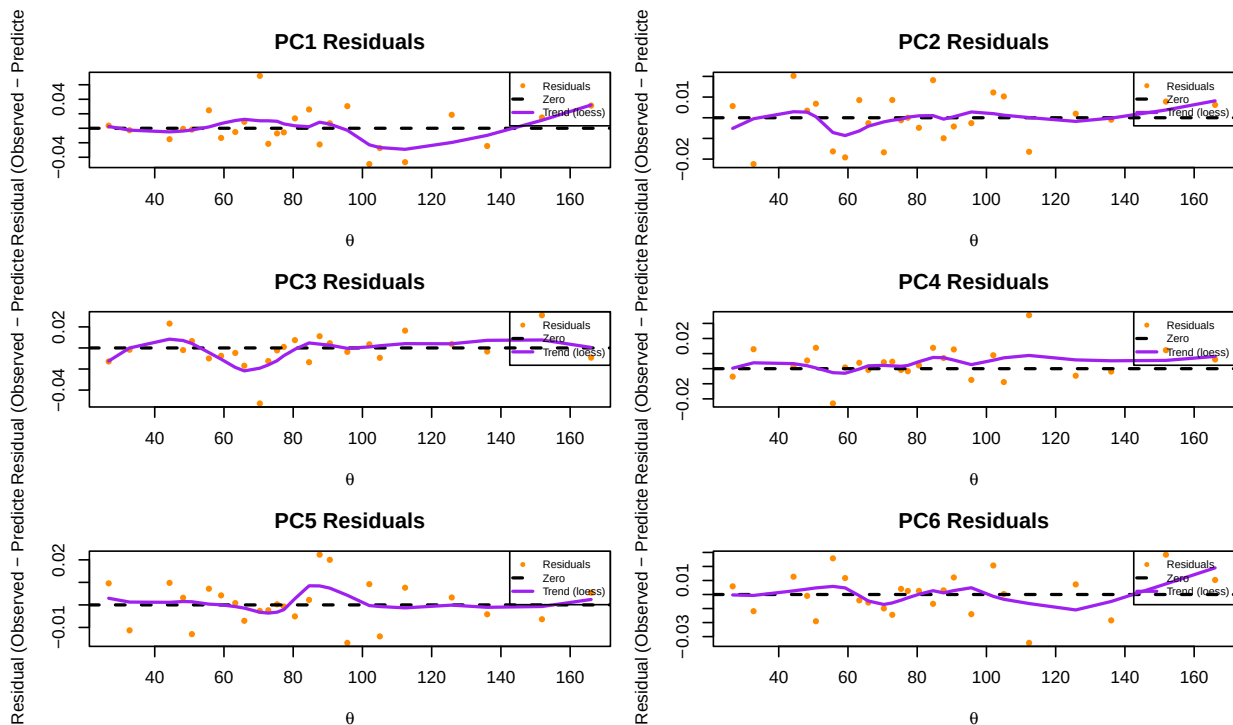
residuals <- test_true - test_pred

plot(test_theta_raw, residuals, pch = 16, col = "darkorange", cex = 0.8,
     main = paste0(col_name, " Residuals"),
     xlab = expression(theta), ylab = "Residual (Observed - Predicted)")
abline(h = 0, col = "black", lwd = 2, lty = 2)

# Add loess smoother to detect systematic bias
if (length(residuals) > 10) {
  lo <- loess(residuals ~ test_theta_raw, span = 0.5)
  x_sorted <- sort(test_theta_raw)
  lines(x_sorted, predict(lo, newdata = data.frame(test_theta_raw = x_sorted)),
        col = "purple", lwd = 2)
}

legend("topright", c("Residuals", "Zero", "Trend (loess)"),
     pch = c(16, NA, NA), lty = c(NA, 2, 1),
     col = c("darkorange", "black", "purple"),
     lwd = c(NA, 2, 2), cex = 0.65)
}

```



```

# --- Plot 4: Summary Statistics ---
par(mfrow = c(1, 2), mar = c(4, 4, 3, 1))

# Compute R^2 and RMSE for all PCs
test_r2 <- sapply(1:M_components, function(pc_idx) {
  col_name <- paste0("PC", pc_idx)
  test_true <- test_scores[, col_name]

```



```

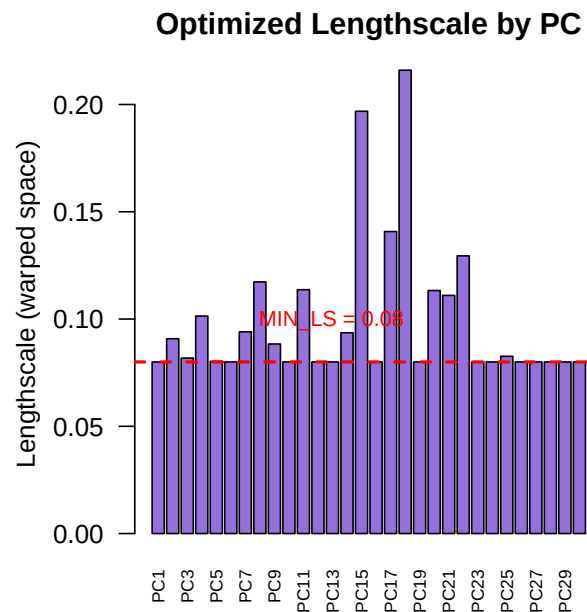
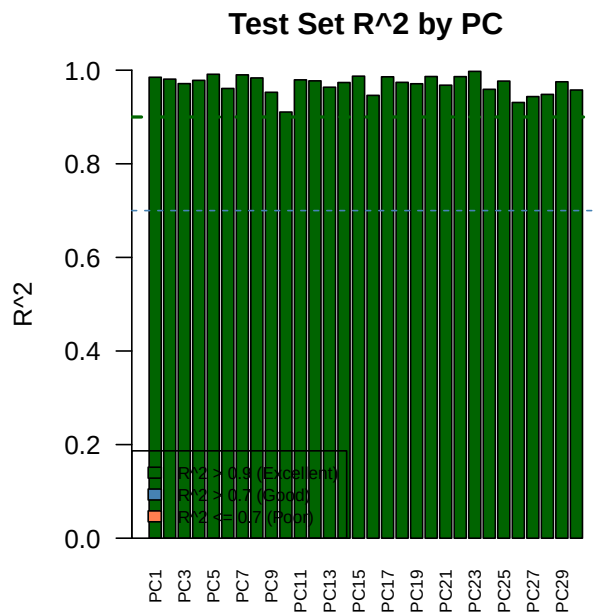
test_pred <- pred_test_df[[col_name]]
if (var(test_true) > 1e-10 && var(test_pred) > 1e-10) cor(test_true, test_pred)^2 else 0
})

test_rmse <- sapply(1:M_components, function(pc_idx) {
  col_name <- paste0("PC", pc_idx)
  test_true <- test_scores[, col_name]
  test_pred <- pred_test_df[[col_name]]
  sqrt(mean((test_true - test_pred)^2))
})

# Bar plot: R^2 by PC
barplot(test_r2, names.arg = paste0("PC", 1:M_components),
  col = ifelse(test_r2 > 0.9, "darkgreen",
    ifelse(test_r2 > 0.7, "steelblue", "coral")),
  main = "Test Set R^2 by PC",
  ylab = "R^2", las = 2, cex.names = 0.7, ylim = c(0, 1))
abline(h = 0.9, col = "darkgreen", lty = 2, lwd = 2)
abline(h = 0.7, col = "steelblue", lty = 2, lwd = 1)
legend("bottomleft", c("R^2 > 0.9 (Excellent)", "R^2 > 0.7 (Good)", "R^2 <= 0.7 (Poor)"),
  fill = c("darkgreen", "steelblue", "coral"), cex = 0.7)

# Bar plot: Lengthscale by PC
all_lengthscales <- sapply(gp_models, function(m) m$lengthscale)
barplot(all_lengthscales, names.arg = paste0("PC", 1:M_components),
  col = "mediumpurple",
  main = "Optimized Lengthscale by PC",
  ylab = "Lengthscale (warped space)", las = 2, cex.names = 0.7)
abline(h = MIN_LS, col = "red", lty = 2, lwd = 2)
text(M_components/2, MIN_LS + 0.02, paste("MIN_LS =", MIN_LS), col = "red", cex = 0.8)

```



```
# --- Print Summary Statistics ---
cat("\n--- GP PERFORMANCE SUMMARY (TEST SET) ---\n")

##
## --- GP PERFORMANCE SUMMARY (TEST SET) ---

cat(sprintf("%-6s %8s %10s %12s\n", "PC", "R^2", "RMSE", "Lengthscale"))

## PC          R^2          RMSE  Lengthscale

cat(paste(rep("-", 40), collapse = ""), "\n")

## -----

for (pc_idx in 1:M_components) {
  cat(sprintf("PC%-4d %8.4f %10.4f %12.4f\n",
              pc_idx, test_r2[pc_idx], test_rmse[pc_idx], all_lengthscales[pc_idx]))
}

## PC1      0.9849      0.0259      0.0800
## PC2      0.9809      0.0113      0.0908
## PC3      0.9713      0.0155      0.0818
## PC4      0.9783      0.0111      0.1014
## PC5      0.9913      0.0095      0.0803
## PC6      0.9610      0.0144      0.0800
```

```
## PC7      0.9900    0.0079    0.0940
## PC8      0.9835    0.0100    0.1173
## PC9      0.9528    0.0116    0.0884
## PC10     0.9106    0.0123    0.0800
## PC11     0.9794    0.0094    0.1137
## PC12     0.9772    0.0112    0.0800
## PC13     0.9636    0.0120    0.0800
## PC14     0.9737    0.0103    0.0936
## PC15     0.9872    0.0073    0.1968
## PC16     0.9462    0.0093    0.0800
## PC17     0.9859    0.0067    0.1408
## PC18     0.9740    0.0080    0.2160
## PC19     0.9710    0.0073    0.0800
## PC20     0.9865    0.0072    0.1133
## PC21     0.9680    0.0061    0.1110
## PC22     0.9863    0.0071    0.1294
## PC23     0.9975    0.0037    0.0800
## PC24     0.9591    0.0087    0.0800
## PC25     0.9767    0.0070    0.0826
## PC26     0.9310    0.0072    0.0800
## PC27     0.9436    0.0064    0.0800
## PC28     0.9483    0.0059    0.0800
## PC29     0.9752    0.0058    0.0800
## PC30     0.9577    0.0065    0.0800
```

```
cat(paste(rep("-", 40), collapse = ""), "\n")
```

```
## -----
```

```
cat(sprintf("%-6s %8.4f %10.4f %12.4f\n",
            "MEAN", mean(test_r2), mean(test_rmse), mean(all_lengthscales)))
```

```
## MEAN      0.9698    0.0094    0.0990
```

```
cat("\n")
```

```
# Check for PCs hitting the lengthscale floor
n_at_floor <- sum(all_lengthscales < MIN_LS * 1.1)
if (n_at_floor > 0) {
  cat("[INFO]", n_at_floor, "PCs have lengthscale near MIN_LS =", MIN_LS, "\n")
  cat(" Gaussian NLL optimizer consistently prefers short LS (tight training fit).\n")
  cat(" MIN_LS=", MIN_LS, "is intentionally set to the inter-sigma trend scale.\n")
  cat(" If test ISE is still >= Naive, consider raising MIN_LS further (e.g. 0.50).\n")
} else {
  cat("[OK] Lengthscales above MIN_LS floor")
}
```

```
## [INFO] 17 PCs have lengthscale near MIN_LS = 0.08
## Gaussian NLL optimizer consistently prefers short LS (tight training fit).
## MIN_LS= 0.08 is intentionally set to the inter-sigma trend scale.
## If test ISE is still >= Naive, consider raising MIN_LS further (e.g. 0.50).
```

```
cat(paste(rep("=", 70), collapse = ""), "\n\n")
```

```
## =====
```

```
# =====
# DIRECT GRID RECONSTRUCTION
# =====
# Reconstruction decodes directly from MFPCA scores to the raw X/Y/Z grids.
# Prediction geometry vectors are NA placeholders because the reconstruction
# function ignores geometry arguments.
# =====
n_test_pk <- length(test_theta_vals)
pred_peak_x_test <- pred_peak_z_test <- pred_width_x_test <- pred_width_z_test <-
  pred_left_width_x_test <- pred_right_width_x_test <-
  pred_left_width_z_test <- pred_right_width_z_test <-
  pred_height_x_test <- pred_height_z_test <- rep(NA_real_, n_test_pk)

mean_train_peak_x_loc <- mean_train_peak_z_loc <-
  mean_train_peak_x_width <- mean_train_peak_z_width <-
  mean_train_peak_x_left_width <- mean_train_peak_x_right_width <-
  mean_train_peak_z_left_width <- mean_train_peak_z_right_width <- NA_real_

cat("Reconstructing directly on raw grids.\n")
```

```
## Reconstructing directly on raw grids.
```

```
cat(paste(rep("=", 70), collapse = ""), "\n\n")
```

```
## =====
```

```
# =====
# --- Plot 5: Held-out theta-level  $R^2$  along the full theta grid ---
binned_r2 <- numeric(M_components)
emp_mean_list <- vector("list", M_components)
gp_mean_list <- vector("list", M_components)

for (pc_idx in 1:M_components) {
  col_name <- paste0("PC", pc_idx)
  model <- gp_models[[pc_idx]]

  # One observed score vector exists at each theta, so no averaging is needed.
  emp_means <- ufPCA_scores_smoothed[[col_name]]

  # GP predicted score at each theta on the full grid
  gp_means <- get_gp_predictions(model, (theta_list - x_center) / x_scale, return_var = FALSE)

  emp_mean_list[[pc_idx]] <- emp_means
  gp_mean_list[[pc_idx]] <- gp_means

  binned_r2[pc_idx] <- if (var(emp_means) > 1e-10 && var(gp_means) > 1e-10)
    cor(emp_means, gp_means)^2 else 0
}
```

```

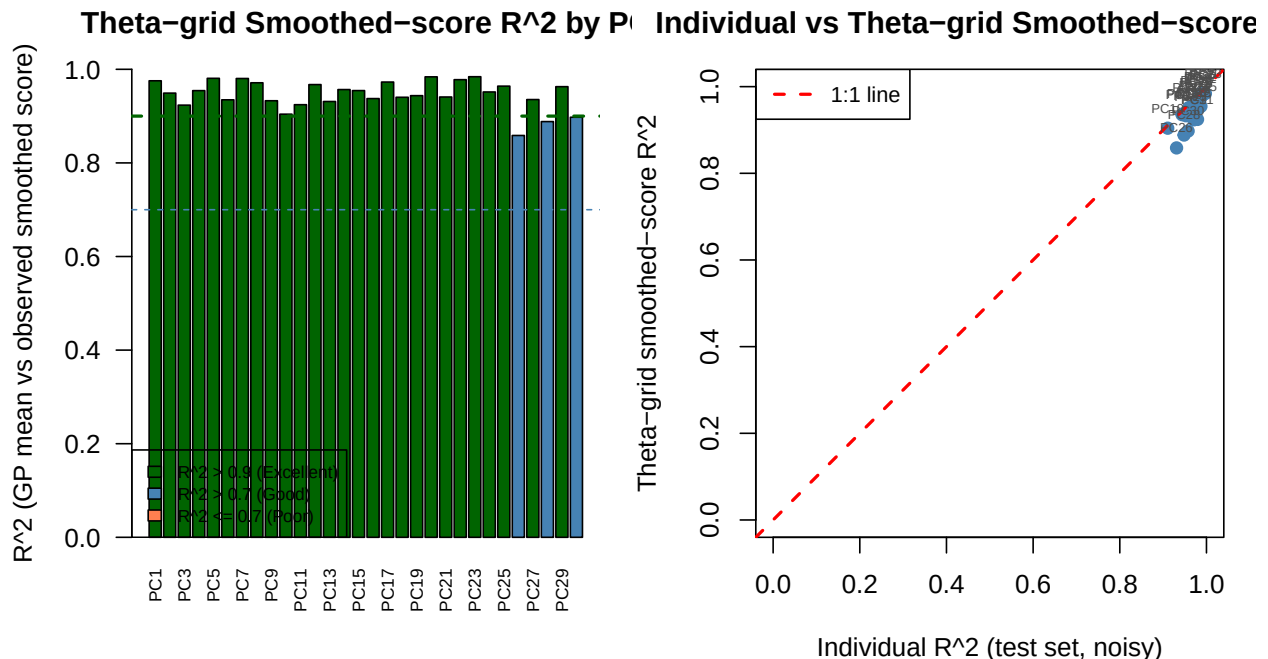
}

# Barplot + scatter comparison
par(mfrow = c(1, 2), mar = c(4, 4, 3, 1))

barplot(binned_r2,
        names.arg = paste0("PC", 1:M_components),
        col = ifelse(binned_r2 > 0.9, "darkgreen",
                     ifelse(binned_r2 > 0.7, "steelblue", "coral")),
        main = "Theta-grid Smoothed-score R^2 by PC",
        ylab = "R^2 (GP mean vs observed smoothed score)",
        las = 2, cex.names = 0.7, ylim = c(0, 1))
abline(h = 0.9, col = "darkgreen", lty = 2, lwd = 2)
abline(h = 0.7, col = "steelblue", lty = 2, lwd = 1)
legend("bottomleft",
       c("R^2 > 0.9 (Excellent)", "R^2 > 0.7 (Good)", "R^2 <= 0.7 (Poor)"),
       fill = c("darkgreen", "steelblue", "coral"), cex = 0.7)

plot(test_r2, binned_r2, pch = 19, col = "steelblue",
     xlim = c(0, 1), ylim = c(0, 1),
     xlab = "Individual R^2 (test set, noisy)",
     ylab = "Theta-grid smoothed-score R^2",
     main = "Individual vs Theta-grid Smoothed-score R^2")
abline(0, 1, col = "red", lty = 2, lwd = 2)
text(test_r2, binned_r2, labels = paste0("PC", 1:M_components),
     pos = 3, cex = 0.5, col = "grey30")
legend("topleft", "1:1 line", col = "red", lty = 2, lwd = 2, cex = 0.8)

```

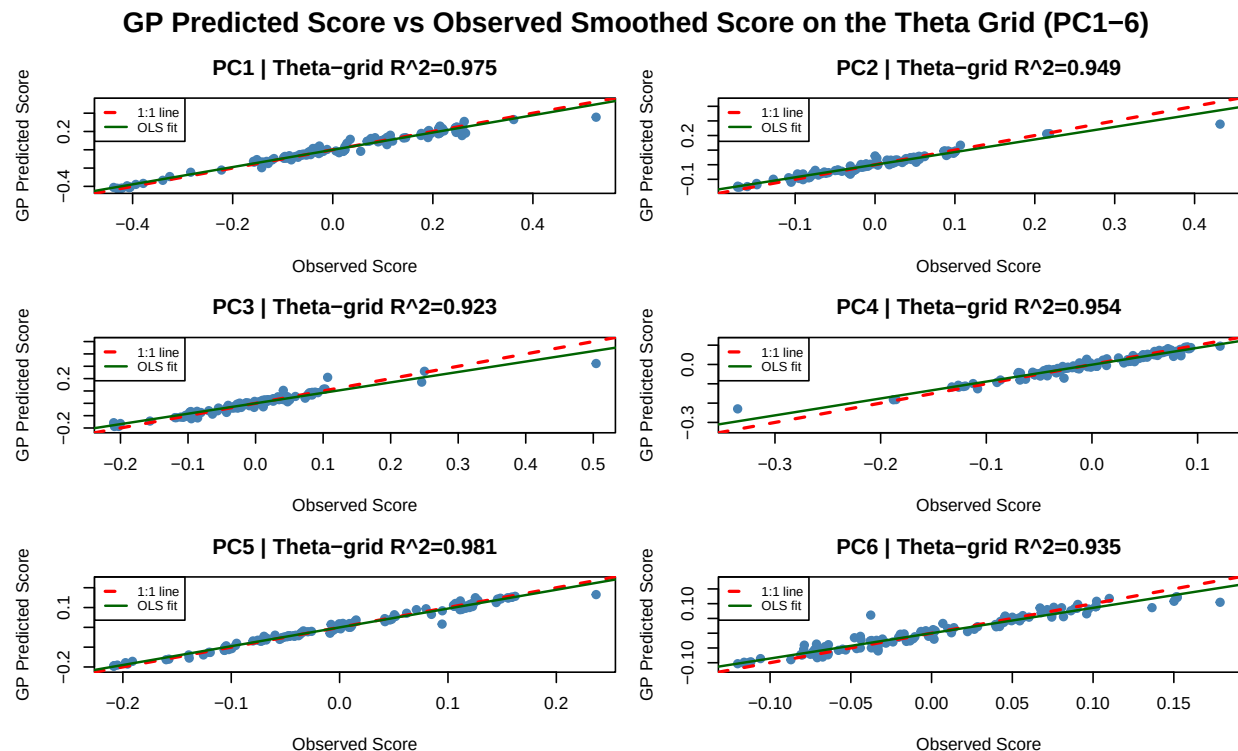


```

# Scatter: GP mean vs observed held-out score for first 6 PCs
par(mfrow = c(3, 2), mar = c(4, 4, 3, 1), oma = c(0, 0, 3, 0))
for (pc_idx in 1:min(6, M_components)) {
  emp_means <- emp_mean_list[[pc_idx]]
  gp_means <- gp_mean_list[[pc_idx]]
  r2_bin <- binned_r2[pc_idx]
  fit <- lm(gp_means ~ emp_means)
  pr <- range(c(emp_means, gp_means))

  plot(emp_means, gp_means, pch = 19, col = "steelblue", cex = 1.0,
       xlim = pr, ylim = pr,
       main = paste0("PC", pc_idx, " | Theta-grid R^2=", round(r2_bin, 3)),
       xlab = "Observed Score",
       ylab = "GP Predicted Score")
  abline(0, 1, col = "red", lwd = 2, lty = 2)
  abline(fit, col = "darkgreen", lwd = 1.5)
  legend("topleft", c("1:1 line", "OLS fit"),
       col = c("red", "darkgreen"), lty = c(2, 1), lwd = c(2, 1.5), cex = 0.7)
}
mtext("GP Predicted Score vs Observed Smoothed Score on the Theta Grid (PC1-6)",
     outer = TRUE, cex = 1.1, font = 2)

```



```

# Print table
cat("\n--- THETA-GRID SMOOTHED-SCORE R^2 SUMMARY ---\n")

```

```

##
## --- THETA-GRID SMOOTHED-SCORE R^2 SUMMARY ---

```

```

cat("(GP predicted scores vs LOESS-smoothed theta-level targets)\n")

## (GP predicted scores vs LOESS-smoothed theta-level targets)

cat(sprintf("%-6s %14s %14s\n", "PC", "Individual R^2", "Held-out R^2"))

## PC      Individual R^2    Held-out R^2

cat(paste(rep("-", 36), collapse = ""), "\n")

## -----

for (pc_idx in 1:M_components) {
  cat(sprintf("PC%-4d %14.4f %14.4f\n",
              pc_idx, test_r2[pc_idx], binned_r2[pc_idx]))
}

## PC1      0.9849      0.9755
## PC2      0.9809      0.9491
## PC3      0.9713      0.9234
## PC4      0.9783      0.9545
## PC5      0.9913      0.9807
## PC6      0.9610      0.9348
## PC7      0.9900      0.9804
## PC8      0.9835      0.9712
## PC9      0.9528      0.9328
## PC10     0.9106      0.9041
## PC11     0.9794      0.9244
## PC12     0.9772      0.9674
## PC13     0.9636      0.9312
## PC14     0.9737      0.9567
## PC15     0.9872      0.9545
## PC16     0.9462      0.9375
## PC17     0.9859      0.9728
## PC18     0.9740      0.9402
## PC19     0.9710      0.9437
## PC20     0.9865      0.9839
## PC21     0.9680      0.9408
## PC22     0.9863      0.9779
## PC23     0.9975      0.9842
## PC24     0.9591      0.9516
## PC25     0.9767      0.9640
## PC26     0.9310      0.8586
## PC27     0.9436      0.9354
## PC28     0.9483      0.8883
## PC29     0.9752      0.9629
## PC30     0.9577      0.8975

cat(paste(rep("-", 36), collapse = ""), "\n")

## -----

```

```
cat(sprintf("%-6s %14.4f %14.4f\n", "MEAN", mean(test_r2), mean(binned_r2)))
```

```
## MEAN          0.9698          0.9460
```

```
cat("\n")
```

```
# =====
# COVERAGE PROBABILITY ANALYSIS
# =====
# Coverage probability measures how well the GP uncertainty is calibrated.
# For a 95% confidence interval, we expect ~95% of test points to fall within.
#
# If coverage << 95%: GP is OVERCONFIDENT (uncertainty too narrow)
# If coverage >> 95%: GP is UNDERCONFIDENT (uncertainty too wide)
# If coverage ~ 95%: GP uncertainty is well-calibrated!
# =====
```

```
cat("\n--- COVERAGE PROBABILITY ANALYSIS (Test Set) ---\n")
```

```
##
## --- COVERAGE PROBABILITY ANALYSIS (Test Set) ---
```

```
cat("Using current known-noise GP: interval variance = var_gp + sigma2_resid_k\n\n")
```

```
## Using current known-noise GP: interval variance = var_gp + sigma2_resid_k
```

```
# Compute coverage for different confidence levels
confidence_levels <- c(0.50, 0.80, 0.90, 0.95, 0.99)

# Storage for coverage results
coverage_by_pc <- matrix(NA, nrow = M_components, ncol = length(confidence_levels))
colnames(coverage_by_pc) <- paste0(confidence_levels * 100, "%")
rownames(coverage_by_pc) <- paste0("PC", 1:M_components)

# get_gp_predictions() is defined in the visualization chunk above (section 6b)

for (pc_idx in 1:M_components) {
  col_name <- paste0("PC", pc_idx)
  model <- gp_models[[pc_idx]]

  # Get predictions with variance for test set.
  pred_test_with_var <- get_gp_predictions(model, test_theta_norm, return_var = TRUE)

  test_true <- test_scores[, col_name]
  test_pred <- pred_test_with_var$mean
  # Score predictive distribution restores the noise stripped by LOESS.
  score_var <- pred_test_with_var$var_gp + sigma2_resid_k[pc_idx]
  test_sd <- sqrt(pmax(score_var, 1e-10))

  # Compute coverage for each confidence level
  for (j in seq_along(confidence_levels)) {
```



```

alpha <- 1 - confidence_levels[j]
z_crit <- qnorm(1 - alpha/2)

# Check if true value falls within [pred - z*sd, pred + z*sd]
lower <- test_pred - z_crit * test_sd
upper <- test_pred + z_crit * test_sd

in_interval <- (test_true >= lower) & (test_true <= upper)
coverage_by_pc[pc_idx, j] <- mean(in_interval) * 100
}
}

# Print coverage table
cat(sprintf("\n%-6s", "PC"))

```

```

##
## PC

```

```

for (cl in confidence_levels) {
  cat(sprintf(" %6s", paste0(cl*100, "%")))
}

```

```

##      50%      80%      90%      95%      99%

```

```

cat("\n")

```

```

cat(paste(rep("-", 6 + 7*length(confidence_levels)), collapse = ""), "\n")

```

```

## -----

```

```

for (pc_idx in 1:M_components) {
  cat(sprintf("PC%-4d", pc_idx))
  for (j in seq_along(confidence_levels)) {
    cov <- coverage_by_pc[pc_idx, j]
    cat(sprintf(" %5.1f%%", cov))
  }
  cat("\n")
}

```

```

## PC1      100.0% 100.0% 100.0% 100.0% 100.0%
## PC2      100.0% 100.0% 100.0% 100.0% 100.0%
## PC3      100.0% 100.0% 100.0% 100.0% 100.0%
## PC4      100.0% 100.0% 100.0% 100.0% 100.0%
## PC5      100.0% 100.0% 100.0% 100.0% 100.0%
## PC6      100.0% 100.0% 100.0% 100.0% 100.0%
## PC7      100.0% 100.0% 100.0% 100.0% 100.0%
## PC8      100.0% 100.0% 100.0% 100.0% 100.0%
## PC9      100.0% 100.0% 100.0% 100.0% 100.0%
## PC10     100.0% 100.0% 100.0% 100.0% 100.0%
## PC11     100.0% 100.0% 100.0% 100.0% 100.0%
## PC12     100.0% 100.0% 100.0% 100.0% 100.0%

```

```
## PC13 100.0% 100.0% 100.0% 100.0% 100.0%
## PC14 100.0% 100.0% 100.0% 100.0% 100.0%
## PC15 100.0% 100.0% 100.0% 100.0% 100.0%
## PC16 100.0% 100.0% 100.0% 100.0% 100.0%
## PC17 100.0% 100.0% 100.0% 100.0% 100.0%
## PC18 100.0% 100.0% 100.0% 100.0% 100.0%
## PC19 100.0% 100.0% 100.0% 100.0% 100.0%
## PC20 100.0% 100.0% 100.0% 100.0% 100.0%
## PC21 100.0% 100.0% 100.0% 100.0% 100.0%
## PC22 100.0% 100.0% 100.0% 100.0% 100.0%
## PC23 100.0% 100.0% 100.0% 100.0% 100.0%
## PC24 100.0% 100.0% 100.0% 100.0% 100.0%
## PC25 100.0% 100.0% 100.0% 100.0% 100.0%
## PC26 100.0% 100.0% 100.0% 100.0% 100.0%
## PC27 100.0% 100.0% 100.0% 100.0% 100.0%
## PC28 100.0% 100.0% 100.0% 100.0% 100.0%
## PC29 100.0% 100.0% 100.0% 100.0% 100.0%
## PC30 100.0% 100.0% 100.0% 100.0% 100.0%
```

```
cat(paste(rep("-", 6 + 7*length(confidence_levels)), collapse = ""), "\n")
```

```
## -----
```

```
cat(sprintf("%-6s", "MEAN"))
```

```
## MEAN
```

```
for (j in seq_along(confidence_levels)) {
  cat(sprintf(" %5.1f%%", mean(coverage_by_pc[, j])))
}
```

```
## 100.0% 100.0% 100.0% 100.0% 100.0%
```

```
cat("\n")
```

```
cat(sprintf("%-6s", "TARGET"))
```

```
## TARGET
```

```
for (cl in confidence_levels) {
  cat(sprintf(" %5.1f%%", cl * 100))
}
```

```
## 50.0% 80.0% 90.0% 95.0% 99.0%
```

```
cat("\n\n")
```

```

# Overall assessment
mean_95_coverage <- mean(coverage_by_pc[, "95%"])
if (abs(mean_95_coverage - 95) < 5) {
  cat("[OK] 95% CI coverage is well-calibrated (", round(mean_95_coverage, 1), "% vs target 95%)\n")
} else if (mean_95_coverage < 95) {
  cat("[WARNING] GP is OVERCONFIDENT: 95% CI coverage =", round(mean_95_coverage, 1),
      "% (target: 95%)\n")
  cat("  Uncertainty bands are too narrow. Consider:\n")
  cat("  - Checking whether sigma2_resid_k is underestimated\n")
  cat("  - Verifying var_gp is in the original score scale\n")
} else {
  cat("[INFO] GP is slightly UNDERCONFIDENT: 95% CI coverage =", round(mean_95_coverage, 1),
      "% (target: 95%)\n")
  cat("  Uncertainty bands are wider than necessary (conservative).\n")
}

```

```

## [INFO] GP is slightly UNDERCONFIDENT: 95% CI coverage = 100 % (target: 95%)
##   Uncertainty bands are wider than necessary (conservative).

```

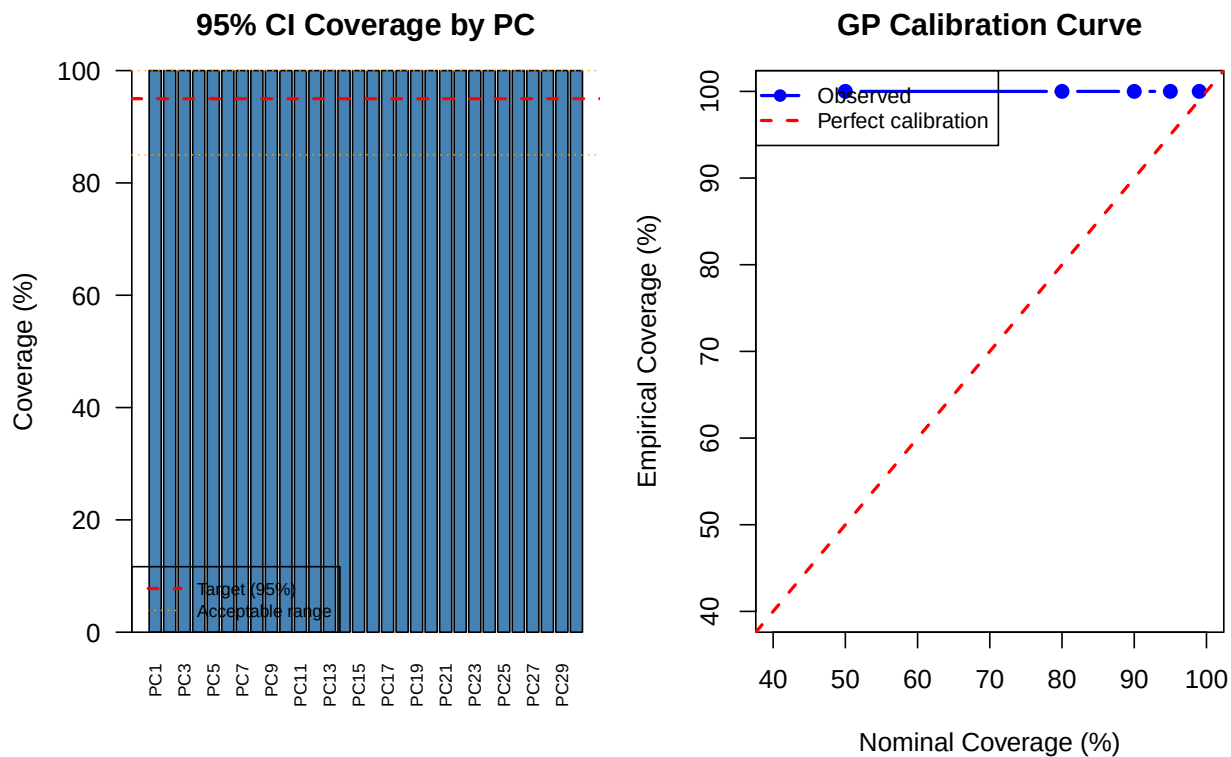
```

# Visualization: Coverage probability plot
par(mfrow = c(1, 2), mar = c(4, 4, 3, 1))

# Plot 1: Coverage by PC for 95% CI
barplot(coverage_by_pc[, "95%"], names.arg = paste0("PC", 1:M_components),
        col = ifelse(abs(coverage_by_pc[, "95%"] - 95) < 10, "steelblue", "coral"),
        main = "95% CI Coverage by PC",
        ylab = "Coverage (%)", las = 2, cex.names = 0.7,
        ylim = c(0, 100))
abline(h = 95, col = "red", lwd = 2, lty = 2)
abline(h = c(85, 100), col = "orange", lwd = 1, lty = 3)
legend("bottomleft", c("Target (95%)", "Acceptable range"),
      col = c("red", "orange"), lty = c(2, 3), lwd = c(2, 1), cex = 0.7)

# Plot 2: Calibration curve (coverage vs nominal)
mean_coverage <- colMeans(coverage_by_pc)
plot(confidence_levels * 100, mean_coverage, type = "b", pch = 19, col = "blue", lwd = 2,
     main = "GP Calibration Curve",
     xlab = "Nominal Coverage (%)", ylab = "Empirical Coverage (%)",
     xlim = c(40, 100), ylim = c(40, 100))
abline(0, 1, col = "red", lwd = 2, lty = 2) # Perfect calibration line
legend("topleft", c("Observed", "Perfect calibration"),
      pch = c(19, NA), lty = c(1, 2), col = c("blue", "red"), lwd = 2, cex = 0.8)

```



```
cat(paste(rep("=", 70), collapse = ""), "\n\n")
```

```
## =====
```

```
# =====
# 7. RECONSTRUCTION (INVERSE SQUARE-ROOT MAP) & EVALUATION
# =====

# Inverse square-root density transform.
inv_clr_transform <- function(sqrt_vec, grid) {
  sqrt_vec <- pmax(as.numeric(sqrt_vec), 0)
  dens_raw <- sqrt_vec^2
  delta <- diff(range(grid)) / (length(grid) - 1)
  mass <- sum(dens_raw) * delta
  if (!is.finite(mass) || mass <= 0) {
    dens_raw <- rep(1 / diff(range(grid)), length(grid))
    mass <- sum(dens_raw) * delta
  }
  dens_raw / mass
}

# Extract mean functions and eigenfunctions
mu_x <- as.vector(mfpca_fit$meanFunction[[1]]@X)
mu_y <- as.vector(mfpca_fit$meanFunction[[2]]@X)
mu_z <- as.vector(mfpca_fit$meanFunction[[3]]@X)

phi_x <- mfpca_fit$functions[[1]]@X
phi_y <- mfpca_fit$functions[[2]]@X
```

```

phi_z <- mfpca_fit$functions[[3]]@X

# Create mapping from theta to row index in density matrices
sigma_to_idx <- setNames(1:length(sigma_list), as.character(round(sigma_list, 6)))

# --- Function to reconstruct densities from scores ---
reconstruct_densities <- function(scores_mat, mu_x, mu_y, mu_z, phi_x, phi_y, phi_z,
                                x_grid, y_grid, z_grid,
                                x_peak_loc = NULL, x_peak_width = NULL,
                                z_peak_loc = NULL, z_peak_width = NULL,
                                x_peak_left_width = NULL, x_peak_right_width = NULL,
                                z_peak_left_width = NULL, z_peak_right_width = NULL) {

  n <- nrow(scores_mat)

  # Square-root density reconstructions
  recon_clr_x <- sweep(scores_mat %*% phi_x, 2, mu_x, "+")
  recon_clr_y <- sweep(scores_mat %*% phi_y, 2, mu_y, "+")
  recon_clr_z <- sweep(scores_mat %*% phi_z, 2, mu_z, "+")

  # Convert directly to densities on the original grids. Geometry arguments are
  # accepted for compatibility and ignored.
  dens_x <- t(apply(recon_clr_x, 1, inv_clr_transform, grid = x_grid))
  dens_y <- t(apply(recon_clr_y, 1, inv_clr_transform, grid = y_grid))
  dens_z <- t(apply(recon_clr_z, 1, inv_clr_transform, grid = z_grid))

  list(x = dens_x, y = dens_y, z = dens_z)
}

# Reconstruct for Test Set using GP-predicted scores (one row per test theta)
scores_pred_test <- as.matrix(pred_test_df[, -1])
recon_test <- reconstruct_densities(scores_pred_test, mu_x, mu_y, mu_z,
                                   phi_x, phi_y, phi_z, x_grid, y_grid, z_grid,
                                   x_peak_loc = pred_peak_x_test,
                                   x_peak_width = pred_width_x_test,
                                   x_peak_left_width = pred_left_width_x_test,
                                   x_peak_right_width = pred_right_width_x_test,
                                   z_peak_loc = pred_peak_z_test,
                                   z_peak_width = pred_width_z_test,
                                   z_peak_left_width = pred_left_width_z_test,
                                   z_peak_right_width = pred_right_width_z_test)

# -----
# NOTE: Post-hoc peak shift correction was trialled but removed because
# shift_density_to_peak() worsened ISE for multimodal X and Z
# (shifting the dominant peak misplaces secondary resonance peaks and loses density
# at grid boundaries). Option B variance inflation is retained instead.

# Naive baseline: MFPCA reconstruction from the average training PC score vector.
# This uses the same decoder as the GP approach; only the latent score input differs.
# GP uses theta-specific predicted scores, while the naive baseline reuses one
# training-average score vector for every held-out theta.
mean_train_scores_vec <- colMeans(train_scores) # training-average score per PC
mean_train_scores_mat <- matrix(mean_train_scores_vec, nrow = 1) # 1 x M matrix

```

```

recon_naive      <- reconstruct_densities(mean_train_scores_mat,
                                         mu_x, mu_y, mu_z,
                                         phi_x, phi_y, phi_z,
                                         x_grid, y_grid, z_grid,
                                         x_peak_loc = mean_train_peak_x_loc,
                                         x_peak_width = mean_train_peak_x_width,
                                         x_peak_left_width = mean_train_peak_x_left_width,
                                         x_peak_right_width = mean_train_peak_x_right_width,
                                         z_peak_loc = mean_train_peak_z_loc,
                                         z_peak_width = mean_train_peak_z_width,
                                         z_peak_left_width = mean_train_peak_z_left_width,
                                         z_peak_right_width = mean_train_peak_z_right_width)

mean_train_dens_x <- as.vector(recon_naive$x)
mean_train_dens_y <- as.vector(recon_naive$y)
mean_train_dens_z <- as.vector(recon_naive$z)

# =====
# 8. EVALUATION METRICS
# =====

# Integrated Squared Error
compute_ISE <- function(f1, f2, grid) {
  delta <- diff(range(grid)) / (length(grid) - 1)
  sum((f1 - f2)^2) * delta
}

compute_hellinger <- function(f1, f2, grid) {
  delta <- diff(range(grid)) / (length(grid) - 1)
  sqrt(0.5 * sum((sqrt(pmax(f1, 0)) - sqrt(pmax(f2, 0)))^2) * delta)
}

# Basis characterisation only: reconstruction ISE is retained for Table 3 and
# the three-stage reconstruction figure. It is not used as a primary
# density-space validation metric for the emulator.
n_test_s <- length(test_theta_vals)

# Mean-only density evaluation: posterior mean reconstruction only.
gp_mean_ise_x <- gp_mean_ise_y <- gp_mean_ise_z <- numeric(n_test_s)
naive_mean_ise_x <- naive_mean_ise_y <- naive_mean_ise_z <- numeric(n_test_s)
gp_mean_hell_x <- gp_mean_hell_y <- gp_mean_hell_z <- numeric(n_test_s)
naive_mean_hell_x <- naive_mean_hell_y <- naive_mean_hell_z <- numeric(n_test_s)

for (i in seq_len(n_test_s)) {
  ref_row <- test_theta_idx[i]
  gp_mean_ise_x[i] <- compute_ISE(recon_test$x[i, ], dens_mat_x[ref_row, ], x_grid)
  gp_mean_ise_y[i] <- compute_ISE(recon_test$y[i, ], dens_mat_y[ref_row, ], y_grid)
  gp_mean_ise_z[i] <- compute_ISE(recon_test$z[i, ], dens_mat_z[ref_row, ], z_grid)
  gp_mean_hell_x[i] <- compute_hellinger(recon_test$x[i, ], dens_mat_x[ref_row, ], x_grid)
  gp_mean_hell_y[i] <- compute_hellinger(recon_test$y[i, ], dens_mat_y[ref_row, ], y_grid)
  gp_mean_hell_z[i] <- compute_hellinger(recon_test$z[i, ], dens_mat_z[ref_row, ], z_grid)

  naive_mean_ise_x[i] <- compute_ISE(mean_train_dens_x, dens_mat_x[ref_row, ], x_grid)
  naive_mean_ise_y[i] <- compute_ISE(mean_train_dens_y, dens_mat_y[ref_row, ], y_grid)

```

```

naive_mean_ise_z[i] <- compute_ISE(mean_train_dens_z, dens_mat_z[ref_row, ], z_grid)
naive_mean_hell_x[i] <- compute_hellinger(mean_train_dens_x, dens_mat_x[ref_row, ], x_grid)
naive_mean_hell_y[i] <- compute_hellinger(mean_train_dens_y, dens_mat_y[ref_row, ], y_grid)
naive_mean_hell_z[i] <- compute_hellinger(mean_train_dens_z, dens_mat_z[ref_row, ], z_grid)
}

gp_mean_l2_x <- sqrt(gp_mean_ise_x)
gp_mean_l2_y <- sqrt(gp_mean_ise_y)
gp_mean_l2_z <- sqrt(gp_mean_ise_z)
naive_mean_l2_x <- sqrt(naive_mean_ise_x)
naive_mean_l2_y <- sqrt(naive_mean_ise_y)
naive_mean_l2_z <- sqrt(naive_mean_ise_z)

# -----
# Diagnostic: GP score error vs the MFPCA reconstruction floor.
# -----
mf pca_test_ise_x <- ise_mfpca_x[test_theta_idx]
mf pca_test_ise_y <- ise_mfpca_y[test_theta_idx]
mf pca_test_ise_z <- ise_mfpca_z[test_theta_idx]

gp_mfpca_decomp <- data.frame(
  Stage = c("MFPCA true scores",
            "GP scores",
            "Naive baseline"),
  X_ISE = c(mean(mf pca_test_ise_x),
            mean(gp_mean_ise_x),
            mean(naive_mean_ise_x)),
  Y_ISE = c(mean(mf pca_test_ise_y),
            mean(gp_mean_ise_y),
            mean(naive_mean_ise_y)),
  Z_ISE = c(mean(mf pca_test_ise_z),
            mean(gp_mean_ise_z),
            mean(naive_mean_ise_z)),
  stringsAsFactors = FALSE
)

cat("\nGP vs MFPCA error decomposition (held-out test set)\n")

##
## GP vs MFPCA error decomposition (held-out test set)

cat(sprintf(" %-30s %10s %10s %10s\n", "Stage", "X ISE", "Y ISE", "Z ISE"))

##      Stage                                X ISE          Y ISE          Z ISE

cat(paste(rep("-", 68), collapse = ""), "\n")

## -----

for (i in seq_len(nrow(gp_mfpca_decomp))) {
  cat(sprintf(" %-30s %10.4f %10.4f %10.4f\n",

```

```

gp_mfpca_decomp$Stage[i],
gp_mfpca_decomp$X_ISE[i],
gp_mfpca_decomp$Y_ISE[i],
gp_mfpca_decomp$Z_ISE[i]))
}

```

```

## MFPCA true scores          1.3088    0.1125    0.7142
## GP scores                  3.7644    0.2807    2.7744
## Naive baseline             4.1377    0.3363    3.1275

```

```
cat(paste(rep("-", 68), collapse = ""), "\n")
```

```
## -----
```

```

cat(sprintf(" GP score uplift over MFPCA floor: X %+0.4f Y %+0.4f Z %+0.4f\n\n",
  mean(gp_mean_ise_x) - mean(mfpca_test_ise_x),
  mean(gp_mean_ise_y) - mean(mfpca_test_ise_y),
  mean(gp_mean_ise_z) - mean(mfpca_test_ise_z)))

```

```
## GP score uplift over MFPCA floor: X +2.4556 Y +0.1682 Z +2.0602
```

```

# -----
# Diagnostic: between-theta variance ratio in density space
# Small ratios indicate the density barely changes with theta, so the GP should
# learn a nearly flat conditional-mean surface.
# -----

```

```

density_var_ratio <- data.frame(
  Variable = c("X", "Y", "Z"),
  between_var = c(
    mean(rowSums(sweep(dens_mat_x, 2, colMeans(dens_mat_x))^2) * (diff(range(x_grid)) / (length(x_grid)
    mean(rowSums(sweep(dens_mat_y, 2, colMeans(dens_mat_y))^2) * (diff(range(y_grid)) / (length(y_grid)
    mean(rowSums(sweep(dens_mat_z, 2, colMeans(dens_mat_z))^2) * (diff(range(z_grid)) / (length(z_grid)
  )),
  total_var = c(
    mean(apply(dens_mat_x, 1, var)),
    mean(apply(dens_mat_y, 1, var)),
    mean(apply(dens_mat_z, 1, var))
  ),
  stringsAsFactors = FALSE
)
density_var_ratio$ratio <- density_var_ratio$between_var / pmax(density_var_ratio$total_var, 1e-10)
cat("Between-theta / total density variance ratio\n")

```

```
## Between-theta / total density variance ratio
```

```

cat(sprintf(" X = %.3f Y = %.3f Z = %.3f\n\n",
  density_var_ratio$ratio[1], density_var_ratio$ratio[2], density_var_ratio$ratio[3]))

```

```
## X = 2.339 Y = 0.627 Z = 1.882
```



```

z_test_recon_diag <- data.frame(
  theta = test_theta_vals,
  mfpcr_raw_ise = ise_z_raw_space[test_theta_idx],
  final_density_ise = ise_z_final_density[test_theta_idx],
  gp_mean_ise = gp_mean_ise_z,
  stringsAsFactors = FALSE
)
x_test_recon_diag <- data.frame(
  theta = test_theta_vals,
  mfpcr_raw_ise = ise_x_raw_space[test_theta_idx],
  final_density_ise = ise_x_final_density[test_theta_idx],
  gp_mean_ise = gp_mean_ise_x,
  stringsAsFactors = FALSE
)

# -----
# Gaussian CRPS
#  $CRPS(N(\mu, \sigma^2), y) = \sigma * [z*(2*\Phi(z)-1) + 2*\phi(z) - 1/\sqrt{\pi}]$ 
# where  $z = (y - \mu) / \sigma$ .
# Reduces to  $|y - \mu|$  when  $\sigma \rightarrow 0$  (deterministic forecast).
# Always  $\geq 0$ ; smaller is better.
# -----
gaussian_crps <- function(y, mu, sd) {
  sd <- pmax(sd, 1e-10)
  z <- (y - mu) / sd
  sd * (z * (2 * pnorm(z) - 1) + 2 * dnorm(z) - 1 / sqrt(pi))
}

# -----
# Integrated CRPS in raw density space
# For each grid point  $x_j$ , the pointwise CRPS is:
#  $CRPS_j = E|X - f_{true}(x_j)| - (1/2) E|X - X'|$ 
# averaged over the ensemble  $\{X_i = f_i(x_j)\}$ .
#
# The pair-distance term uses the sorted-order-statistics identity, giving
#  $O(N \log N)$  per grid point instead of  $O(N^2)$ :
#  $(1/N^2) \sum_{\{i,j\}} |x_i - x_j| = (2/N^2) \sum_i (2i - N - 1) * x_{(i)}$ 
# where  $x_{(i)}$  are the sorted ensemble values.
#
# Final result:  $\sum_j CRPS_j * \text{grid\_step}$  (units: same as  $ISE^{1/2}$ )
# -----
crps_integrated_func <- function(ensemble, y_obs, grid_step) {
  n_samp <- nrow(ensemble)
  n_grid <- ncol(ensemble)
  y_mat <- matrix(y_obs, nrow = n_samp, ncol = n_grid, byrow = TRUE)

  # Term 1: mean  $|X_i - y_{obs}|$  at each grid point (vectorised)
  term1_vec <- colMeans(abs(ensemble - y_mat))

  # Term 2: mean pairwise  $|X_i - X_j|$  via order-statistics trick ( $O(N \log N)$ )
  term2_vec <- apply(ensemble, 2, function(x) {
    xs <- sort(x)
    n <- length(xs)

```

```

    2 / n^2 * sum((2 * seq_len(n) - n - 1) * xs)
  })

  crps_vec <- term1_vec - 0.5 * term2_vec    # pointwise CRPS at each x_j
  sum(crps_vec) * grid_step                # integrate over domain
}

# =====
# Latent-space diagnostic metric: evaluates how accurately the GP predicts
# held-out PC scores at unseen theta values. This is not the primary
# density-space validation metric; iCRPS remains the main end-to-end criterion.
#
# For each held-out test theta and each PC component:
#   GP    predictive: N(GP posterior mean, v_k + sigma2_resid,k)
#   Naive predictive: N(training-average score, empirical training variance)
#
# Both forecasts are Gaussian, so the closed-form Gaussian CRPS
# formula applies. CRPS Skill Score = 1 - CRPS_GP / CRPS_Naive (> 0 = GP wins)
# =====

cat("\n", paste(rep("=", 70), collapse=""), "\n")

##
## =====

cat("Latent-space CRPS on held-out theta-level PC scores\n")

## Latent-space CRPS on held-out theta-level PC scores

cat(paste(rep("=", 70), collapse=""), "\n")

## =====

# Collect GP latent variance and restore the full LOESS residual variance.
# pred_test_var_mat is the score variance propagated through density space:
#   total_var_k(theta*) = v_k(theta*) + sigma2_resid_k
cat("Collecting GP latent variances and adding full LOESS residual variance...\n")

## Collecting GP latent variances and adding full LOESS residual variance...

pred_test_var_mat <- matrix(0, nrow = n_test_s, ncol = M_components)
pred_test_latent_var_mat <- matrix(0, nrow = n_test_s, ncol = M_components)
pred_test_obs_var_mat <- matrix(0, nrow = n_test_s, ncol = M_components)
for (pc_idx in 1:M_components) {
  model <- gp_models[[pc_idx]]
  pred_out <- get_gp_predictions(model, test_theta_norm, return_var = TRUE)
  if (is.null(pred_out$var_noise)) {
    pred_out$var_noise <- rep(0, n_test_s)
  }
}

```

```

if (is.null(pred_out$var_gp)) {
  pred_out$var_gp <- pred_out$var
}
pred_test_latent_var_mat[, pc_idx] <- pred_out$var_gp
pred_test_obs_var_mat[, pc_idx] <- rep(sigma2_resid_k[pc_idx], n_test_s)
}
pred_test_total_var_mat <- pred_test_latent_var_mat + pred_test_obs_var_mat
pred_test_var_mat <- pred_test_total_var_mat

draw_pc_samples <- function(gp_mu, score_var, n_samp) {
  total_sd <- sqrt(pmax(score_var, 1e-10))
  matrix(
    rnorm(n_samp * length(gp_mu),
          mean = rep(gp_mu, each = n_samp),
          sd = rep(total_sd, each = n_samp)),
    nrow = n_samp, ncol = length(gp_mu)
  )
}

CAL_FACTOR <- 1.0

latent_share <- pred_test_latent_var_mat / pmax(pred_test_total_var_mat, 1e-10)
cat("Density-space calibration factor disabled:\n")

```

Density-space calibration factor disabled:

```
cat(sprintf(" Shared c = %.2f (no post-hoc test-set tuning)\n", CAL_FACTOR))
```

Shared c = 1.00 (no post-hoc test-set tuning)

```
cat(sprintf(" Mean latent GP variance share vs latent+obs: %.1f%% [range %.1f%%, %.1f%%]\n",
  100 * mean(latent_share),
  100 * min(latent_share),
  100 * max(latent_share)))
```

Mean latent GP variance share vs latent+obs: 0.8% [range 0.5%, 1.6%]

```
cat(" Propagated score variance: latent GP v_k + full sigma2_resid,k\n")
```

Propagated score variance: latent GP v_k + full sigma2_resid,k

```

# =====
# PC-space variance diagnostic: size of learned latent uncertainty relative to
# the restored full residual variance.
# =====

```

```
variance_ratio <- pred_test_latent_var_mat / pmax(pred_test_obs_var_mat, 1e-10)
```

```
cat(sprintf(
  "\nVariance diagnostic (v_k(theta*) / sigma2_resid,k across %d components and test points):\n",
  M_components))
```

```
##
## Variance diagnostic (v_k(theta*) / sigma2_resid,k across 30 components and test points):

cat(sprintf(" Mean ratio = %.3f (median %.3f, range [%.3f, %.3f])\n",
            mean(variance_ratio), median(variance_ratio),
            min(variance_ratio), max(variance_ratio)))

## Mean ratio = 0.008 (median 0.008, range [0.005, 0.016])

# Naive predictive parameters (constant across test thetas)
naive_mean_vec <- mean_train_scores_vec # training-average score per PC
naive_var_vec <- apply(train_scores, 2, var) # training variance per PC
naive_sd_vec <- sqrt(pmax(naive_var_vec, 1e-10))

# Per-(test-theta, PC) Gaussian CRPS
crps_gp_mat <- matrix(0, nrow = n_test_s, ncol = M_components)
crps_naive_mat <- matrix(0, nrow = n_test_s, ncol = M_components)

for (i in seq_len(n_test_s)) {
  for (k in seq_len(M_components)) {
    col_k <- paste0("PC", k)
    y_k <- test_scores[i, col_k]

    # GP forecast
    mu_gp <- pred_test_df[i, col_k]
    sd_gp <- sqrt(max(pred_test_var_mat[i, k], 1e-10))
    crps_gp_mat[i, k] <- gaussian_crps(y_k, mu_gp, sd_gp)

    # Naive forecast
    crps_naive_mat[i, k] <- gaussian_crps(y_k, naive_mean_vec[k], naive_sd_vec[k])
  }
}

# Aggregate
mean_crps_gp_pc <- colMeans(crps_gp_mat)
mean_crps_naive_pc <- colMeans(crps_naive_mat)
overall_crps_gp <- mean(crps_gp_mat)
overall_crps_naive <- mean(crps_naive_mat)
crps_skill_pc <- 1 - mean_crps_gp_pc / mean_crps_naive_pc

cat("\n Overall mean CRPS in latent PC score space:\n")

##
## Overall mean CRPS in latent PC score space:

cat(sprintf(" GP : %.4f\n", overall_crps_gp))

## GP : 0.0362

cat(sprintf(" Naive : %.4f\n", overall_crps_naive))

## Naive : 0.0326
```

```
cat(sprintf(" CRPS Skill Score (1 - GP/Naive): %.4f  [> 0 means GP better]\n\n",
            1 - overall_crps_gp / overall_crps_naive))
```

```
## CRPS Skill Score (1 - GP/Naive): -0.1103  [> 0 means GP better]
```

```
cat(" Mean CRPS per PC (first 10 shown):\n")
```

```
## Mean CRPS per PC (first 10 shown):
```

```
cat(sprintf(" %-6s %8s %8s %8s\n", "PC", "GP", "Naive", "Skill"))
```

```
## PC GP Naive Skill
```

```
cat(paste(rep("-", 38), collapse = ""), "\n")
```

```
## -----
```

```
for (k in seq_len(min(10, M_components))) {
  cat(sprintf(" PC%-3d %8.4f %8.4f %+8.3f\n",
              k, mean_crps_gp_pc[k], mean_crps_naive_pc[k], crps_skill_pc[k]))
}
```

```
## PC1 0.0964 0.1172 +0.177
## PC2 0.0600 0.0429 -0.397
## PC3 0.0541 0.0472 -0.145
## PC4 0.0504 0.0384 -0.313
## PC5 0.0466 0.0571 +0.184
## PC6 0.0486 0.0399 -0.219
## PC7 0.0418 0.0435 +0.039
## PC8 0.0425 0.0397 -0.070
## PC9 0.0406 0.0278 -0.459
## PC10 0.0405 0.0233 -0.734
```

```
cat(paste(rep("=", 70), collapse=""), "\n\n")
```

```
## =====
```

```
# --- Plot: CRPS by PC ---
```

```
par(mfrow = c(1, 2), mar = c(5, 4, 4, 1), oma = c(0, 0, 3, 0))
```

```
# Left: mean CRPS per PC (side-by-side bars)
```

```
ylim_crps <- c(0, max(c(mean_crps_gp_pc, mean_crps_naive_pc)) * 1.1)
```

```
barplot(rbind(mean_crps_gp_pc, mean_crps_naive_pc),
        beside = TRUE,
        col = c("steelblue", "tomato"),
        names.arg = paste0("PC", seq_len(M_components)),
        las = 2, cex.names = 0.55,
        ylab = "Mean CRPS (lower = better)",
        main = "Mean CRPS per PC",
```

```

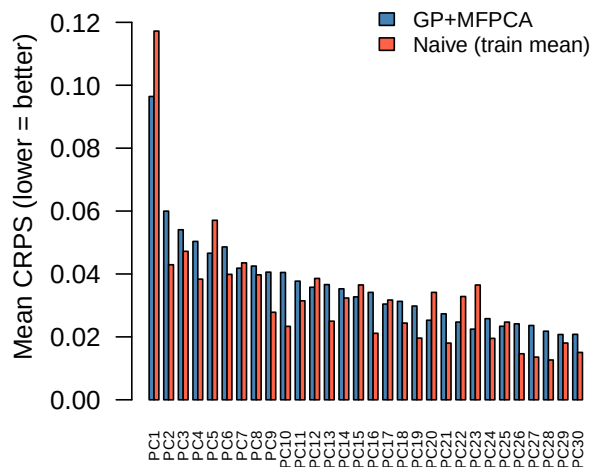
ylim      = ylim_crps,
legend.text = c("GP+MFPCA", "Naive (train mean)",
args.legend = list(x = "topright", bty = "n", cex = 0.8))

# Right: distribution of per-theta average CRPS (boxplot)
crps_gp_per_theta    <- rowMeans(crps_gp_mat)
crps_naive_per_theta  <- rowMeans(crps_naive_mat)
boxplot(list("GP+MFPCA" = crps_gp_per_theta,
            "Naive\n(train avg)" = crps_naive_per_theta),
        col      = c("steelblue", "tomato"),
        ylab      = "Mean CRPS across all PCs",
        main      = "CRPS Distribution\nacross Test Theta Values",
        outline = FALSE)
stripchart(list(crps_gp_per_theta, crps_naive_per_theta),
            vertical = TRUE, method = "jitter", add = TRUE,
            pch = 16, cex = 0.5, col = rgb(0, 0, 0, 0.3))
abline(h = overall_crps_naive, col = "tomato", lty = 2, lwd = 1.5)
abline(h = overall_crps_gp, col = "steelblue", lty = 2, lwd = 1.5)
mtext("CRPS: GP Emulator vs Naive Baseline (PC Score Space)",
      outer = TRUE, cex = 1.1, font = 2)

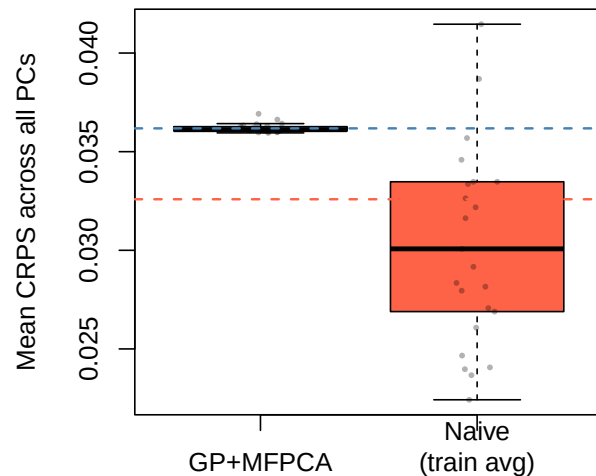
```

CRPS: GP Emulator vs Naive Baseline (PC Score Space)

Mean CRPS per PC



CRPS Distribution across Test Theta Values



```

# =====
# Primary density-space validation metric for distribution prediction.
# iCRPS evaluates the full predictive ensemble in the original density space.
# A single score posterior is drawn per held-out theta and decoded into X, Y, Z.
# =====

cat("\n", paste(rep("=", 70), collapse=""), "\n")

```

```

##
## =====

cat("Density-space evaluation from one shared PC-score posterior per theta\n")

## Density-space evaluation from one shared PC-score posterior per theta

cat(paste(rep("=", 70), collapse=""), "\n")

## =====

N_SAMP <- 100 # ensemble size for density-space evaluation
set.seed(123)

icrps_gp_x <- icrps_naive_x <- numeric(n_test_s)
icrps_gp_y <- icrps_naive_y <- numeric(n_test_s)
icrps_gp_z <- icrps_naive_z <- numeric(n_test_s)

dx <- diff(range(x_grid)) / (length(x_grid) - 1)
dy <- diff(range(y_grid)) / (length(y_grid) - 1)
dz <- diff(range(z_grid)) / (length(z_grid) - 1)

cat(sprintf("Drawing %d samples per test theta (%d total density reconstructions)...\n",
            N_SAMP, N_SAMP * n_test_s * 2))

## Drawing 100 samples per test theta (5000 total density reconstructions)...

for (i in seq_len(n_test_s)) {
  ref_row <- test_theta_idx[i]
  true_x <- dens_mat_x[ref_row, ]
  true_y <- dens_mat_y[ref_row, ]
  true_z <- dens_mat_z[ref_row, ]

  gp_mu <- as.numeric(pred_test_df[i, paste0("PC", seq_len(M_components))])
  gp_var <- pmax(pred_test_var_mat[i, ], 1e-10)
  gp_samples <- draw_pc_samples(gp_mu, gp_var, N_SAMP)

  # Naive samples: draws from N(grand_mean, diag(training_var))
  nav_samples <- matrix(
    rnorm(N_SAMP * M_components,
          mean = rep(naive_mean_vec, each = N_SAMP),
          sd = rep(naive_sd_vec, each = N_SAMP)),
    nrow = N_SAMP, ncol = M_components)

  # Reconstruct both ensembles to density space
  ens_gp <- reconstruct_densities(gp_samples, mu_x, mu_y, mu_z,
                                  phi_x, phi_y, phi_z, x_grid, y_grid, z_grid,
                                  x_peak_loc = rep(pred_peak_x_test[i], N_SAMP),
                                  x_peak_width = rep(pred_width_x_test[i], N_SAMP),
                                  x_peak_left_width = rep(pred_left_width_x_test[i], N_SAMP),
                                  x_peak_right_width = rep(pred_right_width_x_test[i], N_SAMP),

```

```

        z_peak_loc = rep(pred_peak_z_test[i], N_SAMP),
        z_peak_width = rep(pred_width_z_test[i], N_SAMP),
        z_peak_left_width = rep(pred_left_width_z_test[i], N_SAMP),
        z_peak_right_width = rep(pred_right_width_z_test[i], N_SAMP))
ens_nav <- reconstruct_densities(nav_samples, mu_x, mu_y, mu_z,
                                phi_x, phi_y, phi_z, x_grid, y_grid, z_grid,
                                x_peak_loc = rep(mean_train_peak_x_loc, N_SAMP),
                                x_peak_width = rep(mean_train_peak_x_width, N_SAMP),
                                x_peak_left_width = rep(mean_train_peak_x_left_width, N_SAMP),
                                x_peak_right_width = rep(mean_train_peak_x_right_width, N_SAMP),
                                z_peak_loc = rep(mean_train_peak_z_loc, N_SAMP),
                                z_peak_width = rep(mean_train_peak_z_width, N_SAMP),
                                z_peak_left_width = rep(mean_train_peak_z_left_width, N_SAMP),
                                z_peak_right_width = rep(mean_train_peak_z_right_width, N_SAMP))

# (Peak-shift correction removed: worsens ISE for multimodal X and Z)

# Integrated CRPS in raw density space
icrps_gp_x[i] <- crps_integrated_func(ens_gp$x, true_x, dx)
icrps_gp_y[i] <- crps_integrated_func(ens_gp$y, true_y, dy)
icrps_gp_z[i] <- crps_integrated_func(ens_gp$z, true_z, dz)
icrps_naive_x[i] <- crps_integrated_func(ens_nav$x, true_x, dx)
icrps_naive_y[i] <- crps_integrated_func(ens_nav$y, true_y, dy)
icrps_naive_z[i] <- crps_integrated_func(ens_nav$z, true_z, dz)

if (i %% 5 == 0 || i == n_test_s)
  cat(sprintf(" test theta %d / %d done\n", i, n_test_s))
}

```

```

## test theta 5 / 25 done
## test theta 10 / 25 done
## test theta 15 / 25 done
## test theta 20 / 25 done
## test theta 25 / 25 done

```

```
cat("\n", paste(rep("=", 70), collapse=""), "\n")
```

```

##
## =====

```

```
cat("Integrated CRPS summary in density space\n")
```

```
## Integrated CRPS summary in density space
```

```
cat(paste(rep("=", 70), collapse=""), "\n")
```

```
## =====
```

```
cat(" iCRPS = integral_x CRPS(F_x, f_true(x)) dx [units match sqrt(ISE)]\n\n")
```

```
## iCRPS = integral_x CRPS(F_x, f_true(x)) dx [units match sqrt(ISE)]
```



```
cat(sprintf(" %-18s %8s %8s %7s %+7s\n",
            "Variable", "GP", "Naive", "Ratio", "Skill"))
```

```
##      Variable                GP      Naive      Ratio      Skill
```

```
cat(paste(rep("-", 58), collapse=""), "\n")
```

```
## -----
```

```
for (info in list(
  list(gp = icrps_gp_x, mp = icrps_naive_x, nm = "X (Log Period)"),
  list(gp = icrps_gp_y, mp = icrps_naive_y, nm = "Y (Log Duration)"),
  list(gp = icrps_gp_z, mp = icrps_naive_z, nm = "Z (Log Hill)")) {
  ratio <- mean(info$gp) / mean(info$mp)
  skill <- 1 - ratio
  cat(sprintf(" %-18s %8.4f %8.4f %7.3f %+7.3f\n",
              info$nm, mean(info$gp), mean(info$mp), ratio, skill))
}
```

```
##      X (Log Period)          0.8371      1.0029      0.835      +0.165
##      Y (Log Duration)        0.3036      0.3990      0.761      +0.239
##      Z (Log Hill)            0.7634      0.9404      0.812      +0.188
```

```
cat(paste(rep("=", 70), collapse=""), "\n\n")
```

```
## =====
```

```
# -----
# Mean-only density evaluation
# -----
cat("Mean-only density evaluation (posterior mean reconstruction only)\n")
```

```
## Mean-only density evaluation (posterior mean reconstruction only)
```

```
cat(sprintf(" %-18s %8s %8s %8s %8s\n",
            "Variable", "GP ISE", "Naive ISE", "GP L2", "Naive L2"))
```

```
##      Variable                GP ISE  Naive ISE      GP L2  Naive L2
```

```
cat(paste(rep("-", 68), collapse = ""), "\n")
```

```
## -----
```

```
for (info in list(
  list(gp_ise = gp_mean_ise_x, nv_ise = naive_mean_ise_x, gp_l2 = gp_mean_l2_x, nv_l2 = naive_mean_l2_x),
  list(gp_ise = gp_mean_ise_y, nv_ise = naive_mean_ise_y, gp_l2 = gp_mean_l2_y, nv_l2 = naive_mean_l2_y),
  list(gp_ise = gp_mean_ise_z, nv_ise = naive_mean_ise_z, gp_l2 = gp_mean_l2_z, nv_l2 = naive_mean_l2_z)) {
  cat(sprintf(" %-18s %8.4f %8.4f %8.4f %8.4f\n",
              info$nm, mean(info$gp_ise), mean(info$nv_ise),
              mean(info$gp_l2), mean(info$nv_l2)))
}
```

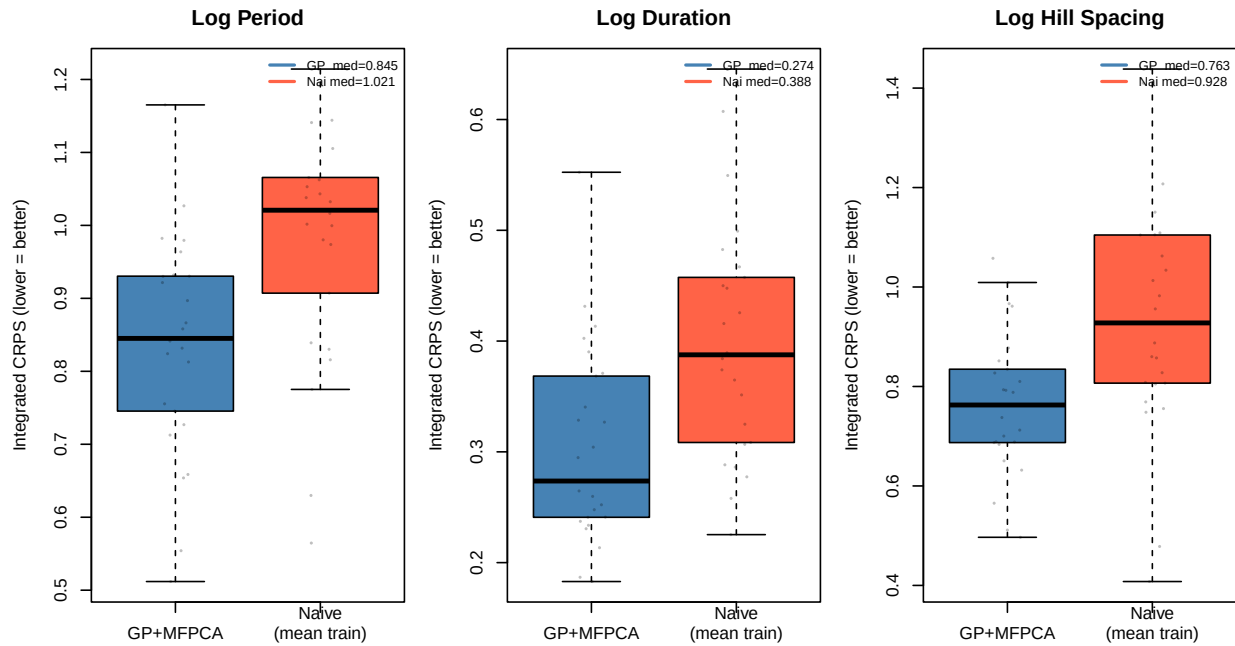
```
## X (Log Period)      3.7644    4.1377    1.8530    1.9264
## Y (Log Duration)   0.2807    0.3363    0.5045    0.5636
## Z (Log Hill)       2.7744    3.1275    1.5401    1.6155
```

```
cat(paste(rep("=", 70), collapse = ""), "\n\n")
```

```
## =====
```

```
# --- Plot: Integrated CRPS boxplots (raw density space) ---
par(mfrow = c(1, 3), mar = c(5, 4, 3, 1), oma = c(0, 0, 3, 0))
for (info in list(
  list(gp = icrps_gp_x, mp = icrps_naive_x, nm = "Log Period"),
  list(gp = icrps_gp_y, mp = icrps_naive_y, nm = "Log Duration"),
  list(gp = icrps_gp_z, mp = icrps_naive_z, nm = "Log Hill Spacing"))) {
  boxplot(list("GP+MFPCA" = info$gp, "Naive\n(mean train)" = info$mp),
    col      = c("steelblue", "tomato"),
    main     = info$nm,
    ylab     = "Integrated CRPS (lower = better)",
    outline  = FALSE)
  stripchart(list(info$gp, info$mp),
    vertical = TRUE, method = "jitter", add = TRUE,
    pch = 16, cex = 0.4, col = rgb(0, 0, 0, 0.25))
  legend("topright",
    legend = c(sprintf("GP med=%.3f", median(info$gp)),
      sprintf("Nai med=%.3f", median(info$mp))),
    col = c("steelblue", "tomato"), lty = 1, lwd = 2,
    bty = "n", cex = 0.8)
}
mtext("Integrated CRPS: GP Emulator vs Naive Baseline (Raw Density Space)",
  outer = TRUE, cex = 1.1, font = 2)
```

Integrated CRPS: GP Emulator vs Naive Baseline (Raw Density Space)



```
# =====
# 9. POSTERIOR PREDICTIVE DISTRIBUTION AT HELD-OUT THETA VALUES
# =====
# Goal: predict the full distribution of planetary statistics at new theta values.
#
# The GP emulator provides a Bayesian posterior over PC scores:
#  $PC_k | \theta_{new}, data \sim N(\mu_k(\theta_{new}), \sigma^2_k(\theta_{new}))$ 
# Propagating this through the MFPCA inverse transform gives a posterior
# predictive distribution over density functions:
#  $f(x) | \theta_{new}, data \sim MFPCA^{-1}(GP \text{ posterior samples})$ 
#
# For each held-out theta we:
# 1. Draw  $N_{POST}$  samples from the GP posterior (epistemic uncertainty only)
# 2. Map each sample back to density space by squaring and normalizing
# 3. Compute pointwise posterior mean + 95% credible band
# 4. Measure density-level coverage: fraction of x-grid where truth is in band
# =====

# --- Step 1: Draw posterior predictive density ensemble for all held-out thetas ---
N_POST <- 500 # posterior draws per held-out theta for density bands
CI_PROB <- 0.95 # credible band probability
alpha <- 1 - CI_PROB
set.seed(42)

cat(sprintf("\nDrawing %d posterior predictive samples for %d test theta values...\n",
            N_POST, n_test_s))
```

##

```
## Drawing 500 posterior predictive samples for 25 test theta values...
```

```
# Storage: list of per-theta ensembles (avoids large 3-D array)
```

```
post_ens <- vector("list", n_test_s)
```

```
for (i in seq_len(n_test_s)) {  
  gp_mu <- as.numeric(pred_test_df[i, paste0("PC", seq_len(M_components))])  
  gp_var <- pmax(pred_test_var_mat[i, ], 1e-10)  
  pc_samp <- draw_pc_samples(gp_mu, gp_var, N_POST)  
  
  ens <- reconstruct_densities(pc_samp, mu_x, mu_y, mu_z,  
                               phi_x, phi_y, phi_z, x_grid, y_grid, z_grid,  
                               x_peak_loc = rep(pred_peak_x_test[i], N_POST),  
                               x_peak_width = rep(pred_width_x_test[i], N_POST),  
                               x_peak_left_width = rep(pred_left_width_x_test[i], N_POST),  
                               x_peak_right_width = rep(pred_right_width_x_test[i], N_POST),  
                               z_peak_loc = rep(pred_peak_z_test[i], N_POST),  
                               z_peak_width = rep(pred_width_z_test[i], N_POST),  
                               z_peak_left_width = rep(pred_left_width_z_test[i], N_POST),  
                               z_peak_right_width = rep(pred_right_width_z_test[i], N_POST))  
  
  # (Peak-shift correction removed: worsens ISE for multimodal X and Z)  
  
  # Pointwise posterior mean and credible band  
  post_ens[[i]] <- list(  
    # Pointwise posterior summaries for the reconstructed density ensemble.  
    mean_x = colMeans(ens$x),  
    lower_x = apply(ens$x, 2, quantile, alpha / 2),  
    upper_x = apply(ens$x, 2, quantile, 1 - alpha / 2),  
    mean_y = colMeans(ens$y),  
    lower_y = apply(ens$y, 2, quantile, alpha / 2),  
    upper_y = apply(ens$y, 2, quantile, 1 - alpha / 2),  
    mean_z = colMeans(ens$z),  
    lower_z = apply(ens$z, 2, quantile, alpha / 2),  
    upper_z = apply(ens$z, 2, quantile, 1 - alpha / 2)  
  )  
  if (i %% 5 == 0 || i == n_test_s)  
    cat(sprintf("  theta %d / %d done\n", i, n_test_s))  
}
```

```
##  theta 5 / 25 done  
##  theta 10 / 25 done  
##  theta 15 / 25 done  
##  theta 20 / 25 done  
##  theta 25 / 25 done
```

```
# --- Step 2: Density-level coverage ---
```

```
# For each test theta: fraction of grid points where truth lies in the 95% band.
```

```
# This is the Bayesian analogue of interval coverage but in function space.
```

```
cov_x <- cov_y <- cov_z <- numeric(n_test_s)
```

```
for (i in seq_len(n_test_s)) {  
  ref_row <- test_theta_idx[i]  
  pe <- post_ens[[i]]
```

```

cov_x[i] <- mean(dens_mat_x[ref_row, ] >= pe$lower_x &
               dens_mat_x[ref_row, ] <= pe$upper_x)
cov_y[i] <- mean(dens_mat_y[ref_row, ] >= pe$lower_y &
               dens_mat_y[ref_row, ] <= pe$upper_y)
cov_z[i] <- mean(dens_mat_z[ref_row, ] >= pe$lower_z &
               dens_mat_z[ref_row, ] <= pe$upper_z)
}
cat(sprintf("\n%d%% Posterior Predictive Credible Band Coverage:\n", round(CI_PROB * 100)))

```

```

##
## 95% Posterior Predictive Credible Band Coverage:

```

```

cat(sprintf("  Log Period:      mean=%.1f%% median=%.1f%%\n",
            mean(cov_x)*100, median(cov_x)*100))

```

```

##   Log Period:      mean=94.1% median=95.5%

```

```

cat(sprintf("  Log Duration:    mean=%.1f%% median=%.1f%%\n",
            mean(cov_y)*100, median(cov_y)*100))

```

```

##   Log Duration:    mean=84.9% median=85.0%

```

```

cat(sprintf("  Log Hill Spacing: mean=%.1f%% median=%.1f%%\n",
            mean(cov_z)*100, median(cov_z)*100))

```

```

##   Log Hill Spacing: mean=94.0% median=93.5%

```

```

cat("  (Ideal: coverage close to the nominal level at each observable.)\n")

```

```

##   (Ideal: coverage close to the nominal level at each observable.)

```

```

# --- Step 3: Visualization ---
# Each held-out theta produces a 3-panel figure:
#   - Dark grey solid: KDE ground truth at that theta
#   - Dark orange dot-dash: MFPCA reconstruction using true held-out scores
#   - Purple dashed: Naive baseline (MFPCA reconstruction from grand mean)
#   - Steelblue solid: GP posterior mean density
#   - Light blue band: 95% pointwise posterior predictive credible interval
#
plot_prediction_bayes <- function(i) {

  sigma_val <- round(test_theta_vals[i], 3)
  ref_row   <- test_theta_idx[i]
  pe        <- post_ens[[i]]

  # Colour palette
  col_truth <- "gray20"
  col_mfpca <- "#D95F02"
  col_naive <- "#7B6D8D"

```

```

col_gp      <- "steelblue"
col_band    <- rgb(0.27, 0.51, 0.71, 0.20)  # translucent steelblue

par(mfrow = c(3, 1), mar = c(4, 4, 3, 1), oma = c(0, 0, 3, 0))

draw_panel <- function(grid, truth, mfpca, naive, gp_mean, lo, hi, xlab, cov_i) {
  yr <- range(c(truth, hi, mfpca, naive, gp_mean), na.rm = TRUE)
  yr[1] <- max(0, yr[1])

  plot(grid, truth, type = "l", col = col_truth, lwd = 3,
        xlab = xlab, ylab = "Density", ylim = yr, main = "")

  # 95% credible band (Bayesian posterior predictive)
  polygon(c(grid, rev(grid)), c(lo, rev(hi)),
          col = col_band, border = NA)

  lines(grid, mfpca, col = col_mfpca, lwd = 2, lty = 3)
  lines(grid, naive, col = col_naive, lwd = 2, lty = 2)
  lines(grid, gp_mean, col = col_gp, lwd = 2)

  mtext(sprintf("%s [band cov: %.0f%%]", xlab, cov_i * 100),
        side = 3, line = 0.3, cex = 0.85)

  legend("topright",
        c("KDE truth", "MFPCA recon", "Naive (mean train)",
          "GP posterior mean", sprintf("%.0f%% credible band", CI_PROB * 100)),
        col = c(col_truth, col_mfpca, col_naive, col_gp, col_band),
        lwd = c(3, 2, 2, 2, 8),
        lty = c(1, 3, 2, 1, 1),
        cex = 0.72, bg = "white", bty = "o")
}

draw_panel(x_grid,
  dens_mat_x[ref_row, ], recon_dens_x_true[ref_row, ], mean_train_dens_x,
  pe$mean_x, pe$lower_x, pe$upper_x,
  "Log Period Ratio", cov_x[i])

draw_panel(y_grid,
  dens_mat_y[ref_row, ], recon_dens_y_true[ref_row, ], mean_train_dens_y,
  pe$mean_y, pe$lower_y, pe$upper_y,
  "Log Duration Ratio", cov_y[i])

draw_panel(z_grid,
  dens_mat_z[ref_row, ], recon_dens_z_true[ref_row, ], mean_train_dens_z,
  pe$mean_z, pe$lower_z, pe$upper_z,
  "Log Hill Spacing", cov_z[i])

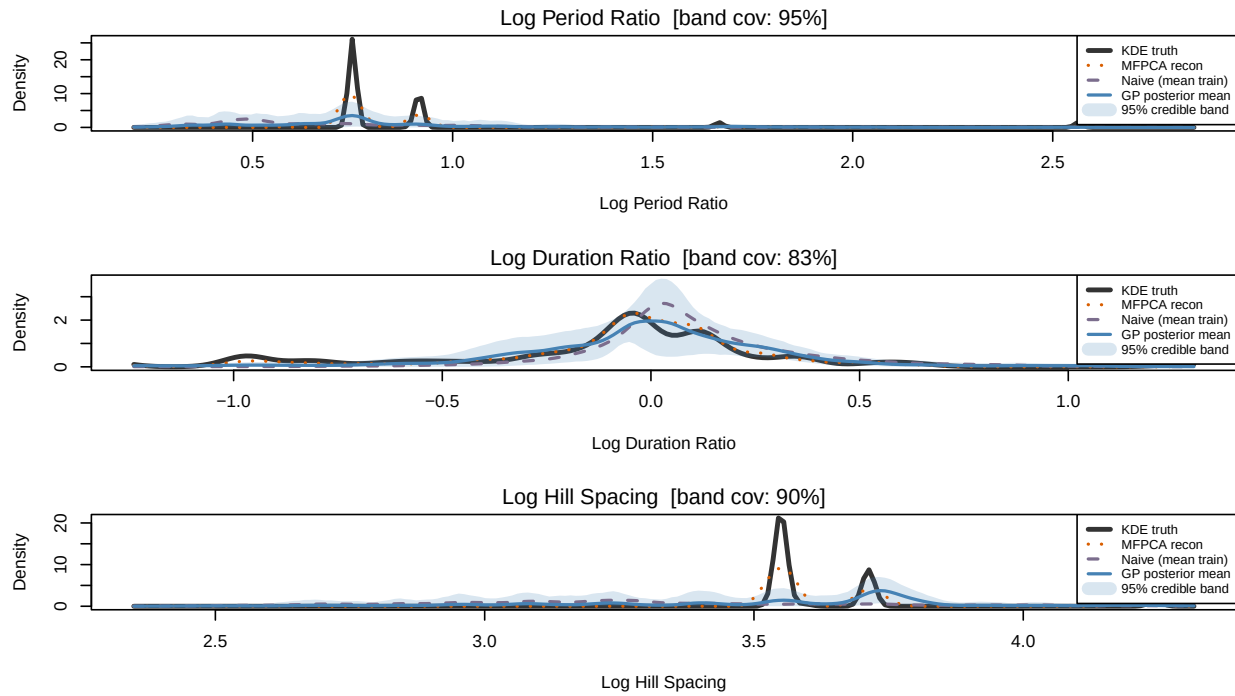
mtext(bquote("Posterior Predictive Distribution at Held-Out" ~
  sigma[z] == .(sigma_val)),
  outer = TRUE, cex = 1.1, font = 2)
}

# Produce one 3-panel figure per held-out test theta

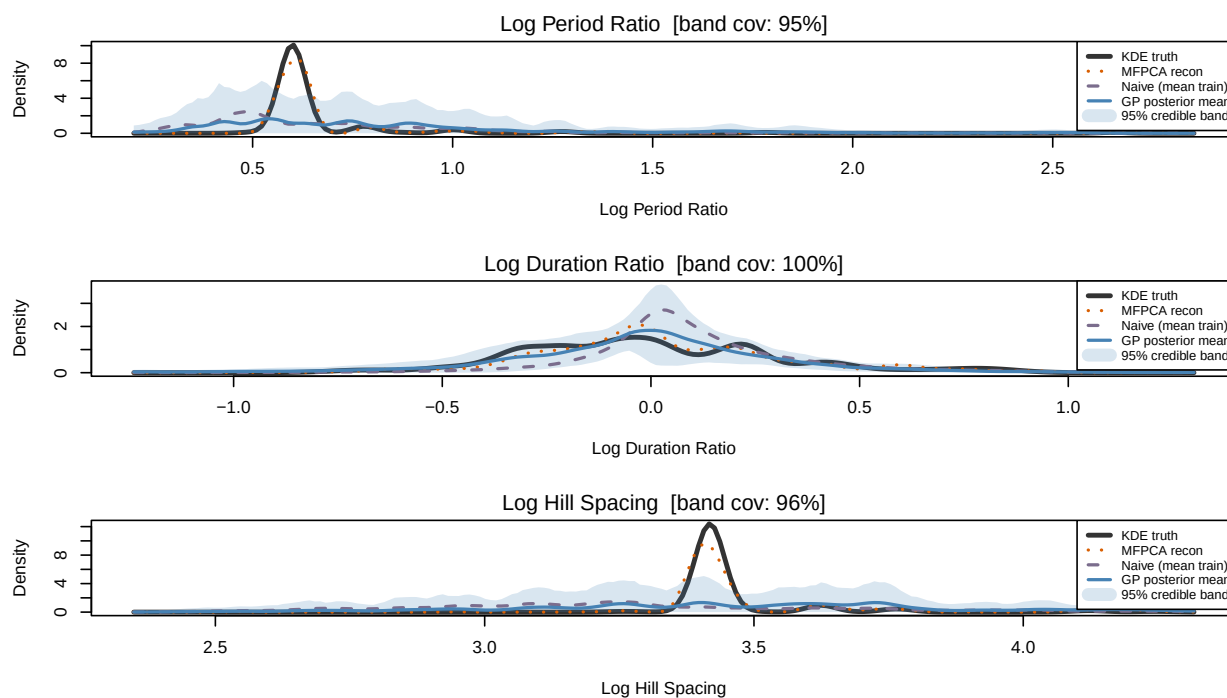
```

```
for (i in seq_along(test_theta_vals)) {
  plot_prediction_bayes(i)
}
```

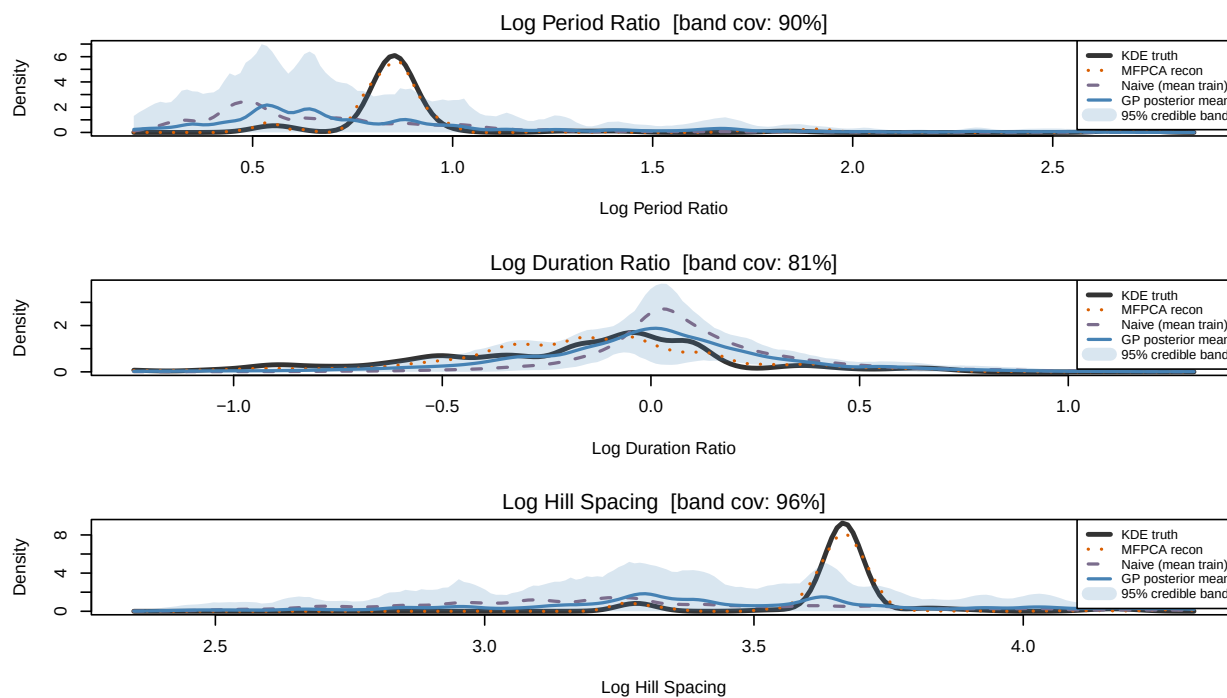
Posterior Predictive Distribution at Held-Out $\sigma_z = 26.697$



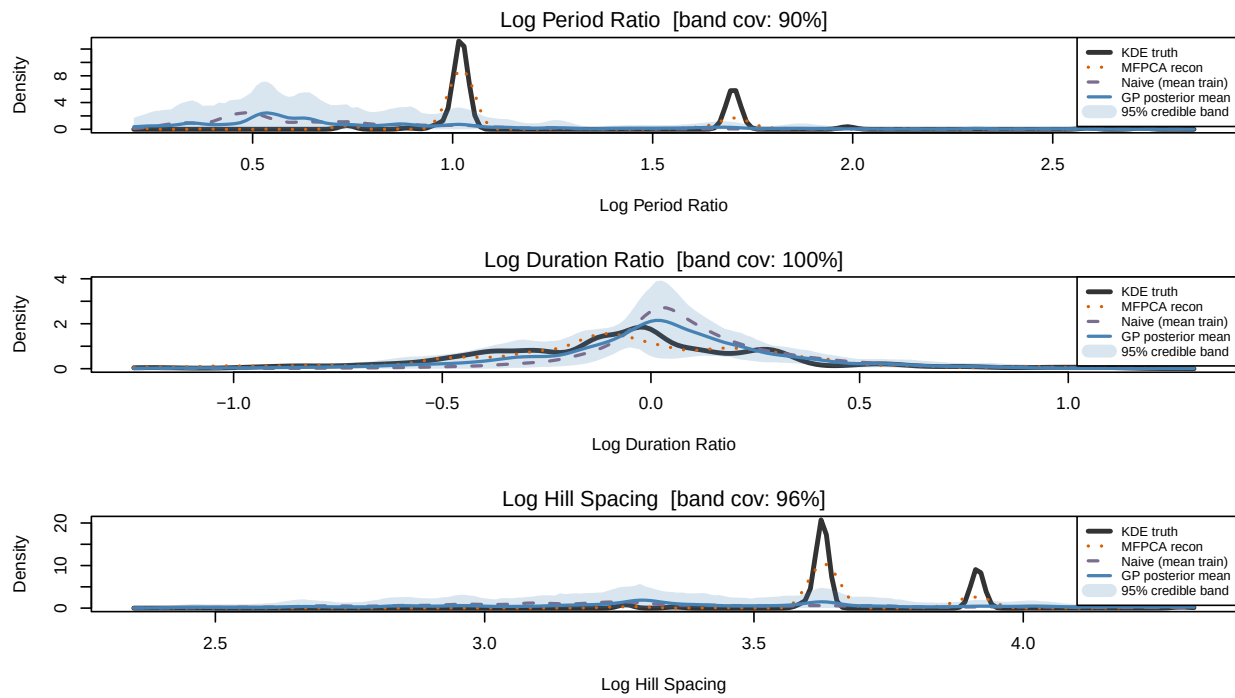
Posterior Predictive Distribution at Held-Out $\sigma_z = 32.725$



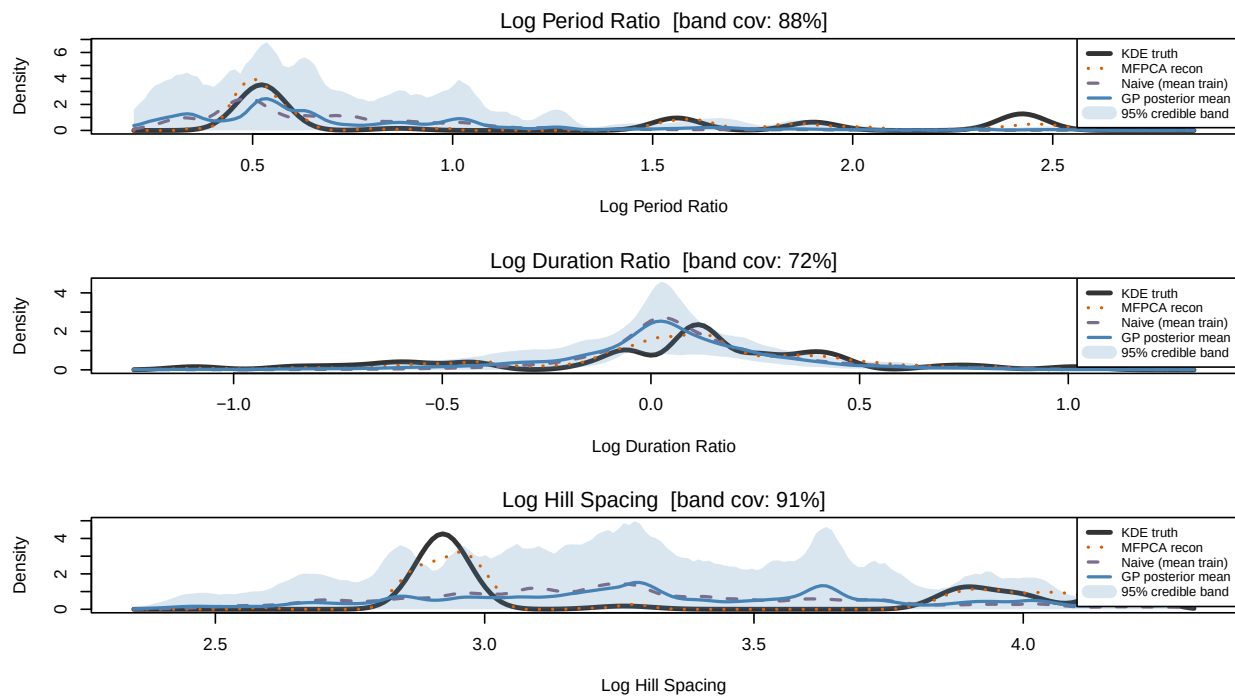
Posterior Predictive Distribution at Held-Out $\sigma_z = 44.276$



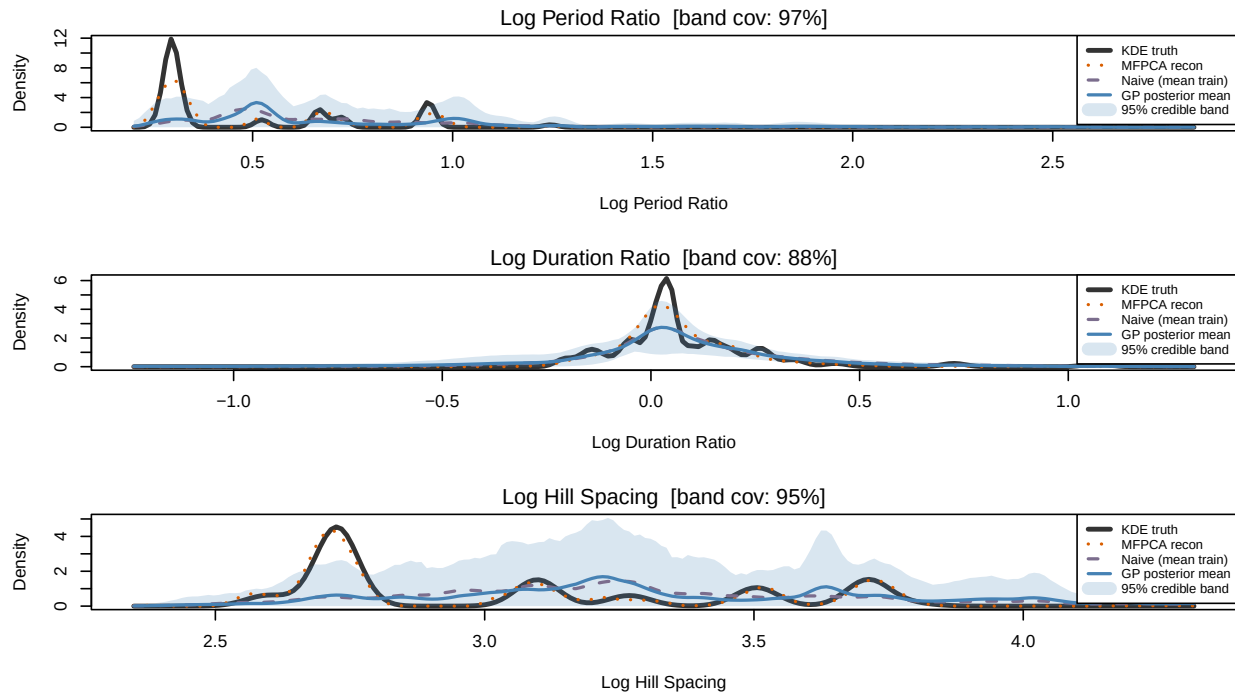
Posterior Predictive Distribution at Held-Out $\sigma_z = 48.201$



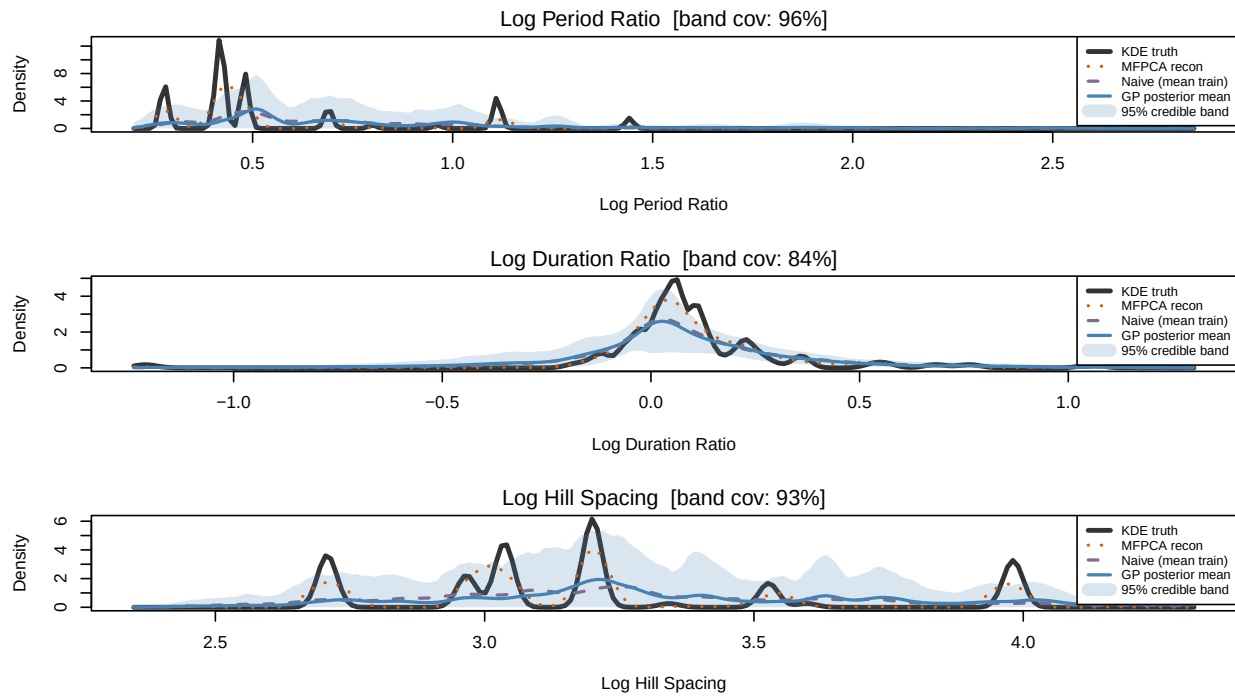
Posterior Predictive Distribution at Held-Out $\sigma_z = 50.736$



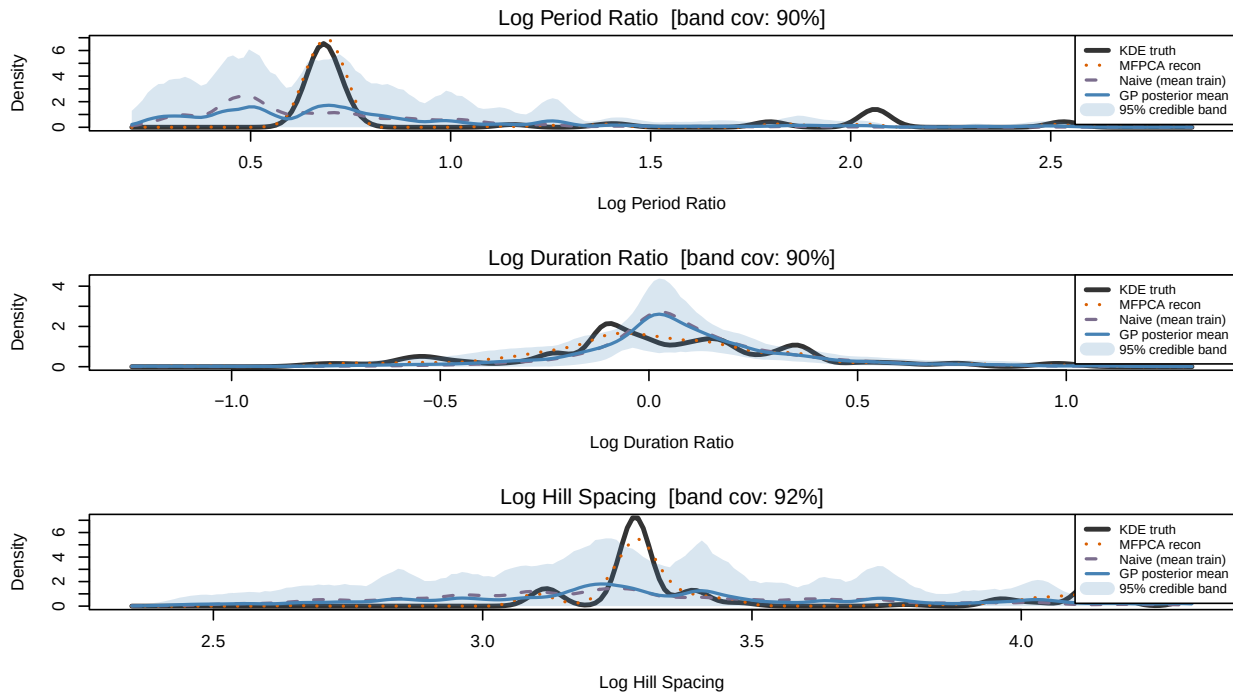
Posterior Predictive Distribution at Held-Out $\sigma_z = 55.651$



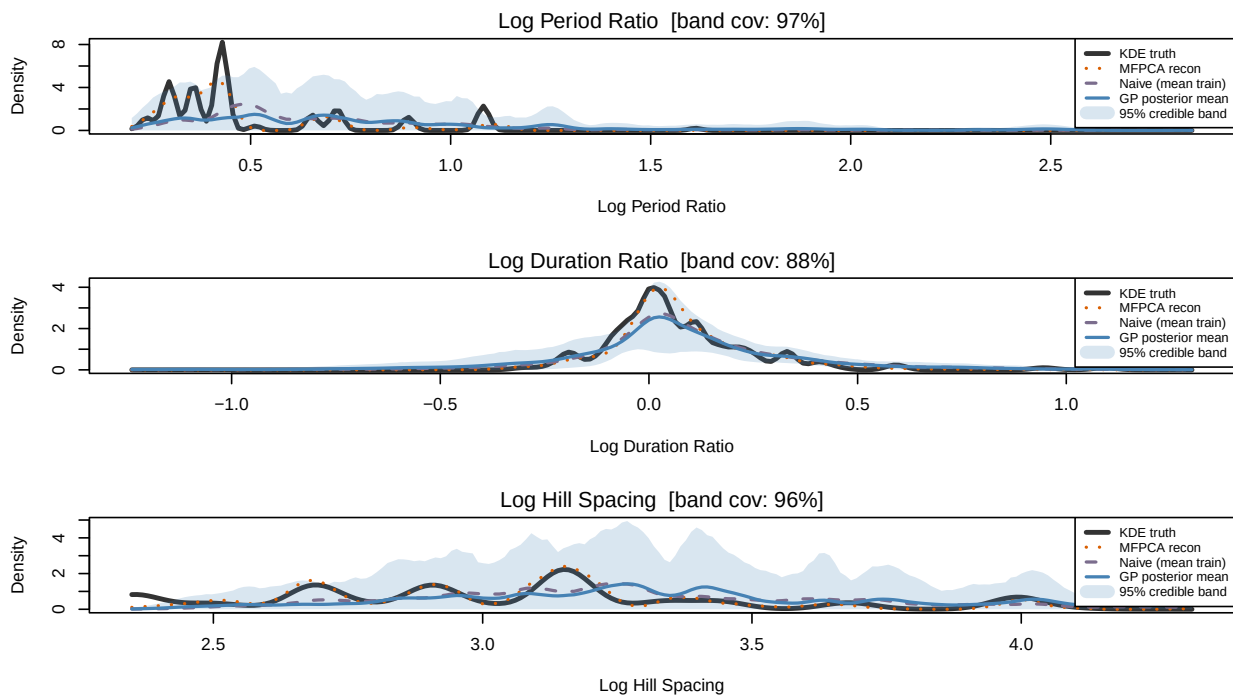
Posterior Predictive Distribution at Held-Out $\sigma_z = 59.178$



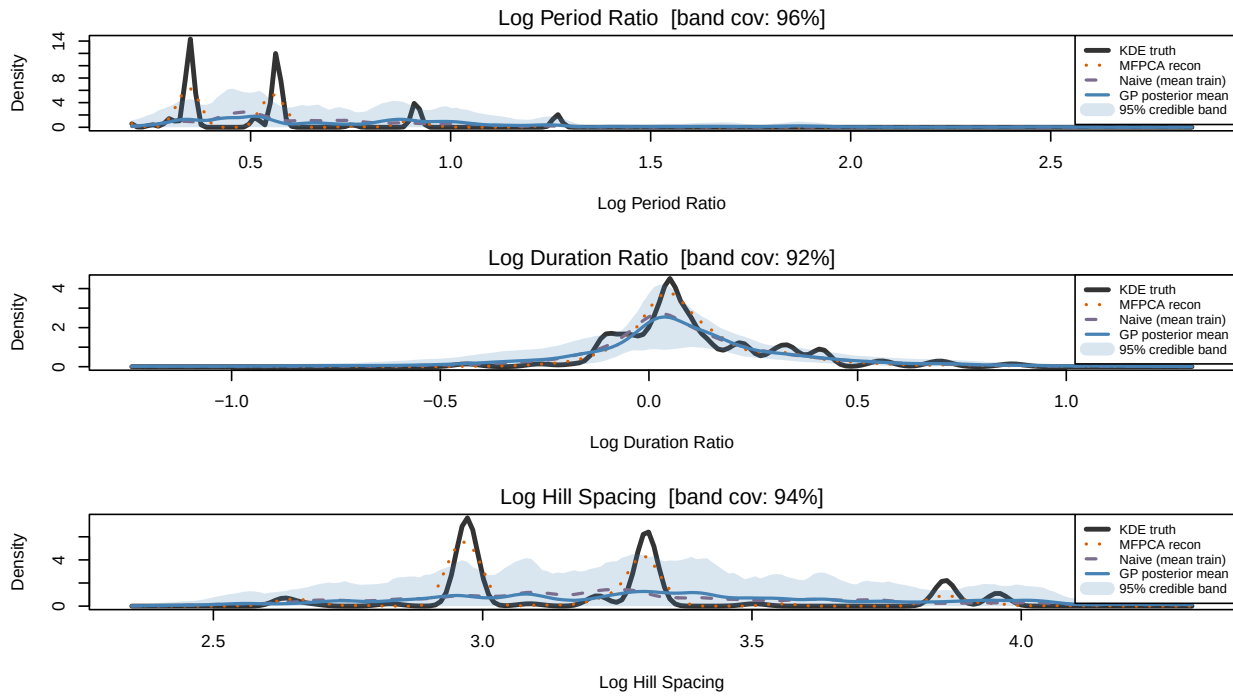
Posterior Predictive Distribution at Held-Out $\sigma_z = 63.265$



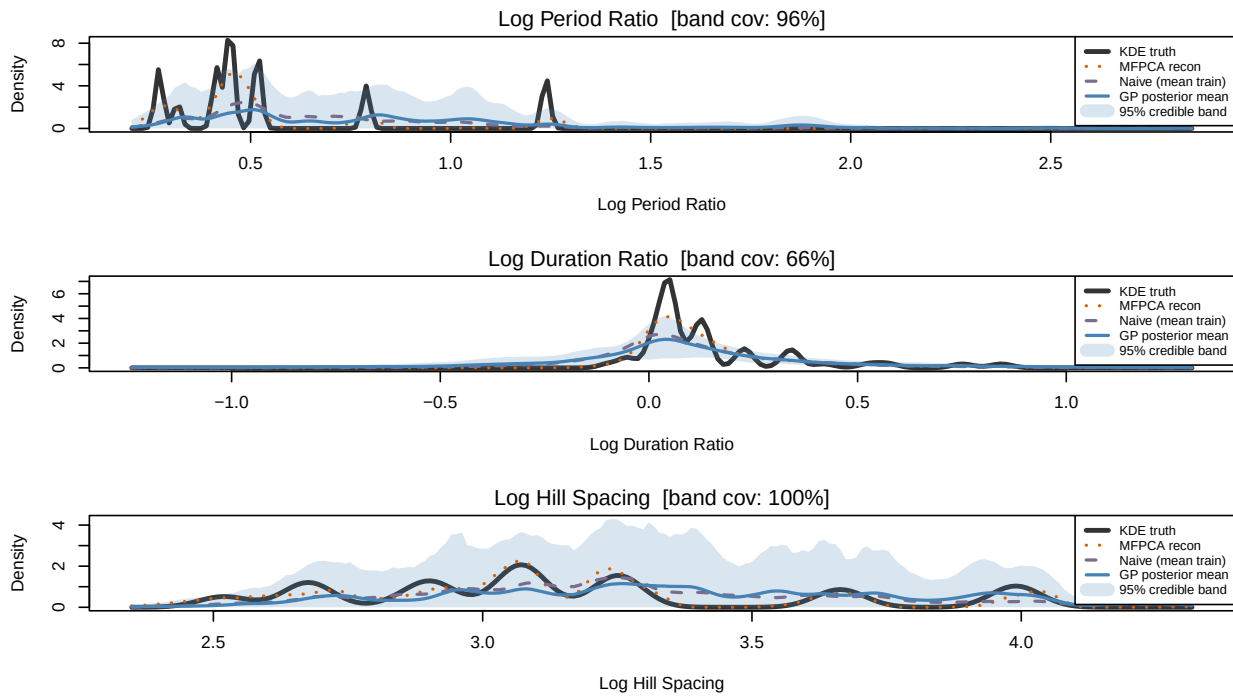
Posterior Predictive Distribution at Held-Out $\sigma_z = 65.874$



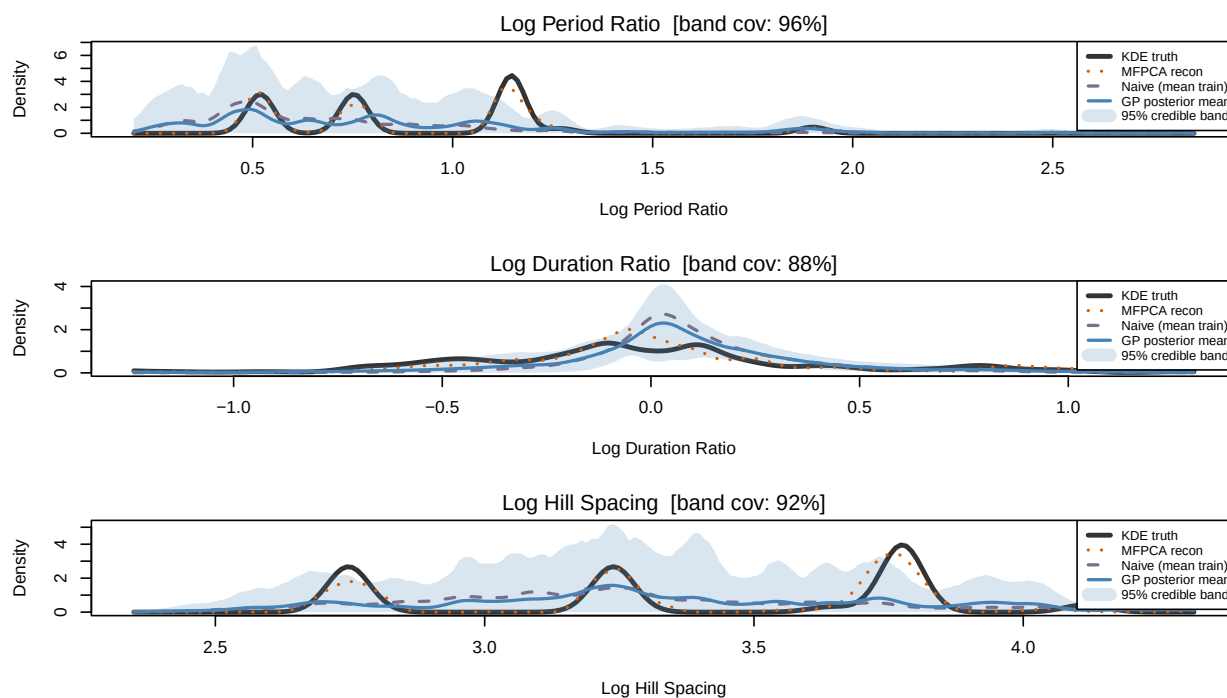
Posterior Predictive Distribution at Held-Out $\sigma_z = 70.365$



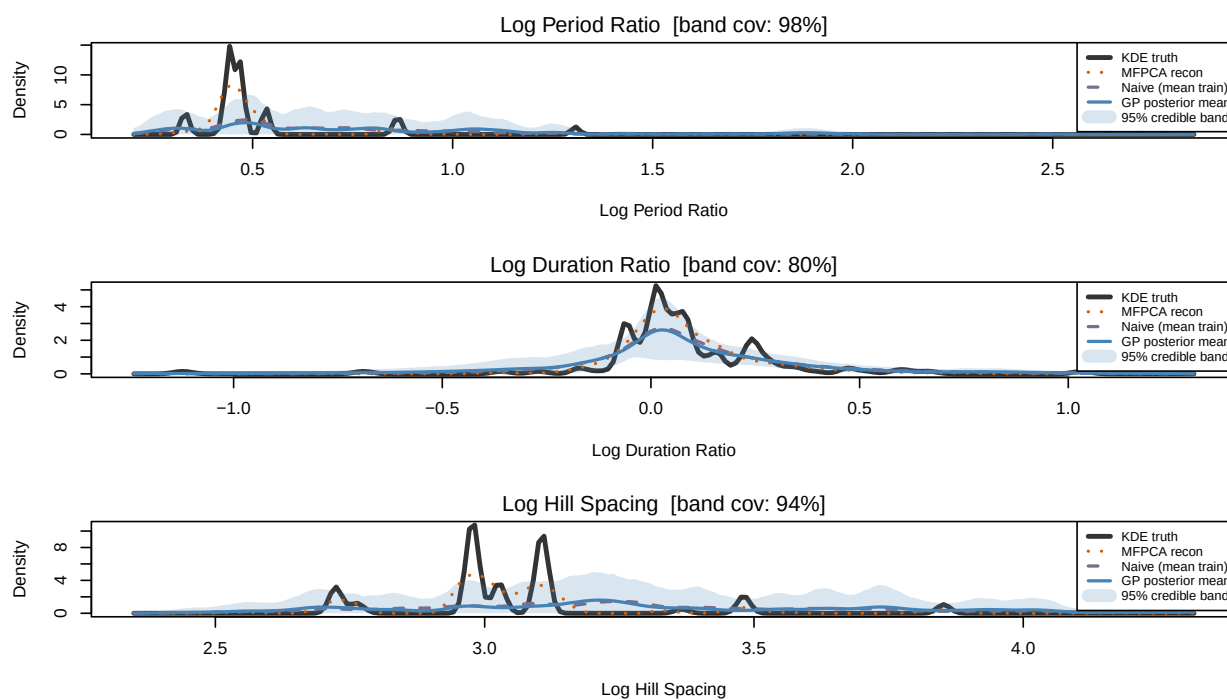
Posterior Predictive Distribution at Held-Out $\sigma_z = 72.803$



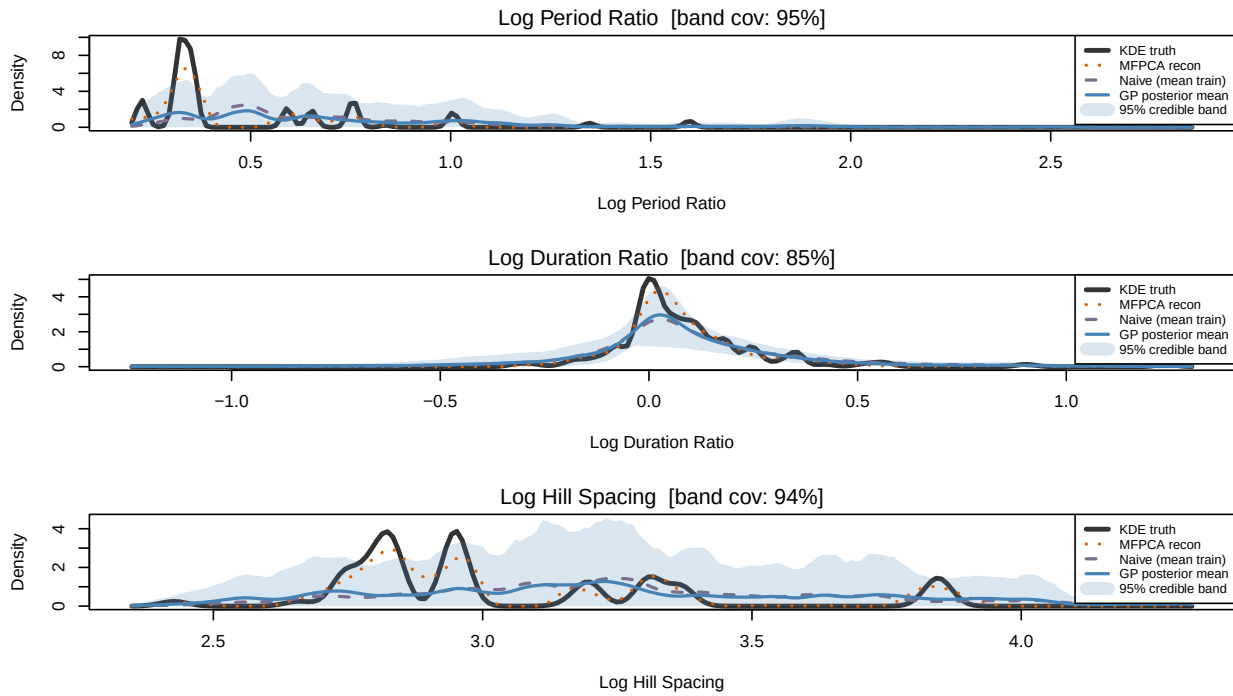
Posterior Predictive Distribution at Held-Out $\sigma_z = 75.337$



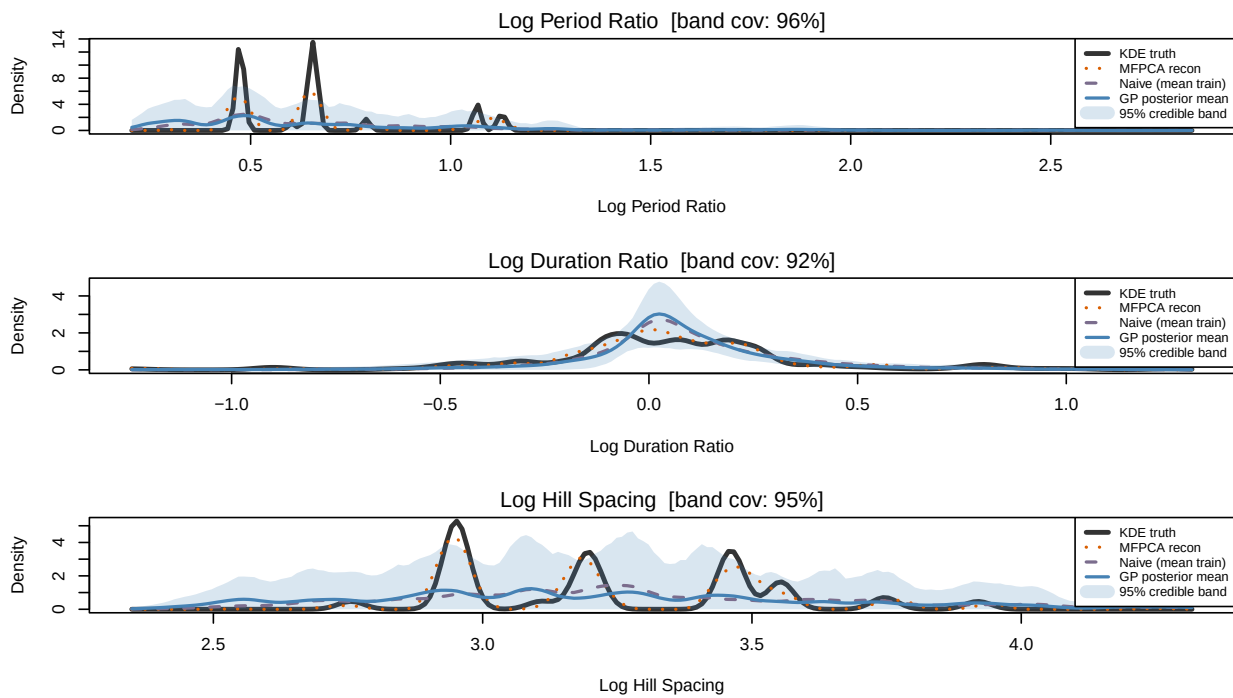
Posterior Predictive Distribution at Held-Out $\sigma_z = 77.294$



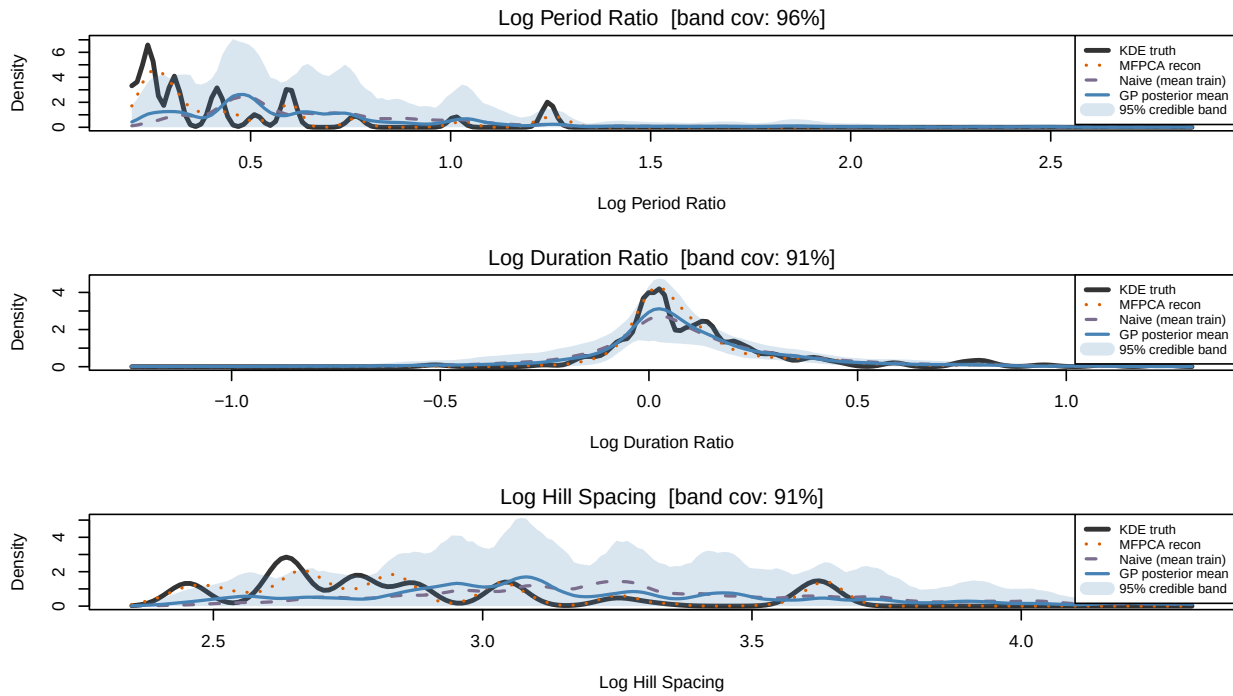
Posterior Predictive Distribution at Held-Out $\sigma_z = 80.497$



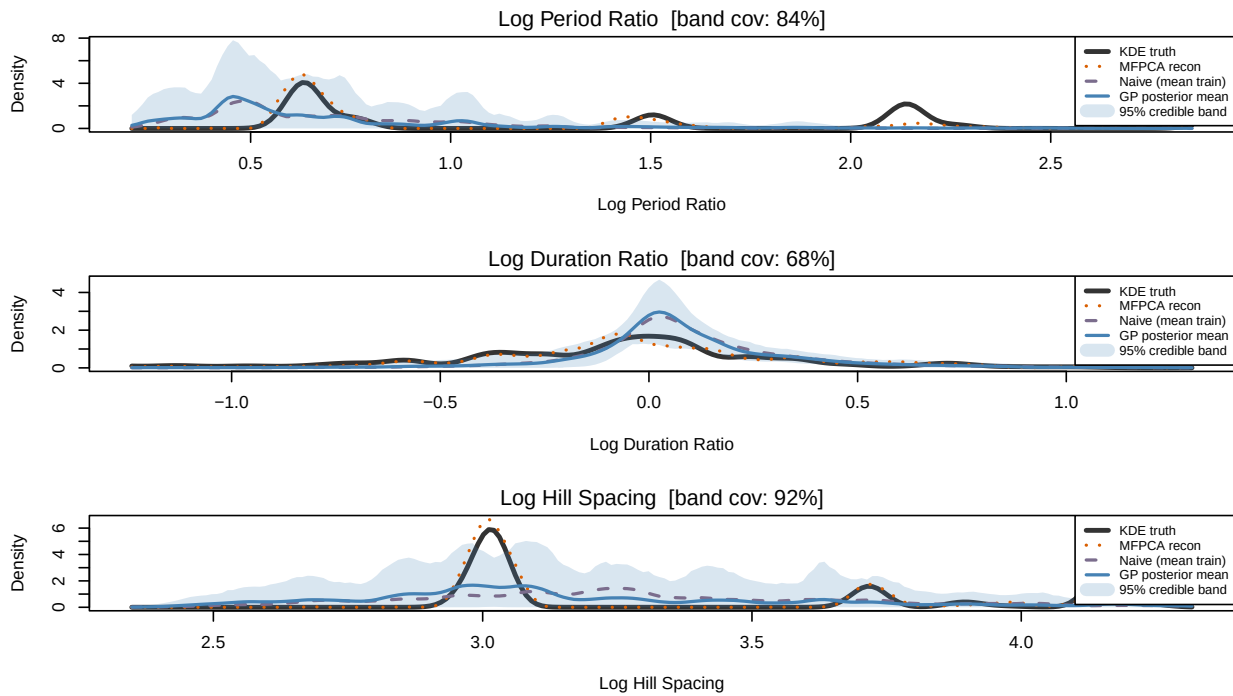
Posterior Predictive Distribution at Held-Out $\sigma_z = 84.582$



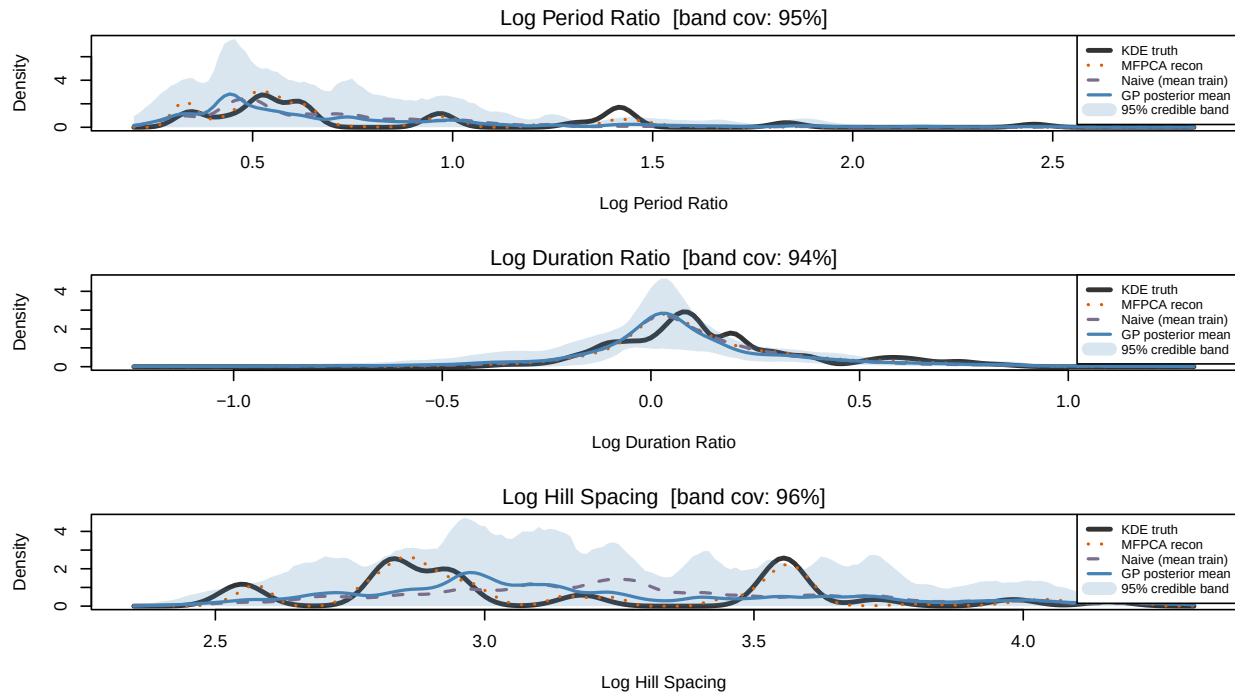
Posterior Predictive Distribution at Held-Out $\sigma_z = 87.63$



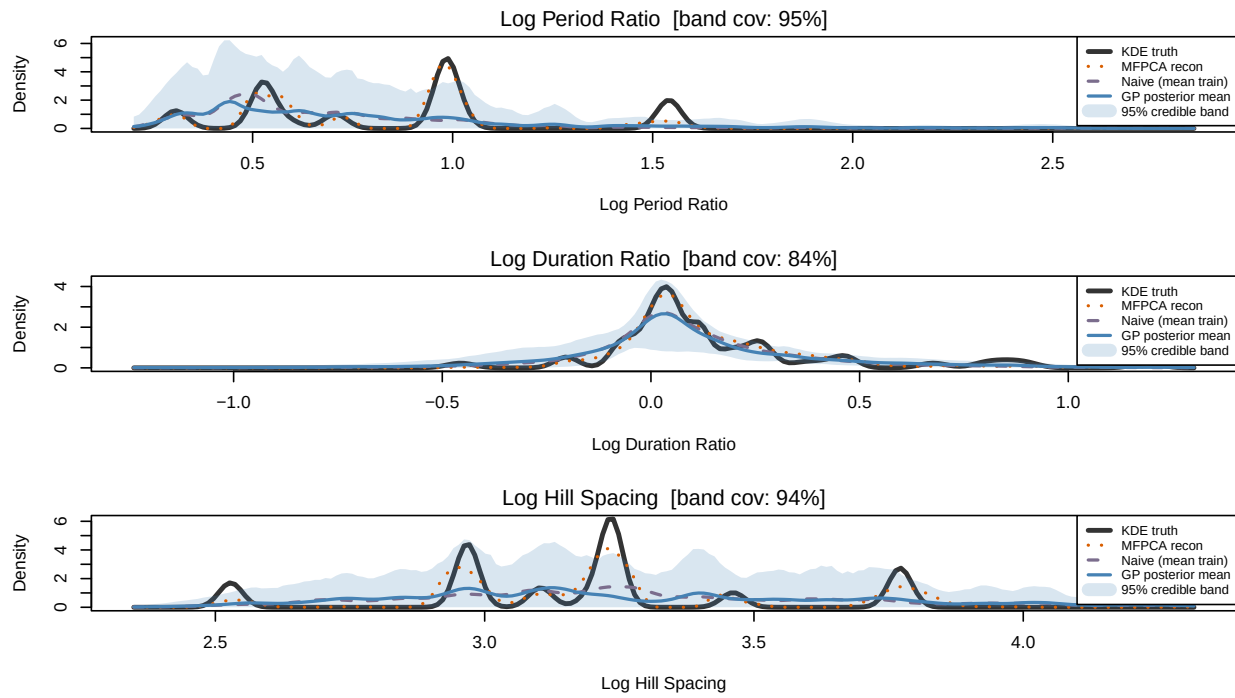
Posterior Predictive Distribution at Held-Out $\sigma_z = 90.576$



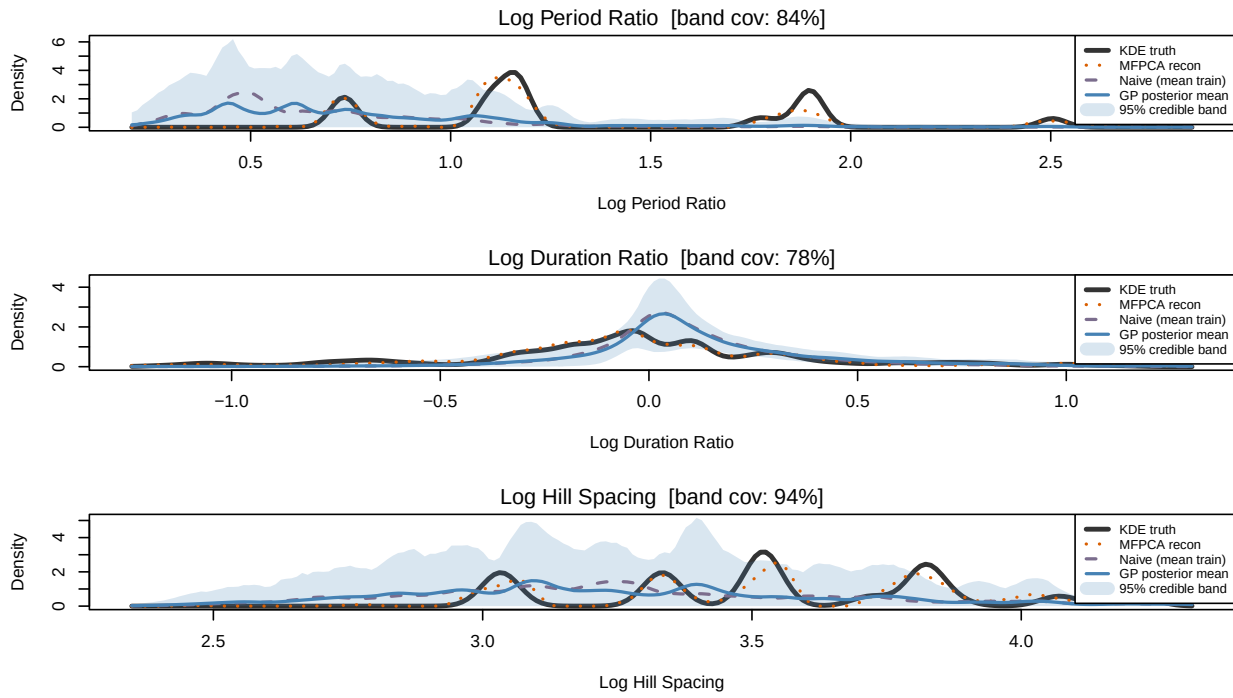
Posterior Predictive Distribution at Held-Out $\sigma_z = 95.636$



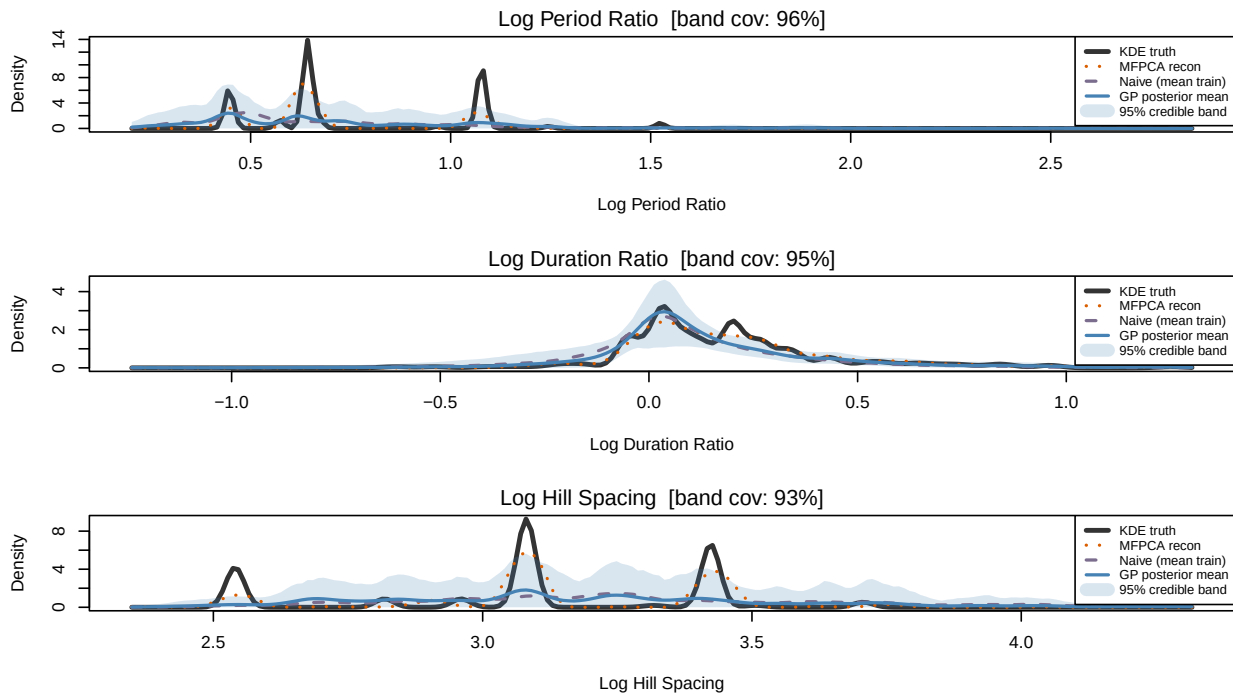
Posterior Predictive Distribution at Held-Out $\sigma_z = 101.983$



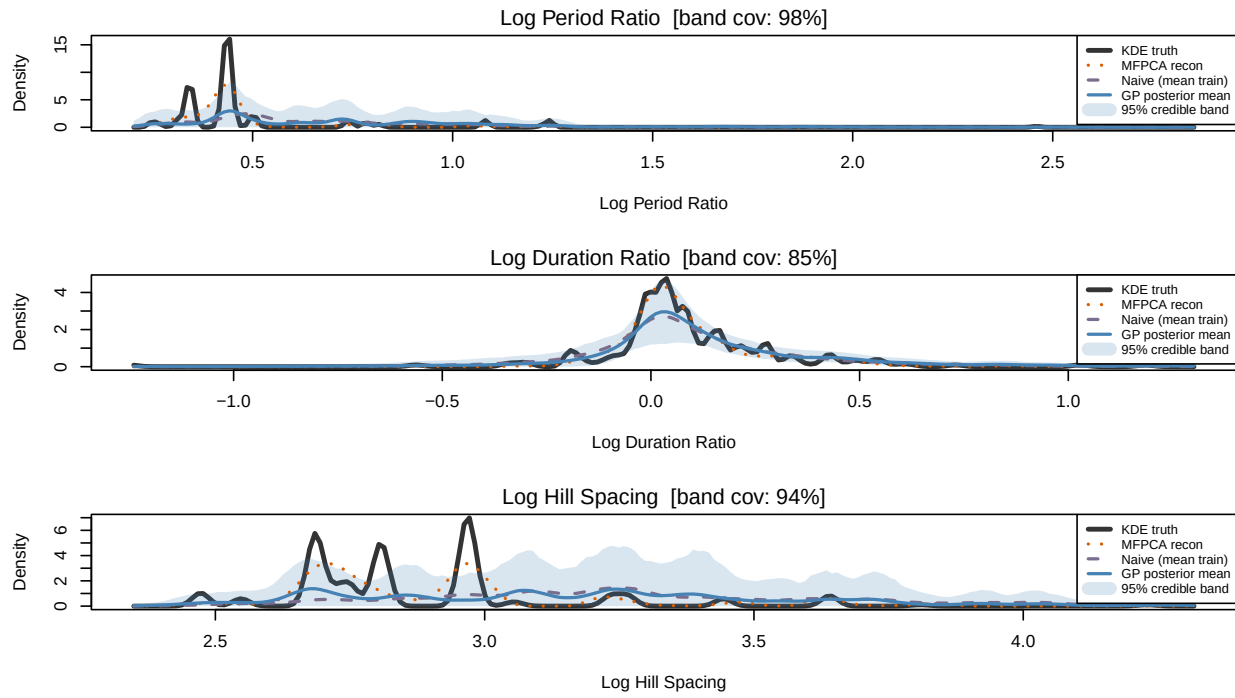
Posterior Predictive Distribution at Held-Out $\sigma_z = 105.023$



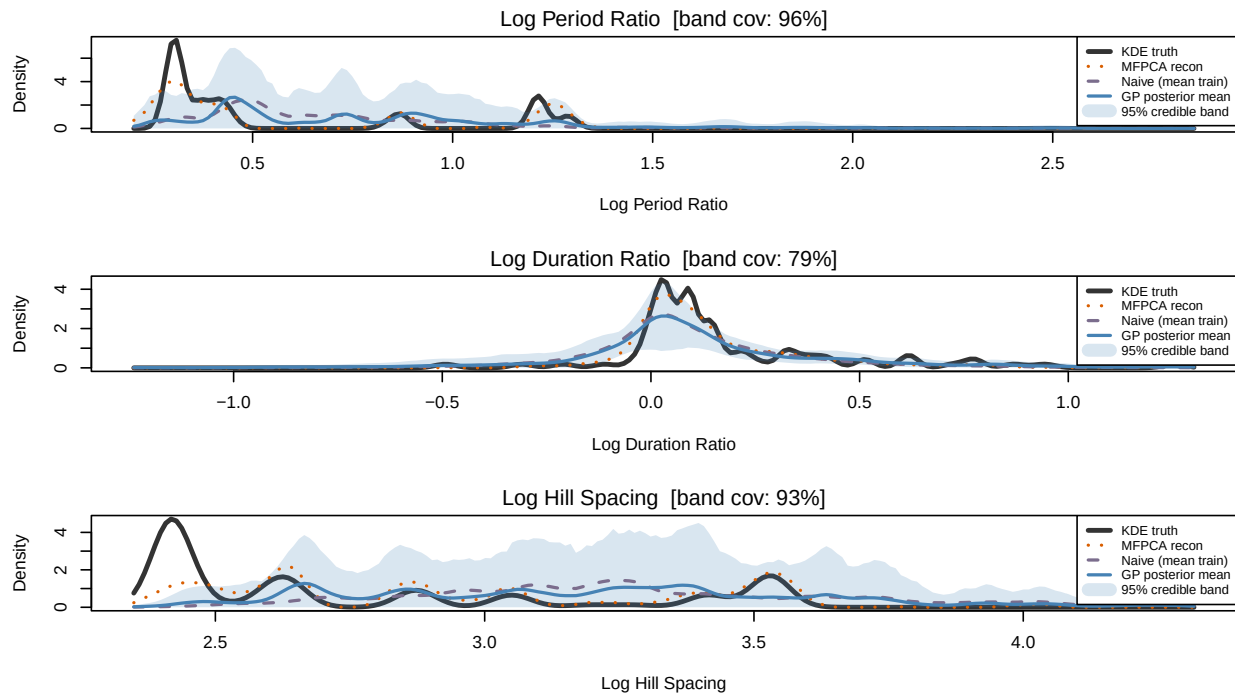
Posterior Predictive Distribution at Held-Out $\sigma_z = 112.32$



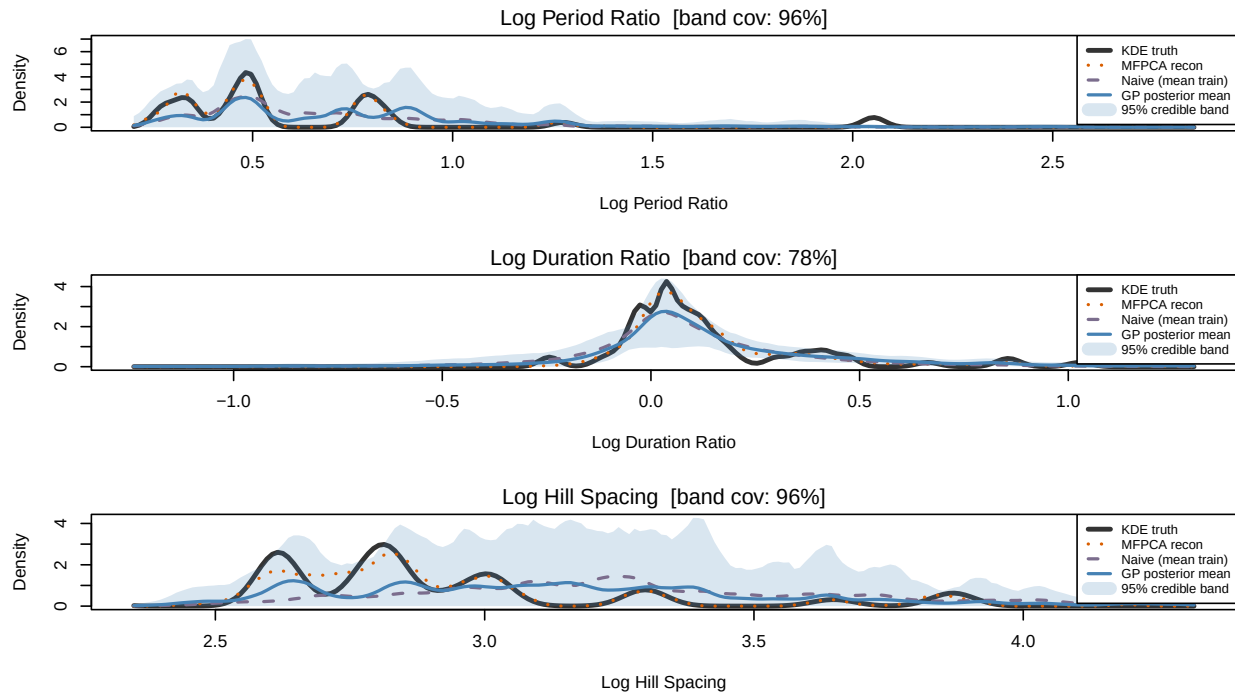
Posterior Predictive Distribution at Held-Out $\sigma_z = 125.792$



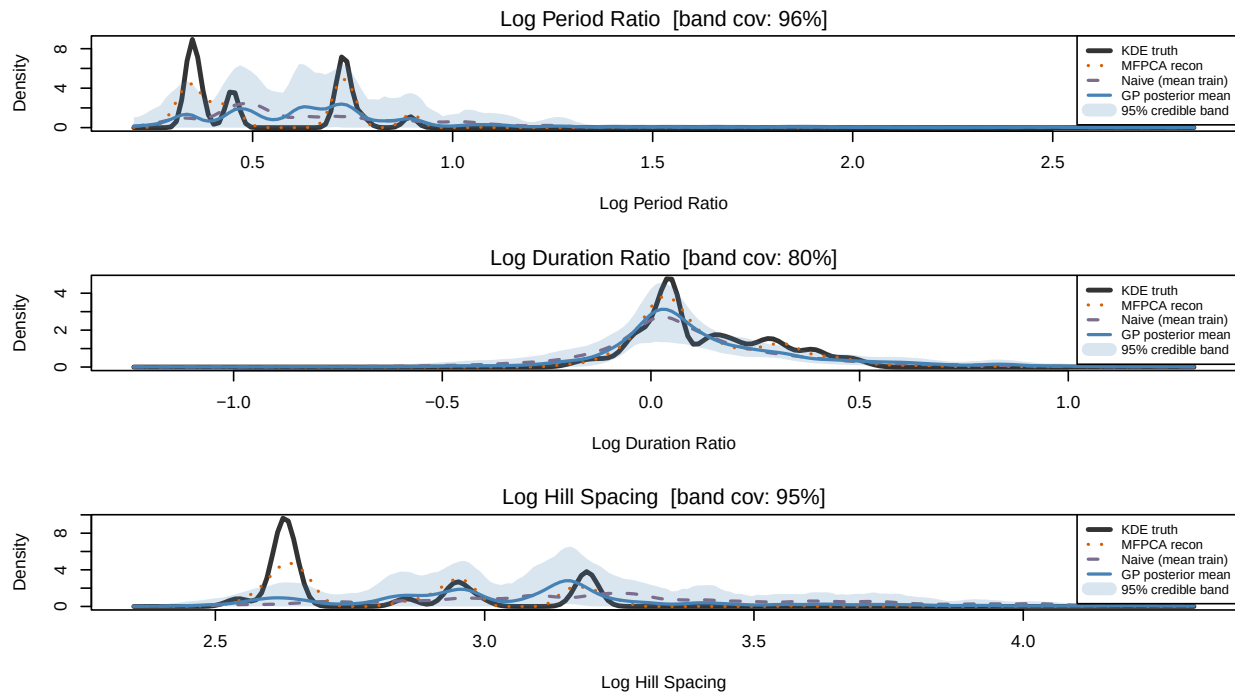
Posterior Predictive Distribution at Held-Out $\sigma_z = 136.035$



Posterior Predictive Distribution at Held-Out $\sigma_z = 151.852$



Posterior Predictive Distribution at Held-Out $\sigma_z = 166.054$



```
# =====
# Goal: the GP emulator maps theta -> f(x|theta), predicting the full density
# of planetary statistics rather than a scalar summary.
```

```

#
# Metric hierarchy:
# 1. iCRPS in density space (primary proper scoring rule)
# 2. Mean-only density reconstruction metrics
# 3. Posterior predictive coverage
# =====

cat("\n\n")

cat(paste(rep("#", 70), collapse=""), "\n")

## #####

cat("DISTRIBUTION PREDICTION\n")

## DISTRIBUTION PREDICTION

cat("Goal: GP emulator predicts f(x | theta_new) in density space.\n")

## Goal: GP emulator predicts f(x | theta_new) in density space.

cat(paste(rep("#", 70), collapse=""), "\n\n")

## #####

cat("PRIMARY VALIDATION METRIC\n")

## PRIMARY VALIDATION METRIC

cat(paste(rep("-", 70), collapse=""), "\n")

## -----

cat(sprintf(" %-22s %8s %8s %8s %8s\n",
            "Metric / Variable", "GP", "Naive", "Ratio", "Gain(%)"))

## Metric / Variable          GP      Naive      Ratio      Gain(%)

cat(paste(rep("-", 70), collapse=""), "\n")

## -----

for (info in list(
  list(gp = icrps_gp_x, mp = icrps_naive_x, nm = "iCRPS      X (Period)"),
  list(gp = icrps_gp_y, mp = icrps_naive_y, nm = "iCRPS      Y (Duration)"),
  list(gp = icrps_gp_z, mp = icrps_naive_z, nm = "iCRPS      Z (Hill)"))) {
  r <- mean(info$gp) / mean(info$mp)
  cat(sprintf(" %-22s %8.4f %8.4f %8.3f %+7.1f\n",
              info$nm, mean(info$gp), mean(info$mp), r, (1 - r) * 100))
}

```

```
## iCRPS X (Period) 0.8371 1.0029 0.835 +16.5
## iCRPS Y (Duration) 0.3036 0.3990 0.761 +23.9
## iCRPS Z (Hill) 0.7634 0.9404 0.812 +18.8
```

```
# Posterior predictive density coverage (95% band)
```

```
cat(sprintf(" %-22s %8.1f%% %8s %8s (target: 95%%)\n",
            "PPC band X (Period)", mean(cov_x)*100, "\u2014", "\u2014"))
```

```
## PPC band X (Period) 94.1% - - (target: 95%)
```

```
cat(sprintf(" %-22s %8.1f%% %8s %8s\n",
            "PPC band Y (Duration)", mean(cov_y)*100, "\u2014", "\u2014"))
```

```
## PPC band Y (Duration) 84.9% - -
```

```
cat(sprintf(" %-22s %8.1f%% %8s %8s\n",
            "PPC band Z (Hill)", mean(cov_z)*100, "\u2014", "\u2014"))
```

```
## PPC band Z (Hill) 94.0% - -
```

```
cat(sprintf(" %-22s %8.2f %8s %8s\n",
            "Calibration c (fixed)", CAL_FACTOR, "\u2014", "\u2014"))
```

```
## Calibration c (fixed) 1.00 - -
```

```
cat(paste(rep("-", 70), collapse=""), "\n\n")
```

```
## -----
```

```
cat("MEAN-ONLY DENSITY ERROR\n")
```

```
## MEAN-ONLY DENSITY ERROR
```

```
cat(" Posterior mean reconstruction only; uncertainty excluded.\n")
```

```
## Posterior mean reconstruction only; uncertainty excluded.
```

```
cat(sprintf(" %-22s %8.4f %8.4f\n", "ISE X: GP / Naive",
            mean(gp_mean_ise_x), mean(naive_mean_ise_x)))
```

```
## ISE X: GP / Naive 3.7644 4.1377
```

```
cat(sprintf(" %-22s %8.4f %8.4f\n", "ISE Y: GP / Naive",
            mean(gp_mean_ise_y), mean(naive_mean_ise_y)))
```

```
## ISE Y: GP / Naive 0.2807 0.3363
```

```
cat(sprintf(" %-22s %8.4f %8.4f\n", "ISE Z: GP / Naive",
            mean(gp_mean_ise_z), mean(naive_mean_ise_z)))
```

```
## ISE Z: GP / Naive          2.7744    3.1275
```

```
cat(sprintf(" %-22s %8.4f %8.4f\n", "Hel X: GP / Naive",
            mean(gp_mean_hell_x), mean(naive_mean_hell_x)))
```

```
## Hel X: GP / Naive          0.5903    0.6335
```

```
cat(sprintf(" %-22s %8.4f %8.4f\n", "Hel Y: GP / Naive",
            mean(gp_mean_hell_y), mean(naive_mean_hell_y)))
```

```
## Hel Y: GP / Naive          0.2246    0.2504
```

```
cat(sprintf(" %-22s %8.4f %8.4f\n", "Hel Z: GP / Naive",
            mean(gp_mean_hell_z), mean(naive_mean_hell_z)))
```

```
## Hel Z: GP / Naive          0.5361    0.5955
```

```
cat(paste(rep("-", 70), collapse=""), "\n\n")
```

```
## -----
```

```
cat("BASIS CHARACTERISATION ONLY\n")
```

```
## BASIS CHARACTERISATION ONLY
```

```
cat(" These values are computed with exact unpredicted scores and are not\n")
```

```
## These values are computed with exact unpredicted scores and are not
```

```
cat(" used as primary density-space validation metrics.\n")
```

```
## used as primary density-space validation metrics.
```

```
cat(sprintf(" %-22s %8.4f\n", "ISE floor X (Period)", mean(ise_mfpca_x)))
```

```
## ISE floor X (Period)      1.4128
```

```
cat(sprintf(" %-22s %8.4f\n", "ISE floor Y (Duration)", mean(ise_mfpca_y)))
```

```
## ISE floor Y (Duration)    0.0913
```

```

cat(sprintf("  %-22s  %8.4f\n", "ISE floor Z (Hill)",      mean(ise_mfpca_z)))

##   ISE floor Z (Hill)          0.7999

cat(sprintf("  %-22s  %8.4f\n", "X raw-space ISE",      mean(ise_x_raw_space)))

##   X raw-space ISE           1.4128

cat(sprintf("  %-22s  %8.4f\n", "X final density ISE",    mean(ise_x_final_density)))

##   X final density ISE       1.4128

cat(sprintf("  %-22s  %8.4f\n", "Z raw-space ISE",      mean(ise_z_raw_space)))

##   Z raw-space ISE           0.7999

cat(sprintf("  %-22s  %8.4f\n", "Z final density ISE",    mean(ise_z_final_density)))

##   Z final density ISE       0.7999

cat(paste(rep("-", 70), collapse=""), "\n\n")

## -----

cat("DIAGNOSTIC (latent PC space)\n")

## DIAGNOSTIC (latent PC space)

cat(paste(rep("-", 70), collapse=""), "\n")

## -----

cat(sprintf("  %-22s  %8.4f\n", "CRPS GP (PC space)",      overall_crps_gp))

##   CRPS GP (PC space)         0.0362

cat(sprintf("  %-22s  %8.4f\n", "CRPS Naive (PC space)",    overall_crps_naive))

##   CRPS Naive (PC space)      0.0326

cat(sprintf("  %-22s  %8.4f\n", "CRPS Skill Score",
          1 - overall_crps_gp / overall_crps_naive))

##   CRPS Skill Score          -0.1103

```

```

cat(paste(rep("#", 70), collapse=""), "\n\n")

## #####

# =====
# X reconstruction diagnostic
# Compare raw X, MFPCA true-score reconstruction, and GP mean reconstruction on
# held-out examples.
# =====
cat("X reconstruction diagnostic (held-out test points)\n")

## X reconstruction diagnostic (held-out test points)

x_diag_ord <- order(x_test_recon_diag$gp_mean_ise, decreasing = TRUE)
x_diag_top <- x_test_recon_diag[x_diag_ord[seq_len(min(6, nrow(x_test_recon_diag)))], ]
print(x_diag_top)

##      theta mfpca_raw_ise final_density_ise gp_mean_ise
## 1  26.69652      5.944836      5.944836      8.873474
## 4  48.20141      1.400680      1.400680      7.903690
## 15 84.58170      2.795487      2.795487      6.081507
## 13 77.29355      2.240599      2.240599      6.045945
## 6   55.65143      1.088821      1.088821      5.894138
## 10 70.36510      2.591914      2.591914      5.800309

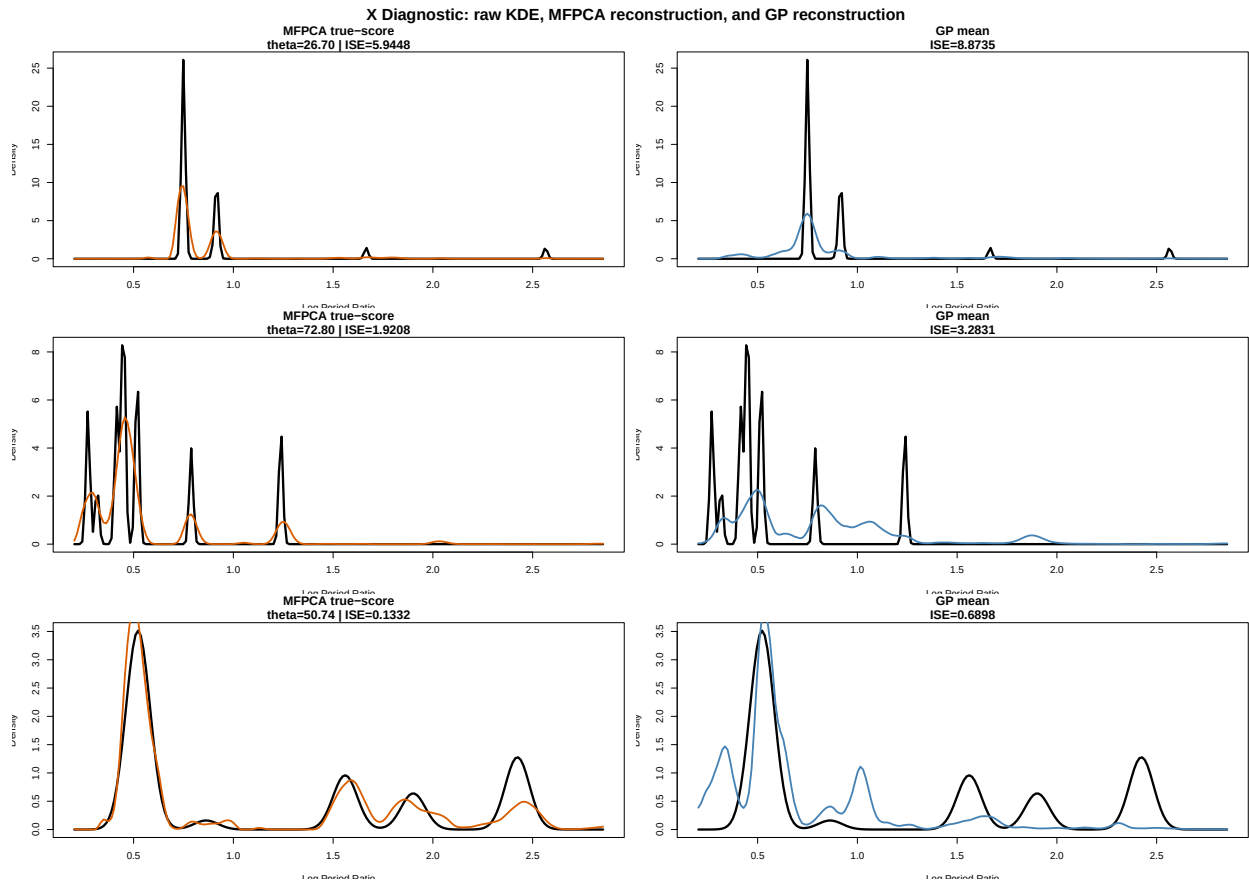
sel_x_idx <- x_diag_ord[c(1, ceiling(length(x_diag_ord) / 2), length(x_diag_ord))]
sel_x_idx <- unique(sel_x_idx)

par(mfrow = c(length(sel_x_idx), 2), mar = c(3.5, 3.5, 2.5, 1), oma = c(0, 0, 3, 0))
for (ii in sel_x_idx) {
  ref_row <- test_theta_idx[ii]
  x_truth_raw <- dens_mat_x[ref_row, ]
  x_recon_mfpca <- recon_dens_x_true[ref_row, ]
  x_recon_final <- recon_test$x[ii, ]
  sigma_lab <- sprintf("theta=%.2f", test_theta_vals[ii])

  plot(x_grid, x_truth_raw, type = "l", lwd = 2.5, col = "black",
       main = sprintf("MFPCA true-score\n%s | ISE=%.4f",
                      sigma_lab, x_test_recon_diag$mfpca_raw_ise[ii]),
       xlab = "Log Period Ratio", ylab = "Density")
  lines(x_grid, x_recon_mfpca, col = "#D95F02", lwd = 2)

  plot(x_grid, x_truth_raw, type = "l", lwd = 2.5, col = "black",
       main = sprintf("GP mean\nISE=%.4f", x_test_recon_diag$gp_mean_ise[ii]),
       xlab = "Log Period Ratio", ylab = "Density")
  lines(x_grid, x_recon_final, col = "steelblue", lwd = 2)
}
mtext("X Diagnostic: raw KDE, MFPCA reconstruction, and GP reconstruction",
     outer = TRUE, cex = 1.1, font = 2)

```

```
# =====
# Z reconstruction diagnostic
# Compare raw Z, MFPCA true-score reconstruction, and GP mean reconstruction on
# held-out examples.
# =====
cat("Z reconstruction diagnostic (held-out test points)\n")
```

```
## Z reconstruction diagnostic (held-out test points)
```

```
z_diag_ord <- order(z_test_recon_diag$gp_mean_ise, decreasing = TRUE)
z_diag_top <- z_test_recon_diag[z_diag_ord[seq_len(min(6, nrow(z_test_recon_diag)))], ]
print(z_diag_top)
```

```
##      theta mfpca_raw_ise final_density_ise gp_mean_ise
## 1  26.69652    4.0816690      4.0816690    10.608265
## 4  48.20141    3.5805693      3.5805693     9.389582
## 2  32.72545    0.4403147      0.4403147     6.130662
## 13 77.29355    1.9984101      1.9984101     5.047650
## 3  44.27641    0.1026714      0.1026714     4.106193
## 25 166.05428    1.0518672      1.0518672     3.271381
```

```

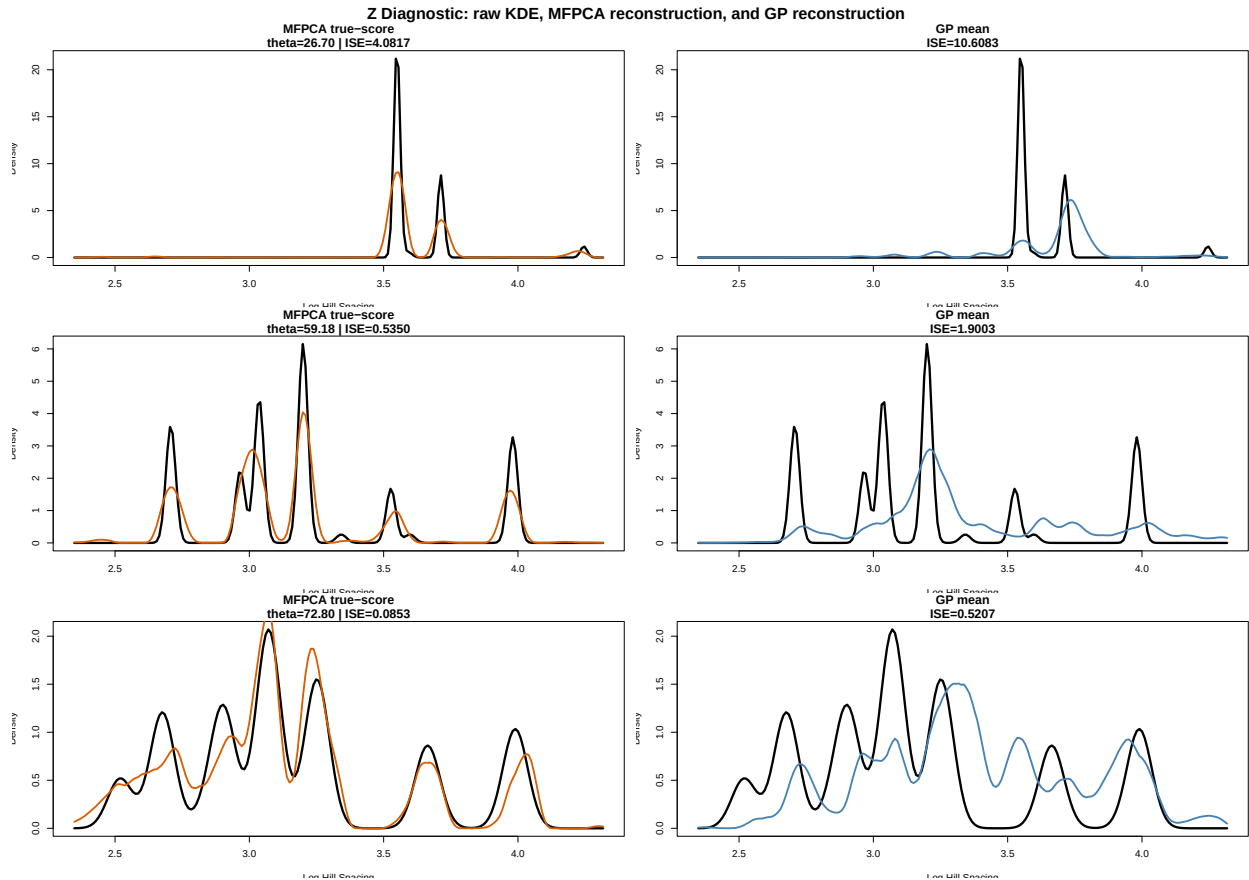
sel_z_idx <- z_diag_ord[c(1, ceiling(length(z_diag_ord) / 2), length(z_diag_ord))]
sel_z_idx <- unique(sel_z_idx)

par(mfrow = c(length(sel_z_idx), 2), mar = c(3.5, 3.5, 2.5, 1), oma = c(0, 0, 3, 0))
for (ii in sel_z_idx) {
  ref_row <- test_theta_idx[ii]
  z_truth_raw <- dens_mat_z[ref_row, ]
  z_recon_mfpca <- recon_dens_z_true[ref_row, ]
  z_recon_final <- recon_test$z[ii, ]
  sigma_lab <- sprintf("theta=%.2f", test_theta_vals[ii])

  plot(z_grid, z_truth_raw, type = "l", lwd = 2.5, col = "black",
       main = sprintf("MFPCA true-score\n%s | ISE=%.4f",
                      sigma_lab, z_test_recon_diag$mfpca_raw_ise[ii]),
       xlab = "Log Hill Spacing", ylab = "Density")
  lines(z_grid, z_recon_mfpca, col = "#D95F02", lwd = 2)

  plot(z_grid, z_truth_raw, type = "l", lwd = 2.5, col = "black",
       main = sprintf("GP mean\nISE=%.4f", z_test_recon_diag$gp_mean_ise[ii]),
       xlab = "Log Hill Spacing", ylab = "Density")
  lines(z_grid, z_recon_final, col = "steelblue", lwd = 2)
}
mtext("Z Diagnostic: raw KDE, MFPCA reconstruction, and GP reconstruction",
      outer = TRUE, cex = 1.1, font = 2)

```



```
# =====
# PAPER FIGURE 2
# Grey: raw sign-corrected scores
# Annotated with lag-1 ACF of smoothed series and residual SD s_k per panel.
# =====

sigma_ord_fig2 <- order(ufpca_scores_raw_aligned$theta)
sig_s_fig2     <- ufpca_scores_raw_aligned$theta[sigma_ord_fig2]

par(mfrow = c(2, 4), mar = c(4, 3.5, 2.8, 1), oma = c(0, 0, 3, 0))

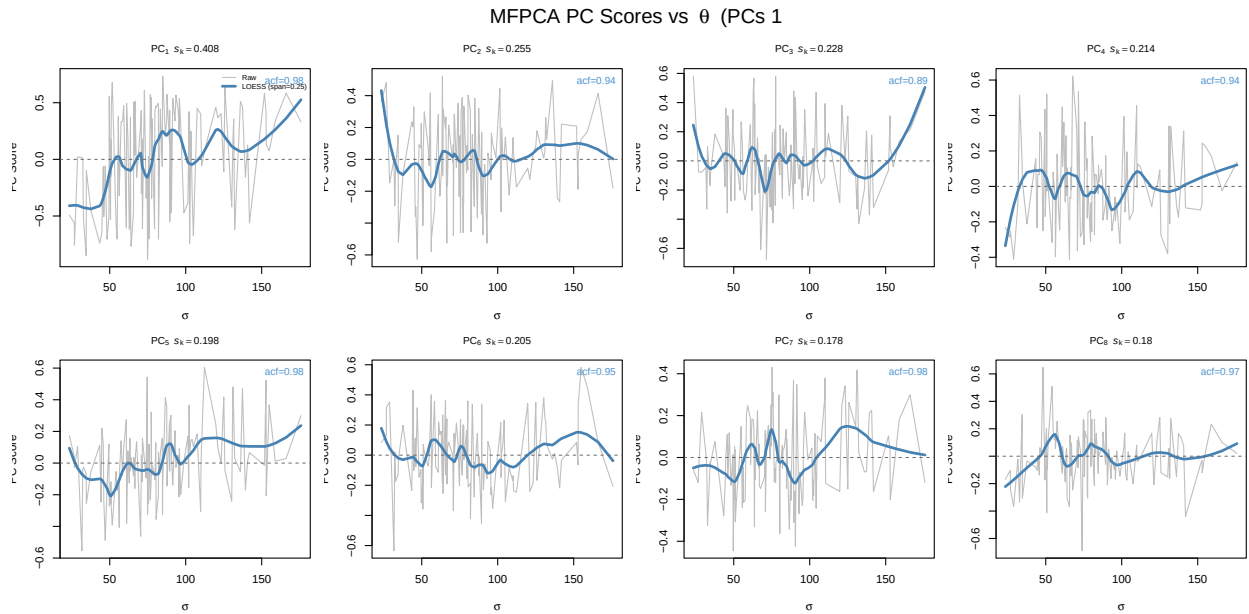
for (pc in 1:8) {
  pc_col   <- paste0("PC", pc)
  raw_s    <- ufpca_scores_raw_aligned[sigma_ord_fig2, pc_col]
  smooth_s <- ufpca_scores_smoothed[sigma_ord_fig2, pc_col]
  s_k_val  <- round(sd(raw_s - smooth_s), 3)
  acf_val  <- round(cor(smooth_s[-length(smooth_s)], smooth_s[-1]), 2)

  plot(sig_s_fig2, raw_s, type = "l", col = "gray75", lwd = 1,
       main = bquote(PC[.(pc)] ~~ italic(s)[k] == .(s_k_val)),
       xlab = expression(sigma), ylab = "PC Score", cex.main = 0.9)
  lines(sig_s_fig2, smooth_s, col = "steelblue", lwd = 2.5)
  abline(h = 0, col = "gray40", lty = 2, lwd = 0.8)
  mtext(paste0("acf=", acf_val), side = 3, line = -1.3, adj = 0.97,
       cex = 0.62, col = "steelblue3")
}
```

```

if (pc == 1)
  legend("topright", c("Raw", "LOESS (span=0.25)"),
        col = c("gray75", "steelblue"), lwd = c(1, 2.5),
        cex = 0.64, bty = "n")
}
mtext(expression("MFPCA PC Scores vs " ~ theta ~ " (PCs 1"),
  outer = TRUE, cex = 1.2, font = 2)

```



```

# =====
# PAPER FIGURE 3
# Layout: 4 rows (low/mid-low/mid-high/high)
# Black: held-out KDE. Dark orange dot-dash: MFPCA reconstruction. Purple dashed: naive baseline. Bl
# Shaded: 95% pointwise credible band. Coverage % annotated per panel.
# =====

idx4      <- round(seq(1, n_test_s, length.out = 4))
col_truth <- "gray20"
col_mfpca <- "#D95F02"
col_naive <- "#7B6D8D"
col_gp    <- "steelblue"
col_band  <- rgb(0.27, 0.51, 0.71, 0.22)

par(mfrow = c(4, 3), mar = c(3.5, 4, 2.5, 1), oma = c(0, 0, 3, 0))

for (ii in seq_along(idx4)) {
  i      <- idx4[ii]
  sigma_val <- round(test_theta_vals[i], 2)
  ref_row <- test_theta_idx[i]
  pe      <- post_ens[[i]]

  panels <- list(

```

```

list(grid = x_grid,
      truth = dens_mat_x[ref_row, ], mfpca = recon_dens_x_true[ref_row, ], naive = mean_train_dens_x,
      gp_m = pe$mean_x, lo = pe$lower_x, hi = pe$upper_x,
      cov_i = cov_x[i], xl = "Log Period Ratio"),
list(grid = y_grid,
      truth = dens_mat_y[ref_row, ], mfpca = recon_dens_y_true[ref_row, ], naive = mean_train_dens_y,
      gp_m = pe$mean_y, lo = pe$lower_y, hi = pe$upper_y,
      cov_i = cov_y[i], xl = "Log Duration Ratio"),
list(grid = z_grid,
      truth = dens_mat_z[ref_row, ], mfpca = recon_dens_z_true[ref_row, ], naive = mean_train_dens_z,
      gp_m = pe$mean_z, lo = pe$lower_z, hi = pe$upper_z,
      cov_i = cov_z[i], xl = "Log Hill Spacing")
)

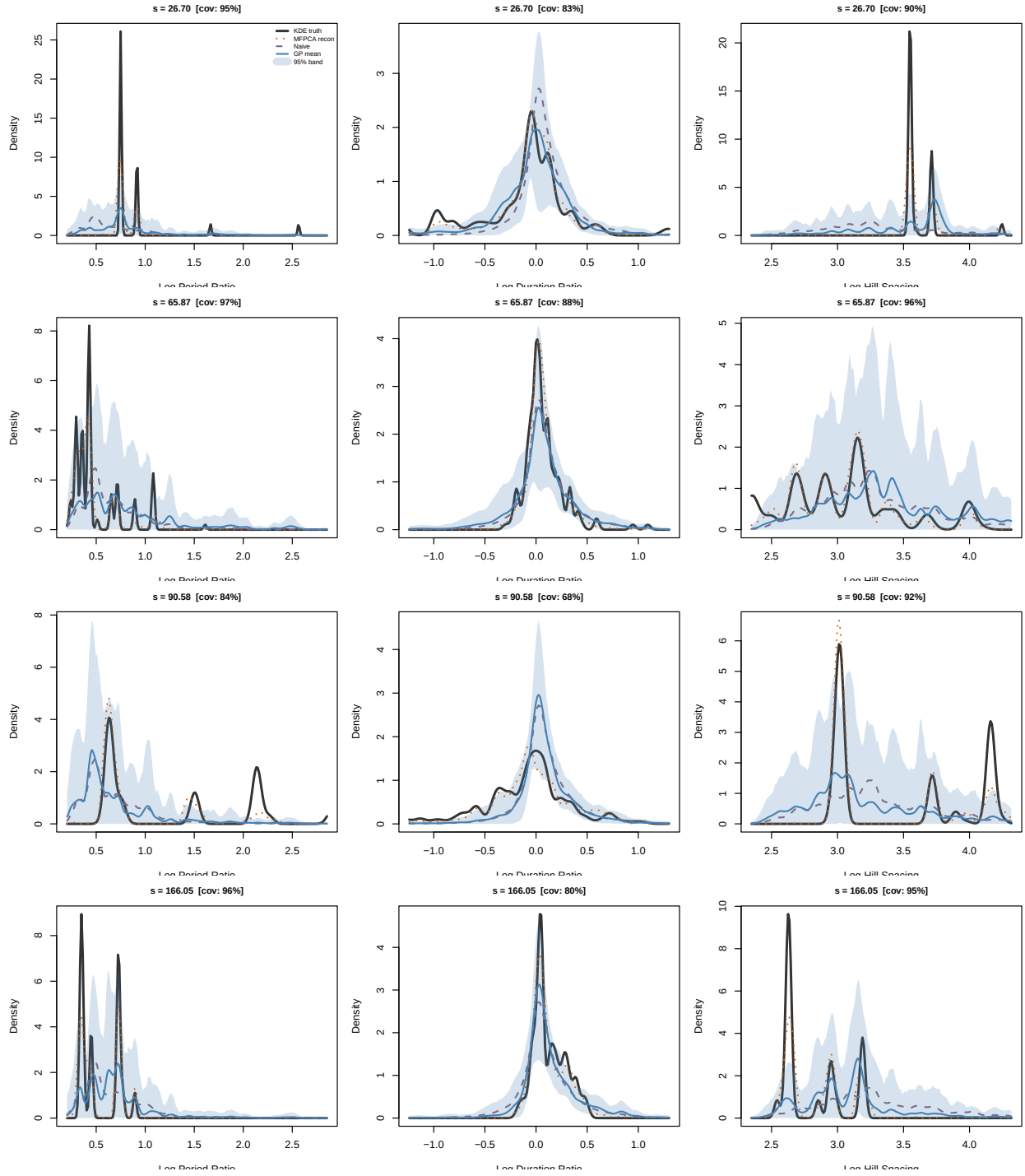
for (p in panels) {
  yr <- range(c(p$truth, p$hi, p$mfpca, p$naive, p$gp_m), na.rm = TRUE)
  yr[1] <- max(0, yr[1])

  plot(p$grid, p$truth, type = "l", col = col_truth, lwd = 2.5,
        xlab = p$xl, ylab = "Density", ylim = yr,
        main = sprintf("\u03c3 = %.2f [cov: %.0f%%]", sigma_val, p$cov_i * 100),
        cex.main = 0.87)
  polygon(c(p$grid, rev(p$grid)), c(p$lo, rev(p$hi)),
          col = col_band, border = NA)
  lines(p$grid, p$mfpca, col = col_mfpca, lwd = 1.8, lty = 3)
  lines(p$grid, p$naive, col = col_naive, lwd = 1.8, lty = 2)
  lines(p$grid, p$gp_m, col = col_gp, lwd = 1.8)

  if (ii == 1 && p$xl == "Log Period Ratio")
    legend("topright",
           c("KDE truth", "MFPCA recon", "Naive", "GP mean", "95% band"),
           col = c(col_truth, col_mfpca, col_naive, col_gp, col_band),
           lwd = c(2.5, 1.8, 1.8, 1.8, 8),
           lty = c(1, 3, 2, 1, 1),
           cex = 0.62, bty = "n")
}
}
mtext(expression("Posterior Predictive Densities at 4 Held-Out " * theta * " Values"),
      outer = TRUE, cex = 1.2, font = 2)

```

Posterior Predictive Densities at 4 Held-Out θ Values



```
# =====
# SIGNIFICANCE TESTS
# H1 (one-sided): GP iCRPS < Naive iCRPS (GP wins if p < 0.05)
# Bootstrap 95% CI on ratio GP/Naive for iCRPS.
# =====
```

```

cat("\n")

cat(paste(rep("=", 64), collapse = ""), "\n")

## =====

cat("PAIRED WILCOXON SIGNED-RANK TESTS (n=25 test pairs, one-sided)\n")

## PAIRED WILCOXON SIGNED-RANK TESTS (n=25 test pairs, one-sided)

cat("H1: GP lower than comparator [* p<0.05 . p<0.10 blank: ns]\n")

## H1: GP lower than comparator [* p<0.05 . p<0.10 blank: ns]

cat(paste(rep("=", 64), collapse = ""), "\n")

## =====

cat(sprintf("%-44s %8s %6s\n", "Comparison", "p-value", "V stat"))

## Comparison                                p-value V stat

cat(paste(rep("-", 62), collapse = ""), "\n")

## -----

test_pairs_wx <- list(
  list(gp = icrps_gp_x, cp = icrps_naive_x, nm = "iCRPS X: GP vs Naive"),
  list(gp = icrps_gp_y, cp = icrps_naive_y, nm = "iCRPS Y: GP vs Naive"),
  list(gp = icrps_gp_z, cp = icrps_naive_z, nm = "iCRPS Z: GP vs Naive")
)

for (tp in test_pairs_wx) {
  wt <- wilcox.test(tp$gp, tp$cp, paired = TRUE, alternative = "less",
    exact = FALSE)
  sig <- if (wt$p.value < 0.05) "*" else if (wt$p.value < 0.10) "." else " "
  cat(sprintf("%-44s %8.4f %6.0f %s\n",
    tp$nm, wt$p.value, wt$statistic, sig))
}

## iCRPS X: GP vs Naive 0.0000 0 *
## iCRPS Y: GP vs Naive 0.0000 3 *
## iCRPS Z: GP vs Naive 0.0000 5 *

cat(paste(rep("=", 64), collapse = ""), "\n\n")

## =====

```

```
# --- Bootstrap 95% CIs on ratio GP/Naive (B=5000) ---
set.seed(42)
B_boot <- 5000
```

```
boot_ci <- function(x, y, B) {
  ratios <- replicate(B, {
    idx <- sample(length(x), replace = TRUE)
    mean(x[idx]) / mean(y[idx])
  })
  quantile(ratios, c(0.025, 0.975))
}

cat("Bootstrap 95% CIs on GP/Naive ratio (< 1.0 = GP wins)\n")
```

```
## Bootstrap 95% CIs on GP/Naive ratio (< 1.0 = GP wins)
```

```
cat(paste(rep("-", 56), collapse = ""), "\n")
```

```
## -----
```

```
for (info in list(
  list(gp = icrps_gp_x, mp = icrps_naive_x, nm = "iCRPS(X)      ratio GP/Naive"),
  list(gp = icrps_gp_y, mp = icrps_naive_y, nm = "iCRPS(Y)      ratio GP/Naive"),
  list(gp = icrps_gp_z, mp = icrps_naive_z, nm = "iCRPS(Z)      ratio GP/Naive")
)) {
  obs <- mean(info$gp) / mean(info$mp)
  ci <- boot_ci(info$gp, info$mp, B_boot)
  below <- if (ci[2] < 1.0) "YES (CI entirely < 1)" else
    if (ci[1] < 1.0) "partial" else "NO"
  cat(sprintf("%-33s obs=%.3f CI=[%.3f, %.3f] GP wins: %s\n",
    info$nm, obs, ci[1], ci[2], below))
}
```

```
## iCRPS(X)      ratio GP/Naive      obs=0.835 CI=[0.810, 0.859] GP wins: YES (CI entirely < 1)
## iCRPS(Y)      ratio GP/Naive      obs=0.761 CI=[0.683, 0.833] GP wins: YES (CI entirely < 1)
## iCRPS(Z)      ratio GP/Naive      obs=0.812 CI=[0.779, 0.848] GP wins: YES (CI entirely < 1)
```

```
cat(paste(rep("=", 64), collapse = ""), "\n\n")
```

```
## =====
```

```
# =====
# FIGURE: THREE-STAGE RECONSTRUCTION COMPARISON
# Rows: X, Y, Z. Columns: KDE target, MFPCA reconstruction, GP reconstruction
# =====

# Select representative test point: median iCRPS for Y
median_idx <- which.min(abs(icrps_gp_y - median(icrps_gp_y)))
theta_star <- test_theta_vals[median_idx]
cat(sprintf("Representative theta* = %.1f g cm^-2 (median iCRPS(Y) test point, index %d)\n",
  theta_star, median_idx))
```



```

## Representative theta* = 70.4 g cm-2 (median iCRPS(Y) test point, index 10)

ref_row <- test_theta_idx[median_idx]

# Build panel data
fig_recon_data <- list()
annotations <- list()

for (v in c("X", "Y", "Z")) {
  grid_v <- switch(v, X = x_grid, Y = y_grid, Z = z_grid)
  kde_v <- switch(v,
    X = dens_mat_x[ref_row, ],
    Y = dens_mat_y[ref_row, ],
    Z = dens_mat_z[ref_row, ])

  # MFPCA reconstruction from true scores
  # ufpga_scores_long rows align with theta_list; extract PC columns as numeric vector
  true_scores <- as.numeric(ufpga_scores_long[ref_row, paste0("PC", seq_len(M_components))])
  recon_true_all <- reconstruct_densities(matrix(true_scores, nrow = 1L), mu_x, mu_y, mu_z,
    phi_x, phi_y, phi_z, x_grid, y_grid, z_grid,
    x_peak_loc = peak_loc_x_all[ref_row],
    x_peak_width = peak_width_x_all[ref_row],
    x_peak_left_width = peak_left_width_x_all[ref_row],
    x_peak_right_width = peak_right_width_x_all[ref_row],
    z_peak_loc = peak_loc_z_all[ref_row],
    z_peak_width = peak_width_z_all[ref_row],
    z_peak_left_width = peak_left_width_z_all[ref_row],
    z_peak_right_width = peak_right_width_z_all[ref_row])

  mfpga_recon <- switch(v, X = as.vector(recon_true_all$x),
    Y = as.vector(recon_true_all$y),
    Z = as.vector(recon_true_all$z))
  recon_ise <- compute_ISE(mfpga_recon, kde_v, grid_v)

  # GP reconstruction: posterior mean
  gp_mu_scores <- as.numeric(pred_test_df[median_idx, paste0("PC", seq_len(M_components))])
  gp_mean_all <- reconstruct_densities(matrix(gp_mu_scores, nrow = 1L), mu_x, mu_y, mu_z,
    phi_x, phi_y, phi_z, x_grid, y_grid, z_grid,
    x_peak_loc = pred_peak_x_test[median_idx],
    x_peak_width = pred_width_x_test[median_idx],
    x_peak_left_width = pred_left_width_x_test[median_idx],
    x_peak_right_width = pred_right_width_x_test[median_idx],
    z_peak_loc = pred_peak_z_test[median_idx],
    z_peak_width = pred_width_z_test[median_idx],
    z_peak_left_width = pred_left_width_z_test[median_idx],
    z_peak_right_width = pred_right_width_z_test[median_idx])

  gp_mean_dens <- switch(v, X = as.vector(gp_mean_all$x),
    Y = as.vector(gp_mean_all$y),
    Z = as.vector(gp_mean_all$z))

  # GP ensemble for credible band, using the Apr29 known-noise posterior variance.
  gp_sd <- sqrt(pmax(pred_test_var_mat[median_idx, ], 1e-10))
  set.seed(999 + which(c("X", "Y", "Z") == v))
  gp_samp <- matrix(
    rnorm(N_SAMP * M_components,

```

```

    mean = rep(gp_mu_scores, each = N_SAMP),
    sd = rep(gp_sd, each = N_SAMP)),
  nrow = N_SAMP, ncol = M_components)
ens_v <- reconstruct_densities(gp_samp, mu_x, mu_y, mu_z,
                              phi_x, phi_y, phi_z, x_grid, y_grid, z_grid,
                              x_peak_loc = rep(pred_peak_x_test[median_idx], N_SAMP),
                              x_peak_width = rep(pred_width_x_test[median_idx], N_SAMP),
                              x_peak_left_width = rep(pred_left_width_x_test[median_idx], N_SAMP),
                              x_peak_right_width = rep(pred_right_width_x_test[median_idx], N_SAMP),
                              z_peak_loc = rep(pred_peak_z_test[median_idx], N_SAMP),
                              z_peak_width = rep(pred_width_z_test[median_idx], N_SAMP),
                              z_peak_left_width = rep(pred_left_width_z_test[median_idx], N_SAMP),
                              z_peak_right_width = rep(pred_right_width_z_test[median_idx], N_SAMP))
ens_mat <- switch(v, X = ens_v$x, Y = ens_v$y, Z = ens_v$z)
band_lo <- apply(ens_mat, 2, quantile, 0.025)
band_hi <- apply(ens_mat, 2, quantile, 0.975)
coverage <- mean(kde_v >= band_lo & kde_v <= band_hi)
icrps_v <- switch(v,
  X = icrps_gp_x[median_idx],
  Y = icrps_gp_y[median_idx],
  Z = icrps_gp_z[median_idx])

dt <- diff(grid_v[1:2])

# Store annotations
annotations[[v]] <- list(ise = recon_ise, icrps = icrps_v, coverage = coverage)

# Panel data
fig_recon_data[[length(fig_recon_data) + 1]] <- data.frame(
  t = grid_v, density = kde_v, stage = "KDE (target)",
  variable = v, line_type = "KDE", stringsAsFactors = FALSE)
fig_recon_data[[length(fig_recon_data) + 1]] <- data.frame(
  t = grid_v, density = mfpca_recon, stage = "MFPCA reconstruction",
  variable = v, line_type = "MFPCA", stringsAsFactors = FALSE)
# KDE reference in MFPCA panel
fig_recon_data[[length(fig_recon_data) + 1]] <- data.frame(
  t = grid_v, density = kde_v, stage = "MFPCA reconstruction",
  variable = v, line_type = "KDE ref", stringsAsFactors = FALSE)
fig_recon_data[[length(fig_recon_data) + 1]] <- data.frame(
  t = grid_v, density = gp_mean_dens, stage = "GP reconstruction",
  variable = v, line_type = "GP mean", stringsAsFactors = FALSE)
# KDE reference in GP panel
fig_recon_data[[length(fig_recon_data) + 1]] <- data.frame(
  t = grid_v, density = kde_v, stage = "GP reconstruction",
  variable = v, line_type = "KDE ref", stringsAsFactors = FALSE)
# Credible band
fig_recon_data[[length(fig_recon_data) + 1]] <- data.frame(
  t = grid_v, density = band_lo, stage = "GP reconstruction",
  variable = v, line_type = "band_lo", stringsAsFactors = FALSE)
fig_recon_data[[length(fig_recon_data) + 1]] <- data.frame(
  t = grid_v, density = band_hi, stage = "GP reconstruction",
  variable = v, line_type = "band_hi", stringsAsFactors = FALSE)
}

```

```

df_plot <- do.call(rbind, fig_recon_data)
df_plot$stage <- factor(df_plot$stage,
  levels = c("KDE (target)", "MFPCA reconstruction", "GP reconstruction"))
df_plot$variable <- factor(df_plot$variable, levels = c("X", "Y", "Z"))

# Separate band data
df_lines <- df_plot[!df_plot$line_type %in% c("band_lo", "band_hi"), ]
df_band_lo <- df_plot[df_plot$line_type == "band_lo", ]
df_band_hi <- df_plot[df_plot$line_type == "band_hi", ]
df_band <- merge(df_band_lo[, c("t", "density", "stage", "variable")],
  df_band_hi[, c("t", "density", "stage", "variable")],
  by = c("t", "stage", "variable"), suffixes = c("_lo", "_hi"))

# Build annotation labels
ann_df <- data.frame(
  stage = factor(c(rep("MFPCA reconstruction", 3), rep("GP reconstruction", 3)),
    levels = levels(df_plot$stage)),
  variable = factor(rep(c("X", "Y", "Z"), 2), levels = c("X", "Y", "Z")),
  label = c(
    sprintf("ISE = %.3f", annotations$X$ise),
    sprintf("ISE = %.3f", annotations$Y$ise),
    sprintf("ISE = %.3f", annotations$Z$ise),
    sprintf("iCRPS = %.3f\nCov = %.1f%%", annotations$X$icrps, 100*annotations$X$coverage),
    sprintf("iCRPS = %.3f\nCov = %.1f%%", annotations$Y$icrps, 100*annotations$Y$coverage),
    sprintf("iCRPS = %.3f\nCov = %.1f%%", annotations$Z$icrps, 100*annotations$Z$coverage)),
  stringsAsFactors = FALSE)

library(ggplot2)
p_recon <- ggplot() +
  geom_ribbon(data = df_band,
    aes(x = t, ymin = density_lo, ymax = density_hi),
    fill = "#c6dbef", alpha = 0.5) +
  geom_line(data = df_lines[df_lines$line_type == "KDE ref", ],
    aes(x = t, y = density), colour = "black", linetype = "dashed", linewidth = 0.4) +
  geom_line(data = df_lines[df_lines$line_type == "KDE", ],
    aes(x = t, y = density), colour = "black", linewidth = 0.6) +
  geom_line(data = df_lines[df_lines$line_type == "MFPCA", ],
    aes(x = t, y = density), colour = "#D95F02", linewidth = 0.6) +
  geom_line(data = df_lines[df_lines$line_type == "GP mean", ],
    aes(x = t, y = density), colour = "#2166ac", linewidth = 0.6) +
  geom_label(data = ann_df, aes(label = label),
    x = Inf, y = Inf, hjust = 1.05, vjust = 1.1, size = 2.5,
    label.size = 0, fill = "white", alpha = 0.8) +
  facet_grid(variable ~ stage, scales = "free") +
  labs(x = "t", y = "Density",
    title = bquote("Three-stage reconstruction at " * theta * " = " *
      .(round(theta_star, 1)) ~ "g" ~ cm^{-2})) +
  theme_bw(base_size = 10) +
  theme(strip.text = element_text(size = 9))

ggsave("fig_reconstruction.pdf", p_recon, width = 10, height = 8)
cat("Saved fig_reconstruction.pdf\n")

```

```
## Saved fig_reconstruction.pdf
```

```
# =====
# FIGURE: MFPCA Score Series
# Main figure (fig_scores.pdf): PCs 1-8
#
# Grey dots = raw theta-level scores      xi_k(theta)
# Blue line = LOESS-smoothed diagnostics xi~_k(theta)
# Annotation = lag-1 autocorrelation of smooth (r1) + residual SD (sk)
# =====
library(ggplot2)
library(gridExtra)

pc_cols_plot <- paste0("PC", 1:M_components)

# Raw theta-level scores on the training subset
train_scores_raw_series <- as.matrix(ufpca_scores_raw_aligned[train_theta_idx, pc_cols_plot, drop = FALSE])

# Build one panel per PC (all 8; 2x4 main figure)
make_score_panel <- function(k, base_size = 10) {
  raw_k <- train_scores_raw_series[, k]
  smth_k <- ufpca_scores_smoothed[train_theta_idx, pc_cols_plot[k]]
  sig <- train_theta_vals
  lag1 <- cor(smth_k[-length(smth_k)], smth_k[-1L])
  sk <- pc_local_resid_sd_mean[k]
  df_k <- data.frame(theta = sig, raw = raw_k, smooth = smth_k)

  ggplot(df_k, aes(x = theta)) +
    geom_point(aes(y = raw), colour = "grey60", size = 1.0, alpha = 0.70) +
    geom_line(aes(y = smooth), colour = "#2166ac", linewidth = 1.0) +
    annotate("label",
      x = min(sig) + 0.03 * diff(range(sig)),
      y = max(raw_k, na.rm = TRUE),
      label = sprintf("r1 = %.3f\nmean local sk = %.3f", lag1, sk),
      hjust = 0, vjust = 1, size = 3.0,
      fill = "white", label.size = 0.2,
      label.padding = unit(0.18, "lines")) +
    labs(x = expression(theta~(g~cm^-2)),
      y = paste0("PC", k, " score")) +
    theme_bw(base_size = base_size) +
    theme(panel.grid.minor = element_blank(),
      plot.margin = margin(5, 8, 5, 5))
}

panels <- lapply(1:8, make_score_panel)

# --- Main figure: PCs 1-8 (2x4) -----
fig_scores_path <- "C:/Users/SOS/Desktop/cowork/fig_scores.pdf"
pdf(fig_scores_path, width = 16, height = 7)
do.call(grid.arrange, c(panels[1:8], list(nrow = 2, ncol = 4,
  top = "MFPCA score series: raw (grey) and LOESS-smoothed (blue), PCs 1-8")))
invisible(dev.off())
cat("Saved:", fig_scores_path, "\n")
```

Saved: C:/Users/SOS/Desktop/cowork/fig_scores.pdf

```
# =====
# FIGURE: Posterior Predictive Densities at 4 held-out theta values
# 3 rows (X, Y, Z) x 4 columns (selected theta values)
# Black solid = held-out KDE
# Red dashed = naive training-mean baseline
# Blue solid = GP posterior mean
# Blue shaded = 95% known-noise GP credible band
# Annotation = empirical density-space coverage
# =====
N_SAMP_FIG <- 500L

# Select 4 test theta values spread across the test range
test_order <- order(test_theta_vals)
sel_positions <- c(3L, 9L, 17L, 22L) # ~12%, 36%, 68%, 88% of test range
sel_idx <- test_order[sel_positions] # indices into 1:n_test_s

panel_data <- list()

for (j in seq_along(sel_idx)) {
  i <- sel_idx[j]

  # --- GP mean and variance (already computed for all held-out theta values) ---
  gp_mu <- as.numeric(pred_test_df[i, paste0("PC", seq_len(M_components))])
  gp_var <- pmax(pred_test_var_mat[i, ], 1e-10)
  gp_sd <- sqrt(gp_var)

  # --- Draw posterior samples ---
  samp_scores <- matrix(
    rnorm(N_SAMP_FIG * M_components,
          mean = rep(gp_mu, each = N_SAMP_FIG),
          sd = rep(gp_sd, each = N_SAMP_FIG)),
    nrow = N_SAMP_FIG, ncol = M_components)

  # --- Reconstruct density ensemble ---
  ens <- reconstruct_densities(samp_scores, mu_x, mu_y, mu_z,
                               phi_x, phi_y, phi_z, x_grid, y_grid, z_grid,
                               x_peak_loc = rep(pred_peak_x_test[i], N_SAMP_FIG),
                               x_peak_width = rep(pred_width_x_test[i], N_SAMP_FIG),
                               x_peak_left_width = rep(pred_left_width_x_test[i], N_SAMP_FIG),
                               x_peak_right_width = rep(pred_right_width_x_test[i], N_SAMP_FIG),
                               z_peak_loc = rep(pred_peak_z_test[i], N_SAMP_FIG),
                               z_peak_width = rep(pred_width_z_test[i], N_SAMP_FIG),
                               z_peak_left_width = rep(pred_left_width_z_test[i], N_SAMP_FIG),
                               z_peak_right_width = rep(pred_right_width_z_test[i], N_SAMP_FIG))

  # --- GP posterior mean density (single sample at gp_mu) ---
  mean_mat <- matrix(gp_mu, nrow = 1L)
  ens_mean <- reconstruct_densities(mean_mat, mu_x, mu_y, mu_z,
                                     phi_x, phi_y, phi_z, x_grid, y_grid, z_grid,
                                     x_peak_loc = pred_peak_x_test[i],
                                     x_peak_width = pred_width_x_test[i],
                                     x_peak_left_width = pred_left_width_x_test[i],
```

```

x_peak_right_width = pred_right_width_x_test[i],
z_peak_loc = pred_peak_z_test[i],
z_peak_width = pred_width_z_test[i],
z_peak_left_width = pred_left_width_z_test[i],
z_peak_right_width = pred_right_width_z_test[i])

# --- True KDE at this held-out theta ---
ref_row <- test_theta_idx[i]
true_x <- dens_mat_x[ref_row, ]
true_y <- dens_mat_y[ref_row, ]
true_z <- dens_mat_z[ref_row, ]

theta_lbl <- paste0("theta==", round(test_theta_vals[i], 1L), "~g~cm^-2") # plotmath for label_parse

for (vn in c("x", "y", "z")) {
  grid_v <- switch(vn, x = x_grid, y = y_grid, z = z_grid)
  true_v <- switch(vn, x = true_x, y = true_y, z = true_z)
  naive_v <- switch(vn, x = mean_train_dens_x, y = mean_train_dens_y, z = mean_train_dens_z)
  samp_v <- switch(vn, x = ens$x, y = ens$y, z = ens$z)
  mean_v <- switch(vn, x = as.numeric(ens_mean$x),
                    y = as.numeric(ens_mean$y),
                    z = as.numeric(ens_mean$z))

  q_lo <- apply(samp_v, 2L, quantile, 0.025)
  q_hi <- apply(samp_v, 2L, quantile, 0.975)
  cov <- mean(true_v >= q_lo & true_v <= q_hi) * 100

  panel_data[[length(panel_data) + 1L]] <- data.frame(
    grid = grid_v,
    true_dens = true_v,
    naive = naive_v,
    gp_mean = mean_v,
    q_lo = q_lo,
    q_hi = q_hi,
    variable = switch(vn,
                      x = "X: log period ratio",
                      y = "Y: log duration ratio",
                      z = "Z: log Hill spacing"),
    theta_lbl = theta_lbl,
    coverage = round(cov, 1L),
    stringsAsFactors = FALSE
  )
}
}

all_panels <- do.call(rbind, panel_data)
all_panels$variable <- factor(all_panels$variable,
  levels = c("X: log period ratio", "Y: log duration ratio", "Z: log Hill spacing"))
all_panels$theta_lbl <- factor(all_panels$theta_lbl, levels = unique(all_panels$theta_lbl))

# Per-panel coverage annotation
annot_df <- unique(all_panels[, c("variable", "theta_lbl", "coverage")])
annot_df$cov_lbl <- sprintf("Cov = %.1f%%", annot_df$coverage)

```

```

fig_ppd <- ggplot(all_panels, aes(x = grid)) +
  geom_ribbon(aes(ymin = q_lo, ymax = q_hi),
    fill = "#4393c3", alpha = 0.28) +
  geom_line(aes(y = naive), colour = "#d73027", linetype = "dashed", linewidth = 0.75) +
  geom_line(aes(y = true_dens), colour = "black", linewidth = 0.85) +
  geom_line(aes(y = gp_mean), colour = "#2166ac", linewidth = 0.85) +
  geom_text(data = annot_df,
    aes(x = -Inf, y = Inf, label = cov_lbl),
    hjust = -0.10, vjust = 1.35, size = 2.5,
    colour = "grey20", inherit.aes = FALSE) +
  facet_grid(variable ~ theta_lbl,
    scales = "free",
    labeller = labeller(theta_lbl = label_parsed)) + # plotmath: renders theta (
  labs(x = "Value",
    y = "Density",
    title = expression("Posterior predictive densities at 4 held-out " * theta * " values"),
    caption = paste("Black: held-out KDE | Red dashed: training-mean naive baseline",
      " | Blue: GP posterior mean | Shaded: 95% known-noise GP credible band")) +
  theme_bw(base_size = 9) +
  theme(strip.text.x = element_text(size = 8.0, face = "bold"),
    strip.text.y = element_text(size = 7.5, face = "bold"),
    strip.background = element_rect(fill = "grey92", colour = "grey70"),
    panel.grid.minor = element_blank(),
    plot.title = element_text(size = 9, face = "plain"),
    plot.caption = element_text(size = 6.5, colour = "grey40",
      hjust = 0.5, margin = margin(t = 4)))

fig_ppd_path <- "C:/Users/SOS/Desktop/cowork/fig_ppd.pdf"
ggsave(fig_ppd_path, fig_ppd, width = 11, height = 7, device = "pdf")
cat("Saved:", fig_ppd_path, "\n")

```

```
## Saved: C:/Users/SOS/Desktop/cowork/fig_ppd.pdf
```

```

# =====
# H: Variance diagnostic
#   For each test  $\theta_k$ , compare the learned latent GP uncertainty
#    $v_k(\theta_k)$  to the full restored residual variance  $\sigma^2_{\text{resid},k}$ .
# =====
cat("=== H: Variance diagnostic ===\n")

```

```
## === H: Variance diagnostic ===
```

```
cat("Latent variance share  $v_k / (v_k + \sigma^2_{\text{resid},k})$  across K=30 components\n\n")
```

```
## Latent variance share  $v_k / (v_k + \sigma^2_{\text{resid},k})$  across K=30 components
```

```

var_share <- pred_test_latent_var_mat / pmax(pred_test_latent_var_mat + pred_test_obs_var_mat, 1e-10)
var_ratio <- pred_test_latent_var_mat / pmax(pred_test_obs_var_mat, 1e-10)

cat(sprintf("Mean latent share : % .3f\n", mean(var_share)))

```

```
## Mean latent share : 0.008
```

```
cat(sprintf("Median share      : %.3f\n", median(var_share)))
```

```
## Median share      : 0.008
```

```
cat(sprintf("Share range      : [%.3f, %.3f]\n", min(var_share), max(var_share)))
```

```
## Share range      : [0.005, 0.016]
```

```
cat(sprintf("Mean v/sigma2_resid : %.3f\n", mean(var_ratio)))
```

```
## Mean v/sigma2_resid : 0.008
```

```
cat(sprintf("Median v/sigma2_resid: %.3f\n", median(var_ratio)))
```

```
## Median v/sigma2_resid: 0.008
```